



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO
INGENIERÍA EN INFORMÁTICA

Plataforma multisensorial con visión por computador y realidad aumentada para la educación infantil

Luminia

Autor

Nuria Manzano Mata

Director

Jose Antonio Morente Molinera



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, 11 de Noviembre de 2025

Plataforma multisensorial con visión por computador y realidad aumentada para la educación infantil

Luminia

Autor

Nuria Manzano Mata

Director

Jose Antonio Morente Molinera

Plataforma multisensorial con visión por computador y realidad aumentada para la educación infantil: Luminia

Nuria Manzano Mata

Palabras clave: Computación Ubicua, Realidad Aumentada, Gamificación, Aprendizaje infantil, Interacción implícita, Conciencia de contexto, Procesamiento del lenguaje natural.

Resumen

Cada vez somos más conscientes de que las tecnologías se integran de forma natural en nuestra vida cotidiana. Es una realidad innegable que, bien utilizada, puede ofrecernos grandes oportunidades. Sin embargo, como todo avance, su valor depende de la intencionalidad y del uso que se le otorgue. Surge entonces una pregunta, ¿cómo podemos aprovechar su potencial dentro de un entorno seguro y controlado para los más pequeños?

Este Trabajo de Fin de Grado presenta el desarrollo de una plataforma educativa inmersiva e interactiva, orientada a niños de entre 4 y 6 años, cuyo objetivo es potenciar el aprendizaje de conceptos básicos (letras, números, animales) y, al mismo tiempo, favorecer el desarrollo de habilidades cognitivas y motoras. A través de tecnologías como la realidad aumentada, la interacción implícita, la conciencia de contexto y la gamificación, la propuesta ofrece un entorno de aprendizaje lúdico, seguro y motivador.

La plataforma se distingue por integrar mundos temáticos con desafíos progresivos que combinan juego y enseñanza, apoyados por un asistente virtual que guía la experiencia, estimula la curiosidad y refuerza la motivación mediante recompensas y retroalimentación continua. Además, incorpora un sistema de gestión dinámica de contenidos, garantizando un entorno personalizado, seguro y flexible para cada niño.

El resultado es una herramienta educativa innovadora que demuestra cómo la tecnología inmersiva puede trascender la experimentación académica para convertirse en un recurso eficaz, atractivo y con un impacto real en el aprendizaje infantil.

Multisensory Platform with Computer Vision and Augmented Reality for Early Childhood Education: Luminia

Nuria Manzano Mata

Keywords: Ubiquitous Computing, Augmented Reality, Gamification, Early Childhood Learning, Implicit Interaction, Context Awareness, Natural Language Processing.

Abstract

We are increasingly aware that technology is becoming a natural part of our daily lives. It is an undeniable reality that, when used appropriately, can offer great opportunities. However, as with any advancement, its value depends on the intention behind it and the way it is used. This raises an important question: how can we harness its potential within a safe and controlled environment for young children?

This Undergraduate Final Project presents the development of an immersive and interactive educational platform designed for children aged 4 to 6 years, with the goal of enhancing the learning of basic concepts (letters, numbers, animals) while simultaneously fostering cognitive and motor skill development. Through technologies such as augmented reality, implicit interaction, context awareness, and gamification, the proposal offers a playful, safe, and engaging learning environment.

The platform stands out for its integration of thematic worlds with progressive challenges that combine play and education, supported by a virtual assistant that guides the experience, stimulates curiosity, and reinforces motivation through rewards and continuous feedback. In addition, it includes a dynamic content management system, ensuring a personalized, secure, and flexible environment for each child.

The result is an innovative educational tool that demonstrates how immersive technology can transcend academic experimentation to become an effective, appealing, and impactful resource for early childhood learning.

Yo, **Nuria Manzano Mata**, alumna de la titulación Ingeniería Informática y Administración y Dirección de Empresas de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 77993355X, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Nuria Manzano Mata

Granada a 11 de Noviembre de 2025 .

D. **Jose Antonio Morente Molinera**, profesor del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado *Plataforma multisensorial con visión por computador y realidad aumentada para la educación infantil, Luminia un mundo mágico*, ha sido realizado bajo su supervisión por **Nuria Manzano Mata**, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firme el presente informe en Granada a 11 de Noviembre de 2025 .

El tutor:

Jose Antonio Morente Molinera

Agradecimientos

A lo largo de mi vida he aprendido que el esfuerzo, el cariño y la dedicación son la base para alcanzar cualquier meta. Por suerte, el ejemplo lo he tenido siempre en casa, mi padre y mis abuelos, personas trabajadoras y honestas que han perseguido sus sueños hasta hacerlos realidad. A ellos van dirigidos mis más sinceros agradecimientos. Han sido, sin duda, mis mayores referentes, tanto en lo personal como en lo profesional. Me han enseñado a superarme, a no rendirme y a no conformarme nunca.

A mi abuela, mi gran figura materna, ha sido y será siempre mi mayor ejemplo de amor, ternura y dedicación. Gracias a ella he aprendido a completarme como persona y a valorar lo verdaderamente importante.

A mi padre, mi pilar fundamental, ha sido siempre mi apoyo incondicional y mi fuente de seguridad. Su esfuerzo y sacrificio me han permitido llegar hasta donde estoy hoy. Sin él, nada de esto habría sido posible.

También quiero agradecer a mi pareja, otro pilar esencial en mi vida. Su confianza, apoyo y motivación me han impulsado a seguir adelante incluso cuando las cosas no salían como esperaba. Gracias a su compañía y enseñanzas, he crecido, madurado y aprendido tanto a nivel personal como académico y profesional.

Por último, expresar mi agradecimiento a mi tutor, José Antonio Morente Molinera, por su guía, dedicación y apoyo a lo largo de todo este proceso. Su orientación ha sido imprescindible para culminar con éxito este proyecto.

Índice general

1. Introducción	1
1.1. Fundamentación pedagógica y psicológica	2
1.1.1. Rango de edad recomendado	3
1.2. Motivación	4
1.2.1. Motivación personal	4
1.2.2. Motivación profesional	4
1.2.3. Motivación social	5
1.3. Objetivos	5
2. Marco teórico	7
2.1. ¿Qué es la Computación Ubicua?	8
2.2. Sistemas conscientes del contexto (<i>context-aware</i>)	8
2.3. Interacción implícita hombre-máquina	9
2.3.1. Evolución histórica de los paradigmas de interacción .	10
2.4. Tecnologías para una interacción implícita y consciente del contexto	11
2.4.1. Reconocimiento facial	11
2.4.2. Reconocimiento de voz y procesamiento del lenguaje natural (NLP)	12
2.4.3. Realidad aumentada (AR)	13
2.5. Aplicaciones educativas inmersivas: panorama, brechas y me- joras	15
2.5.1. Panorama y tendencias actuales	16
2.5.2. Brechas críticas y posibles mejoras	16
2.5.3. Ética, regulación y buenas prácticas	17
3. Estado del Arte	19
3.1. Aplicaciones educativas con gamificación y narrativa	19
3.2. Aplicaciones con realidad aumentada	19
3.3. Juegos educativos con mundos temáticos	20
3.4. Comparativa	20
3.5. Síntesis	20

4. Planificación del proyecto	23
4.1. Metodología de desarrollo	23
4.1.1. Principios adoptados	23
4.1.2. Etapas de desarrollo	24
4.2. Cronograma del proyecto	24
4.3. Presupuesto	26
4.4. Gestión de riesgos	28
5. Ingeniería de requisitos	29
5.1. Descripción de los implicados	29
5.2. Especificación de requisitos	30
5.2.1. Requisitos funcionales	30
5.2.2. Requisitos no funcionales	31
5.2.3. Requisitos de información	31
5.2.4. Requisitos del sistema	32
5.2.5. Requisitos pedagógicos	33
5.2.6. Requisitos de denominaciones	33
5.3. Casos de uso	34
5.4. Descripción detallada de casos de uso	35
5.5. Diagrama de paquetes	39
5.6. Diagramas de actividad	41
6. Análisis de requisitos	47
6.1. Modelo conceptual	47
6.2. Diagramas secuencia	47
6.3. Historias de usuario	50
6.3.1. Product backlog	50
6.3.2. Tarjetas de Historias de Usuario	50
7. Diseño	57
7.1. Diseño de la interacción usuario-sistema	57
7.2. Diseño de la experiencia inmersiva y la interfaz educativa	58
7.3. Diseño de los módulos tecnológicos	61
7.4. Diseño de la base de datos	65
8. Implementación	67
8.1. Entorno y herramientas utilizadas	67
8.2. Integración de tecnologías	69
8.3. Desarrollo de los módulos principales	70
8.3.1. Módulo principal <code>main.py</code>	71
8.3.2. Módulo de gestión de datos <code>data_manager.py</code>	72
8.3.3. Módulos: <code>voice_recognition.py</code> y <code>audio_manager.py</code>	74
8.3.4. Módulo <code>ui_renderer</code>	76
8.3.5. Módulo <code>gestorJuegosAR</code>	77

8.3.6. Clases de soporte para la realidad aumentada <code>myVideo</code> , <code>modeloGLTF</code> y <code>escenaPYGFX</code>	78
9. Pruebas	83
9.1. Estrategia de pruebas	83
9.2. Pruebas funcionales	84
9.3. Pruebas unitarias	88
9.4. Pruebas de integración	91
9.5. Pruebas de usabilidad	93
9.6. Pruebas de rendimiento	94
10. Conclusiones y trabajos futuros	99
10.1. Cumplimiento de los objetivos	99
10.2. Conclusiones generales	101
10.3. Limitaciones	101
10.4. Propuestas de mejora	102
10.5. Líneas de trabajo futuras	102
11. Glosario de términos	105
Bibliografía	115
A. Manual de Usuario	117
A.1. Requisitos	117
A.2. Instalación y ejecución	117
A.3. Descripción	119
A.3.1. Fase: Inicio	119
A.3.2. Fase: Reconocimiento Facial	119
A.3.3. Fase: Registro de usuario	120
A.3.4. Fase: Menú Principal	122
A.3.5. Fase: Perfil	123
A.3.6. Fase: Progreso	124
A.3.7. Fase: Disfraces	124
A.3.8. Fase: Mundos de Juego	125
A.3.9. Fase: Jugando	127
A.3.10. Mensajes de error comunes	130

Índice de figuras

4.1. Diagrama de Gantt.	26
5.1. Diagrama general de casos de uso del sistema Luminia.	34
5.2. Diagrama de paquetes del sistema Luminia.	40
5.3. Diagrama de actividad CU-1 y CU-2: Reconocimiento facial.	41
5.4. Diagrama de actividad CU-6: Visualización RA.	42
5.5. Diagrama de actividad CU-8: Juegos y desafíos educativos.	43
5.6. Diagrama de actividad CU-13 y CU-14: Comando de salida.	44
6.1. Diagrama de secuencia del registro de sesión.	48
6.2. Diagrama de secuencia del inicio de sesión.	48
6.3. Diagrama de secuencia de la comunicación por voz.	49
6.4. Diagrama de secuencia de la visualización de la Realidad Au- mentada.	49
7.1. Boceto de la interfaz educativa de Luminia durante la fase de menú principal.	59
7.2. Boceto de la interfaz educativa de Luminia durante la fase de mundo elegido.	60
7.3. Boceto de la interfaz educativa de Luminia durante la fase de juego.	60
7.4. Diagrama de estados principal del sistema <i>Luminia</i>	61
7.5. Diagrama de clases del sistema Luminia.	64
7.6. Diagrama de componentes y dependencias del sistema <i>Luminia</i>	66
8.1. Muestra de un ejemplo de colección en MogoDB Compass.	69
8.2. Tablero de juego.	70
9.1. Primera evidencia de reconocimiento de voz en ejecución.	84
9.2. Segunda evidencia de reconocimiento de voz en ejecución.	84
9.3. Patrón de Calibrado.	85
9.4. Visualización correcta del modelo 3D de la mascota Tina.	86
9.5. Visualización correcta de los modelos 3D en el menú principal.	86

9.6. Visualización correcta de los modelos 3D en el mundo de las letras.	87
9.7. Visualización correcta de los modelos 3D en el juego de memoria del mundo de las letras.	87
9.8. Ejecución de las pruebas unitarias en los módulos principales.	90
9.9. Pruebas de integración entre módulos principales de Luminia.	93
9.10. Pruebas de rendimiento entre módulos principales de Luminia.	97
A.1. Pantalla de inicio.	119
A.2. Pantalla de reconocimiento facial.	120
A.3. Pantalla de reconocimiento facial con detección del rostro. . .	120
A.4. Fase de registro: introducción del nombre.	121
A.5. Fase de registro: nombre existe.	121
A.6. Fase de registro: introducción del idioma.	122
A.7. Menú principal: navegación por voz entre mundos y opciones.	123
A.8. Gestión del perfil.	123
A.9. Ejemplo actualización nombre usuario.	124
A.10. Visualización del progreso de un usuario.	124
A.11. Pantalla de disfraces.	125
A.12. Muestra de la compra y equipamiento de un disfraz.	125
A.13. Ejemplo del mundo de las letras con comandos de minijuego.	126
A.14. Ejemplo del mundo de los animales con comandos de minijuego.	126
A.15. Ejemplo del mundo de las frutas y verduras con comandos de minijuego.	126
A.16. Ejemplo de interacción durante el minijuego adivina del mundo de las letras.	127
A.17. Ejemplo de interacción durante el minijuego clasificar del mundo de los animales.	128
A.18. Ejemplo de respuesta correcta durante un minijuego.	128
A.19. Ejemplo de interacción durante el minijuego adivina del mundo de las frutas y verduras.	129
A.20. Ejemplo de interacción durante el minijuego adivina del mundo de los números.	129
A.21. Ejemplo de interacción durante el minijuego contar del mundo de final.	130
A.22. Ejemplo de mensaje de error de reconocimiento de voz. . . .	131
A.23. Ejemplo de mensaje de error mientras se está jugando. . . .	131

Índice de cuadros

3.1. Comparativa de aplicaciones educativas existentes.	21
4.1. Reparto de tareas y horas dedicadas por iteración	25
5.1. Caso de uso CU-1: Registrarse / Reconocimiento facial	35
5.2. Caso de uso CU-2: Iniciar sesión / Reconocimiento facial . . .	36
5.3. Caso de uso CU-3: Comunicación natural por voz	36
5.4. Caso de uso CU-4: Guía interactiva y narración contextual . .	36
5.5. Caso de uso CU-5: Evaluación y retroalimentación	37
5.6. Caso de uso CU-6: Visualización en realidad aumentada . . .	37
5.7. Caso de uso CU-7: Selección de mundo educativo	37
5.8. Caso de uso CU-8: Juegos y desafíos educativos	38
5.9. Caso de uso CU-9: Personalización de Tina y recompensas . .	38
5.10. Caso de uso CU-10: Consulta y gestión del progreso	38
5.11. Caso de uso CU-11: Edición de datos del perfil	38
5.12. Caso de uso CU-12: Guardado automático y cierre seguro . .	39
5.13. Caso de uso CU-13: Comando de salida rápida	39
5.14. Caso de uso CU-14: Comando pantalla anterior	39
5.15. Matriz de trazabilidad de requisitos y casos de uso	45
6.1. Product backlog del sistema Luminia	50
6.2. HU1 – Identificación del jugador	51
6.3. HU2 – Comunicación natural por voz	51
6.4. HU3 – Guía interactiva y narraciones	52
6.5. HU4 – Visualización en realidad aumentada	52
6.6. HU5 – Selección del mundo educativo	53
6.7. HU6 – Juegos y desafíos educativos	53
6.8. HU7 – Evaluación y retroalimentación	54
6.9. HU8 – Recompensas y personalización	54
6.10. HU9 – Consulta del progreso	55
6.11. HU10 – Edición del perfil de usuario	55
6.12. HU11 – Guardado automático y cierre seguro	56
6.13. HU12 – Salida rápida por comando de voz	56

Capítulo 1

Introducción

En los últimos años, el uso de herramientas digitales en la educación infantil se ha convertido en una necesidad más que en una opción. La creciente exposición de los niños y niñas a la tecnología desde edades tempranas exige replantear los métodos de enseñanza tradicionales y adaptarlos a entornos más interactivos, visuales y significativos. Diversas revisiones sistemáticas indican que el tiempo excesivo frente a pantallas en los primeros años está asociado con peores resultados en desarrollo cognitivo, lenguaje y autorregulación [33, 52]. Por ello, organismos como la World Health Organization (OMS) y la American Academy of Pediatrics (AAP) recomiendan limitar el uso de pantallas o posponerlo en niños menores de 5 años [17]. Sin embargo, estas recomendaciones se refieren principalmente a la exposición pasiva a pantallas, como la televisión o el consumo de vídeos sin interacción. En contraste, las aplicaciones educativas interactivas y basadas en tecnologías inmersivas representan una alternativa segura y valiosa cuando su uso es guiado, de alta calidad y complementario al juego físico y la interacción social. Estas herramientas promueven un aprendizaje activo y significativo, favoreciendo la comprensión profunda de conceptos básicos mediante experiencias interactivas donde los niños manipulan objetos, exploran relaciones y resuelven desafíos en un entorno controlado. Al ser contenidos guiados y adaptables, estas plataformas combinan el aprendizaje con la exploración, refuerzan la motricidad fina y la coordinación visomotora, y aseguran que la tecnología se utilice con un propósito educativo claro y bajo supervisión adulta.

Luminia es una aplicación educativa diseñada para niños y niñas de entre 4 y 6 años, cuyo objetivo principal es favorecer el aprendizaje de conceptos básicos mientras se desarrollan, de forma paralela, habilidades cognitivas, a través de actividades que implican razonamiento, atención, memoria y clasificación; y motoras que se potencian mediante la interacción física y digital del entorno. Para ello, la aplicación integra tecnologías inmersivas como la realidad aumentada, la interacción implícita y la conciencia de contexto,

ofreciendo una experiencia de aprendizaje dinámica, progresiva y guiada.

El personaje central de la aplicación es *Tina*, una asistente virtual y acompaña a los pequeños en su recorrido por el mundo de Luminia. La narrativa se construye en torno a una misión, ayudar a Tina a devolver la luz a su mundo. Para lograrlo, los usuarios deben superar diferentes niveles ambientados en mundos temáticos (inspirados en propuestas como *Super Mario Galaxy*), en los que se plantean diversos desafíos. Cada mundo (por ejemplo, los números o los animales) incluye tres retos que deben completarse de manera progresiva. Al superar cada desafío, los niños reciben "destellos", es decir, una cantidad de 1 a 3 estrellas, lo que permite medir el nivel de desempeño y fomentar la motivación. Una vez conseguido 3 destellos en los retos de un mundo, se desbloquea el acceso a nuevos mundos con un grado de complejidad mayor. El recorrido culmina en un **mundo final**, en el que se presentan desafíos que combinan los aprendizajes adquiridos previamente, consolidando los conocimientos de forma integral.

Además, la aplicación incluye elementos de gamificación que refuerzan la motivación. Por ejemplo, los niños pueden obtener lumios en función de los destellos conseguidos y utilizarlos para desbloquear disfraces para Tina. También se plantea un reto adicional: quienes logren completar todos los desafíos con la máxima puntuación, recibirán un **premio único y exclusivo** dentro del mundo de Luminia, lo que incentiva la superación personal y el perfeccionamiento de resultados.

En definitiva, *Luminia* persigue que los niños aprendan mientras se divierten, aprovechando el potencial de las tecnologías de vanguardia para ofrecer experiencias inmersivas, interactivas y con un alto valor educativo. La aplicación no solo contribuye al desarrollo de conocimientos teóricos y prácticos, sino que también fomenta habilidades transversales como la comunicación, la escucha, la organización, la coordinación y la retroalimentación continua.

1.1. Fundamentación pedagógica y psicológica

El diseño de *Luminia* se apoya en fundamentos pedagógicos y psicológicos ampliamente validados que demuestran la relevancia del aprendizaje lúdico mediado por tecnologías digitales en la infancia temprana. Según Piaget [60], durante la etapa preoperacional (aproximadamente de los 2 a los 7 años), los niños desarrollan habilidades simbólicas, la capacidad de clasificar y la comprensión incipiente de relaciones lógicas simples. Esto convierte al rango de edad de entre 4 y 6 años en un periodo especialmente propicio para actividades que combinen juego, símbolos, imágenes y experiencias interactivas.

Investigaciones recientes en el ámbito de la psicología educativa muestran que el uso de tecnologías como la realidad aumentada puede mejorar la

atención, la memoria de trabajo y la motivación intrínseca de los niños [8, 64]. Asimismo, se ha comprobado que la gamificación y el aprendizaje basado en retos favorecen el desarrollo de la autonomía y la autorregulación en edades tempranas [32].

En la misma línea, diversos estudios sostienen que las experiencias inmersivas con RA facilitan el aprendizaje significativo en áreas básicas como el reconocimiento de formas, colores y números, además de fomentar la coordinación visomotora [66, 79]. Este efecto se intensifica cuando las actividades se presentan en formato de juego estructurado, ya que el componente narrativo y motivacional aumenta la adherencia y la implicación de los niños [61].

Desde un punto de vista normativo, la **Ley Orgánica 3/2020, de 29 de diciembre**, que modifica la Ley Orgánica de Educación (LOMLOE), establece en su artículo 111 bis que las tecnologías de la información y la comunicación constituyen una herramienta esencial para la enseñanza y el aprendizaje, promoviendo su integración en las diferentes etapas educativas como medio de innovación pedagógica [2]. Esto refuerza la legitimidad del uso de plataformas como *Luminia*, que ponen la tecnología al servicio de un aprendizaje más accesible y motivador.

1.1.1. Rango de edad recomendado

Aunque las mecánicas de *Luminia* podrían ser adaptadas a un rango amplio de edad, la evidencia psicológica y pedagógica sugiere que la franja óptima se encuentra entre los **4 y 6 años**. En este periodo:

- Los niños ya han desarrollado las habilidades perceptivas necesarias para identificar colores, formas y sonidos, pero aún requieren refuerzo sistemático para consolidarlas.
- Inician el aprendizaje formal de los números y comienzan a comprender nociones de cantidad, orden y clasificación.
- Muestran gran receptividad al aprendizaje lúdico y simbólico, lo cual hace que la narrativa del personaje *Tina*, sea altamente motivadora.
- Poseen un nivel de coordinación motora fina y gruesa adecuado para interactuar con dispositivos de manera sencilla y efectiva.

De acuerdo con investigaciones en desarrollo infantil [10, 14], a partir de los 7 años los niños entran progresivamente en la etapa de operaciones concretas, donde su capacidad cognitiva se orienta hacia tareas más abstractas y menos centradas en el juego simbólico. Esto implica que la propuesta de *Luminia* tendría menor impacto motivacional y pedagógico en edades superiores a 6 años, consolidando así la franja de **4 a 6 años** como la más apropiada para su implementación.

En síntesis, *Luminia* se fundamenta tanto en teorías clásicas del desarrollo cognitivo como en estudios contemporáneos sobre aprendizaje mediado por tecnologías inmersivas, y se ajusta al marco normativo español en materia de innovación educativa. Esto asegura que la aplicación no solo sea atractiva y motivadora, sino también pedagógicamente sólida y socialmente relevante.

1.2. Motivación

La realización de este proyecto responde a una motivación múltiple que combina aspectos personales, profesionales y sociales. Por un lado, nace de una vocación ligada a la enseñanza y al deseo de aportar valor educativo real a la sociedad; por otro, constituye una oportunidad para consolidar y ampliar competencias técnicas en el ámbito de las tecnologías emergentes. Todo ello converge en un proyecto que busca trascender lo meramente académico para ofrecer un producto con impacto positivo en la educación infantil.

1.2.1. Motivación personal

El origen de este trabajo se encuentra en la experiencia previa con la aplicación Kids&Veggies (proyecto realizado para la asignatura de Computación Ubícua e Inteligencia Ambiental) que mostró el potencial de la interacción implícita y la conciencia de contexto en entornos educativos. Esta idea conecta directamente con una de mis mayores pasiones: la enseñanza. Como docente de programación para jóvenes de entre 5 y 18 años, he podido comprobar cómo una experiencia de aprendizaje bien diseñada puede estimular la curiosidad, fomentar la autonomía y potenciar el desarrollo de habilidades cognitivas. En este sentido, este proyecto representa una extensión natural de esa vocación, la oportunidad de transformar la tecnología en un medio capaz de facilitar aprendizajes significativos desde edades tempranas.

1.2.2. Motivación profesional

Desde el punto de vista profesional, este TFG supone un desafío que permitirá afianzar conocimientos técnicos avanzados y aplicarlos en un contexto real. Este proyecto constituye una oportunidad para profundizar en la integración de tecnologías como el reconocimiento de voz, la realidad aumentada y el uso de bibliotecas de visión por computador, además de abordar un ciclo de desarrollo de software completo, desde la arquitectura hasta la implementación y validación de funcionalidades.

Asimismo, mi interés por la Computación Ubícua y la Inteligencia Ambiental (despertado en la asignatura homónima y reforzado con la Matrícula de Honor obtenida) encuentra en este trabajo un marco ideal para evolucionar.

nar de un prototipo experimental a una solución tecnológica madura y de aplicación práctica.

1.2.3. Motivación social

Más allá de la dimensión personal y profesional, este proyecto se fundamenta en una motivación social, contribuir a mejorar la educación infantil mediante el uso de tecnologías inmersivas. En un mundo cada vez más digital, los niños están expuestos desde edades tempranas a entornos tecnológicos; sin embargo, no siempre lo hacen en contextos adecuados o con un propósito formativo claro. **Luminia** busca ofrecer una alternativa constructiva, donde la realidad aumentada, la gamificación y la interacción adaptativa se ponen al servicio del aprendizaje. El valor social del proyecto radica en:

- Fomentar el aprendizaje lúdico y significativo en edades clave para el desarrollo cognitivo y emocional.
- Potenciar competencias transversales como la comunicación, la colaboración, la organización y la retroalimentación continua.
- Promover un uso responsable y guiado de la tecnología, garantizando la seguridad del entorno digital.

En definitiva, este trabajo busca demostrar que es posible diseñar soluciones tecnológicas que no solo aporten un valor pedagógico, sino que también contribuyan a un objetivo social más amplio, aprovechando el potencial de la innovación digital para crear entornos de aprendizaje motivadores y con impacto real en la sociedad.

1.3. Objetivos

La culminación de este proyecto se define a través de un objetivo general y una serie de objetivos específicos. El propósito es desarrollar un entorno educativo inmersivo, interactivo y adaptable, dirigido a niños de entre 4 y 6 años, que combine realidad aumentada, interacción implícita, conciencia de contexto y gamificación para ofrecer una experiencia didáctica personalizada y motivadora.

Objetivo general

El objetivo principal es desarrollar una plataforma educativa completa que fomente el aprendizaje significativo y la exploración activa, aprovechando tecnologías inmersivas y asegurando un alto impacto pedagógico. La solución debe demostrar viabilidad técnica, potenciar habilidades cognitivas y

motoras, refuerzar la motivación infantil y garantizar una experiencia segura y adaptada a la psicología infantil.

Objetivos específicos

Para alcanzar el objetivo general, se han definido los siguientes objetivos específicos:

- **Potenciar la atención y motivación infantil:** mediante estímulos visuales y auditivos interactivos que fomenten la participación activa.
- **Facilitar el aprendizaje significativo:** a través de la exploración, la gamificación y la resolución de desafíos progresivos.
- **Modularización del sistema:** modularización de la arquitectura del software para garantizar escalabilidad y mantenibilidad, permitiendo la incorporación de nuevas funcionalidades educativas futuras.
- **Diseño e implementación de base de datos:** gestionar perfiles de usuario y progreso de aprendizaje de manera segura y eficiente.
- **Desarrollo de mundos temáticos educativos:** implementar módulos centrados en áreas principales para la educación infantil (letras, números, animales, colores, etc.) con desafíos progresivos.
- **Integración de gamificación y recompensas:** incorporar recompensas visuales para reforzar la motivación y la superación personal.
- **Diseño de interfaz de usuario (UI) y experiencia de juego (UX):** diseño intuitivo, simple y atractivo, con navegación sencilla, retroalimentación adaptativa y elementos inmersivos.
- **Implementación de funcionalidades avanzadas:** incluir reconocimiento de objetos mediante visión por computador y otras herramientas que aumenten la inmersión y la interacción.
- **Garantizar seguridad y privacidad:** proteger los datos de los usuarios infantiles y asegurar un entorno digital controlado.

Capítulo 2

Marco teórico

Este capítulo presenta el marco teórico del campo que enmarca el presente proyecto, abordando las tecnologías y conceptos fundamentales que lo definen. El análisis se estructura en tres secciones.

En primer lugar, se traza un recorrido histórico por la evolución de la interacción hombre-máquina (HMI), desde los paradigmas explícitos y reactivos hasta la visión de la computación ubicua y la interacción implícita. Esta perspectiva diacrónica es esencial para comprender cómo el enfoque tecnológico se ha desplazado de dispositivos centralizados a entornos inteligentes y perceptivos, capaces de anticipar las necesidades del usuario.

A continuación, el análisis se profundiza en las tecnologías específicas que habilitan dicha evolución. Se examinará de manera individual el reconocimiento facial, el reconocimiento de voz, el procesamiento del lenguaje natural (NLP) y la realidad aumentada (AR). Cada apartado ofrecerá una visión detallada de la trayectoria histórica de la tecnología, sus componentes técnicos, las opciones disponibles en la actualidad y las ventajas y desventajas inherentes a su implementación. Este estudio permitirá identificar las herramientas más adecuadas para el desarrollo del proyecto y justificar las decisiones de diseño.

Finalmente, el capítulo abordará de forma reducida el panorama actual de las aplicaciones educativas inmersivas, analizando las soluciones existentes en el mercado, sus logros y sus limitaciones. Este análisis se complementará con una reflexión crítica sobre los desafíos éticos y regulatorios que surgen al trabajar con tecnologías que recopilan datos contextuales de usuarios, especialmente en el ámbito de la infancia. Al examinar el panóptico digital y las buenas prácticas en el diseño, se establecerá un marco riguroso que permitirá evaluar de manera objetiva el aporte y la relevancia del presente trabajo en el contexto académico y social.

2.1. ¿Qué es la Computación Ubicua?

La computación ubicua, o *ubicomputing*, es un paradigma que tiene como objetivo principal la integración de la tecnología en el entorno cotidiano de las personas de una forma tan fluida que esta se vuelve prácticamente invisible. A diferencia de la computación tradicional, donde el usuario tiene que buscar y usar activamente un dispositivo, la *ubicomputing* propone que los ordenadores estén presentes en el entorno, en los objetos del día a día, y que la interacción con ellos se produzca de forma natural y casi sin esfuerzo.

Esta visión fue articulada por primera vez por el científico informático **Mark Weiser** en 1991, en su seminal artículo "The Computer for the 21st Century" [77]. Weiser, trabajando en Xerox PARC, predijo un futuro donde la tecnología se mezclara en el tejido de la vida, presente en todo momento y lugar. Él imaginaba un mundo en el que los dispositivos de computación (llamados *tabs*, *pads* y *boards*) serían omnipresentes pero estarían en segundo plano, facilitando las tareas humanas sin exigir una atención constante. Su idea era que la tecnología debía adaptarse al ser humano, no al revés.

A lo largo de los años, el concepto de la *ubicomputing* se ha ramificado en campos afines que han contribuido a su desarrollo. En los años 2000, la **computación pervasiva** [68] se centró en la creación de redes de dispositivos móviles y sensores para que la información estuviera siempre disponible. Por su parte, la **interacción humano-computadora (HCI)** se ha centrado en hacer la tecnología más intuitiva, mientras que los **sistemas ciberfísicos** (CPS) han abordado la integración de la computación con procesos físicos, como los que se encuentran en la robótica o la domótica.

La trayectoria de la *ubicomputing* ha sido notable. En la actualidad, su visión se ha materializado en gran medida a través de la proliferación de **dispositivos inteligentes** como smartphones, relojes, asistentes de voz (Alexa, Google Home) y sensores integrados en los hogares. Estos dispositivos trabajan en red y recogen información de su entorno, lo que ha dado lugar al **internet de las cosas (IoT)**.

2.2. Sistemas conscientes del contexto (*context-aware*)

La visión de la computación ubicua, donde la tecnología se integra de manera invisible en el entorno, solo es posible si los dispositivos son capaces de entender y reaccionar a la situación del usuario. Aquí es donde entran los **sistemas conscientes del contexto** o *context-aware*. Un sistema de este tipo utiliza información de su entorno para adaptar su comportamiento sin que el usuario tenga que dar órdenes explícitas.

Una de las definiciones más citadas en la literatura académica fue propuesta por Dey y Abowd en 2001: el contexto es "cualquier información

que puede ser usada para caracterizar la situación de una entidad (una persona, un lugar u objeto) que es considerada relevante para la interacción entre esa entidad y una aplicación” [23]. Esta definición no solo se limita a la ubicación, sino que abarca el estado del usuario (¿está en movimiento?, ¿está hablando?, ¿su mirada se dirige a un objeto?), la actividad que está realizando, el momento del día y las condiciones ambientales.

Los sistemas *context-aware* han demostrado ser especialmente prometedores en el ámbito educativo, ofreciendo un potencial significativo para mejorar la personalización y la eficacia del aprendizaje. Numerosos estudios y revisiones sistemáticas respaldan esta afirmación, destacando cómo la adaptación al contexto del usuario puede aumentar la motivación, el compromiso y el rendimiento académico. Por ejemplo, investigaciones han demostrado que los entornos de aprendizaje ubicuo y consciente del contexto captan de manera más efectiva la atención de los estudiantes, resultando en un mayor compromiso [16, 71]. Del mismo modo, la capacidad de estos sistemas para ofrecer recursos y actividades personalizadas, adaptadas a la situación del alumno, ha mostrado efectos positivos en la eficiencia y los resultados del aprendizaje [43].

Históricamente, los primeros sistemas *context-aware* se centraron en la **ubicación** del usuario. Proyectos iniciales en los años 90 utilizaban tecnologías como GPS y RFID para ofrecer información contextual. Con el tiempo, el concepto se expandió para incluir otros tipos de información, dando lugar a un modelo de *context-aware* más rico.

Las arquitecturas típicas de estos sistemas siguen un flujo bien definido:

1. **Adquisición del contexto:** se utilizan sensores (acelerómetros, micrófonos, cámaras, GPS) para capturar datos del entorno.
2. **Modelado del contexto:** los datos brutos se procesan y se estructuran en un modelo coherente.
3. **Interpretación del contexto:** se aplican algoritmos (a menudo de *machine learning*) para inferir el estado del usuario o del entorno (por ejemplo, “el niño está prestando atención” o “la luz es escasa”).
4. **Adaptación de la aplicación:** la aplicación reacciona a la información interpretada, ofreciendo una experiencia personalizada.

2.3. Interacción implícita hombre-máquina

La evolución de la interacción hombre-máquina (HMI) se puede entender como una progresión desde la interacción explícita hacia paradigmas cada vez más implícitos. Para ello, es crucial diferenciar ambos conceptos.

- **Interacción explícita:** es el modelo de interacción tradicional, donde el usuario debe proporcionar comandos directos y deliberados para que

el sistema realice una acción [39]. Un ejemplo clásico es la interacción a través de una interfaz de línea de comandos o el clic de un ratón sobre un icono en una interfaz gráfica de usuario (GUI). En este paradigma, el usuario es el iniciador consciente de la interacción, y la máquina responde de forma reactiva.

- **Interacción implícita:** en contraste, la interacción implícita se produce sin la instrucción o el conocimiento explícito del usuario [39]. El sistema infiere las intenciones o necesidades del usuario al monitorear su contexto, comportamiento y entorno a través de sensores y algoritmos. Este tipo de interacción busca un flujo de uso más natural y fluido que no interrumpa la tarea principal del usuario, permitiendo que la tecnología se adapte de manera autónoma.

2.3.1. Evolución histórica de los paradigmas de interacción

La historia de la HMI refleja un movimiento continuo hacia una mayor naturalidad y conciencia del contexto:

1. **Era de los controles físicos y la interacción por línea de comandos (Pre-1980):** los primeros sistemas de interacción se basaban en controles físicos, tarjetas perforadas y, posteriormente, consolas de texto. La interacción era puramente explícita y requería un conocimiento técnico profundo por parte del usuario. No había lugar para la inferencia, solo para comandos precisos y tangibles [55].
2. **La era de las interfaces gráficas de usuario (GUI) (1980s-2000s):** la popularización del ratón y la GUI, liderada por equipos de investigación como los de Douglas Engelbart y posteriormente popularizada por Xerox y Apple, transformó la interacción [69]. Aunque la interacción seguía siendo explícita (clic, arrastrar y soltar), se volvió significativamente más intuitiva y accesible para el usuario común, democratizando la computación personal [13].
3. **Hacia la Computación Ubicua y las interfaces naturales (2000s-Presente):** con la proliferación de dispositivos móviles y sensores, el enfoque de la HMI se expandió más allá del escritorio. Este nuevo paradigma, conocido como **Computación Ubicua**, defiende la integración de la tecnología en el entorno del usuario, haciéndola “invisible” y accesible en cualquier momento y lugar [22]. La clave para esta integración es la **consciencia de contexto**, un concepto seminal definido por Dey y Abowd como cualquier información que puede ser utilizada para caracterizar la situación de una entidad” [22]. Es este cambio de enfoque, de una interacción centrada en el dispositivo a una centrada en la situación del usuario, lo que sienta las bases para los sistemas implícitos.

La evolución de la interacción hombre-máquina (HMI) se puede entender como una progresión desde la interacción explícita hacia paradigmas cada vez más implícitos.

2.4. Tecnologías para una interacción implícita y consciente del contexto

Para que el proyecto sea un sistema verdaderamente ubicuo y con inteligencia ambiental, es imprescindible integrar tecnologías que permitan la percepción y la inferencia del comportamiento del usuario de manera natural. El proyecto se apoya en cuatro pilares tecnológicos principales, el reconocimiento facial, el reconocimiento de voz, el procesamiento del lenguaje natural (NLP) y la realidad aumentada (AR). Cada una de estas tecnologías contribuye a un modelo de interacción donde la máquina no solo responde a comandos, sino que proactivamente anticipa y se adapta a las necesidades del niño.

2.4.1. Reconocimiento facial

El reconocimiento facial es una de las tecnologías de percepción más importantes en la actualidad y un pilar fundamental para la creación de sistemas conscientes de contexto e interacción implícita. Su evolución, desde la biometría rudimentaria hasta la inteligencia artificial de vanguardia, ilustra el progreso de la computación ubicua.

Trayectoria y evolución:

La investigación en reconocimiento facial comenzó en la década de 1960. Los primeros sistemas, como el desarrollado por Goldstein, Harmon y Lesk en 1971, se basaban en la medición y el análisis de características faciales manuales, como las distancias entre los puntos clave del rostro (ojos, nariz, boca) [29]. Esta aproximación, aunque pionera, era laboriosa y susceptible a variaciones en la pose o la iluminación.

En la década de 1990, el campo experimentó un salto cualitativo con la aparición de los primeros algoritmos automatizados basados en la visión por computador. El método más influyente de esta era fue Eigenfaces, propuesto por Turk y Pentland en 1991 [76]. Este enfoque utilizaba el análisis de componentes principales (PCA) para reducir la dimensionalidad de las imágenes faciales, extrayendo los componentes más significativos, conocidos como "eigenfaces". Estos componentes formaban una base de datos de características únicas que permitían la identificación. Aunque innovador, este método aún luchaba con las variaciones de la iluminación y las expresiones faciales. Posteriormente, el algoritmo Fisherfaces, desarrollado por Belhumeur, Hespanha y Kriegman, mejoró este concepto al utilizar el análisis

discriminante lineal (LDA) para maximizar la separabilidad de las clases de rostros, ofreciendo mayor robustez frente a la iluminación variable [9].

Sin embargo, el verdadero punto de inflexión llegó en la década de 2010 con la irrupción del aprendizaje profundo (deep learning). Las redes neuronales convolucionales (CNN) revolucionaron el campo al superar drásticamente a los métodos tradicionales. A diferencia de los algoritmos anteriores que dependían de características diseñadas manualmente, las CNN son capaces de aprender de forma automática y jerárquica las características más relevantes para el reconocimiento, desde bordes y texturas en las primeras capas hasta representaciones abstractas de rostros en las capas más profundas [5, 31]. Modelos de vanguardia como DeepFace de Facebook (2014) y FaceNet de Google (2015) demostraron una precisión a nivel humano, haciendo que la tecnología fuera viable para una amplia gama de aplicaciones, incluyendo la identificación en tiempo real y en entornos no controlados.

2.4.2. Reconocimiento de voz y procesamiento del lenguaje natural (NLP)

Aunque a menudo se confunden, el reconocimiento de voz (*Automatic Speech Recognition*, ASR) y el procesamiento del lenguaje natural (*Natural Language Processing*, NLP) son dos tecnologías complementarias, pero con roles distintos. Juntas, forman una de las interfaces más poderosas para la interacción implícita, ya que permiten a una máquina no solo escuchar “sino también entender” el lenguaje humano.

Reconocimiento de voz (ASR)

El reconocimiento de voz es el proceso tecnológico que convierte la voz humana en texto.

Traectoria y evolución: Las raíces del reconocimiento de voz se encuentran en la década de 1950, con los primeros sistemas de reconocimiento de dígitos desarrollados por Bell Laboratories [40]. Durante las décadas siguientes, los avances estuvieron impulsados por la investigación en inteligencia artificial y la lingüística computacional, dando lugar a los modelos basados en fonemas en los años 70. Sin embargo, el verdadero punto de inflexión llegó en la década de 1980 con el uso de Modelos Ocultos de Markov (HMM), un enfoque estadístico que se convirtió en el estándar de la industria durante más de dos décadas. Los HMM modelaban la secuencia de sonidos en el habla, permitiendo el desarrollo de los primeros sistemas comerciales de dictado. En la década de 2010, la llegada de los modelos de *deep learning* (aprendizaje profundo), en particular las redes neuronales profundas (DNN) y las redes neuronales recurrentes (RNN), revolucionó la precisión del ASR. Estos modelos superaron a los HMM al ser capaces de aprender patrones acústicos más complejos, lo que permitió una drástica mejora en

la transcripción del habla natural, incluso con ruidos de fondo [34].

Proceso técnico: Un sistema de ASR moderno típicamente sigue un proceso de dos etapas. Primero, el modelo acústico utiliza la red neuronal para mapear las características del audio (como el espectro de frecuencia) a los fonemas o subunidades de sonido. Segundo, un modelo de lenguaje predice la probabilidad de una secuencia de palabras dada la secuencia de fonemas, lo que ayuda a corregir errores y a estructurar el texto de manera coherente [41].

Procesamiento del lenguaje natural (NLP)

El NLP es un campo de la inteligencia artificial que se centra en la interacción entre computadoras y el lenguaje humano, permitiendo a las máquinas procesar y comprender el significado del texto.

Traectoria y evolución: La historia del NLP comenzó con los sistemas basados en reglas a finales de la década de 1950, que intentaban modelar la gramática y la sintaxis del lenguaje con reglas lógicas. Aunque estos sistemas eran precisos para tareas limitadas, carecían de la flexibilidad para manejar la ambigüedad y la complejidad del lenguaje humano. En la década de 1990, la investigación en NLP dio un giro hacia los métodos estadísticos basados en la frecuencia de las palabras, como los modelos de *N-gramas*, que permitieron a las máquinas aprender patrones de lenguaje a partir de grandes volúmenes de texto [18]. Al igual que con el ASR, la revolución más significativa llegó con el aprendizaje profundo. La aparición de arquitecturas como las redes neuronales recurrentes (RNN) y, más recientemente, los modelos de transformadores como BERT (2018), que utilizan el mecanismo de atención para capturar el contexto a largo plazo de una frase, han permitido a las máquinas comprender el significado semántico y contextual de una manera sin precedentes [21].

Proceso y tareas: El NLP puede realizar una variedad de tareas para inferir el significado de una frase:

- **Análisis de sentimientos:** determinar si el tono del texto es positivo, negativo o neutro.
- **Reconocimiento de entidades nombradas (*Named Entity Recognition*, NER):** identificar y categorizar entidades en el texto, como nombres de personas, lugares u organizaciones.
- **Análisis de intención:** determinar el propósito de una frase, como si es una pregunta, una orden o una afirmación.

2.4.3. Realidad aumentada (AR)

La realidad aumentada (AR) es una tecnología que expande la percepción del mundo físico al superponer información y objetos digitales en tiempo real.

A diferencia de la realidad virtual, que sumerge al usuario en un entorno completamente simulado, la AR mantiene al usuario conectado con el mundo real, enriqueciendo su experiencia con elementos virtuales contextuales. Este enfoque híbrido la convierte en un componente clave para la computación ubicua, ya que permite que los sistemas digitales interactúen con nuestro entorno de manera fluida y significativa.

Trayectoria y evolución

El concepto de realidad aumentada se gestó mucho antes de que la tecnología fuera viable. Un precursor notable es el Sensorama de Morton Heilig (1962), una máquina que buscaba estimular múltiples sentidos para crear una experiencia inmersiva. Sin embargo, el verdadero hito fundacional se atribuye a Ivan Sutherland, quien en 1968 desarrolló el primer visor montado en la cabeza (HMD) para realidad aumentada, apodado “la espada de Damocles” por su peso y sujeción del techo [72]. Este dispositivo primitivo ya permitía a los usuarios ver gráficos por ordenador superpuestos sobre el mundo real.

El término “realidad aumentada” fue acuñado por primera vez en 1992 por el investigador de Boeing Thomas Caudell, para describir un sistema que ayudaba a los operarios a ensamblar arneses de cableado al proyectar esquemas digitales directamente sobre el entorno de trabajo [15]. Durante las décadas siguientes, la AR se mantuvo principalmente en nichos industriales y militares debido a los altos costes y la complejidad técnica.

La democratización de la tecnología llegó en la década de 2010 con la masificación de los teléfonos inteligentes, que integraban las cámaras y los sensores (acelerómetros, giroscopios, GPS) necesarios para ejecutar aplicaciones de AR a gran escala. Títulos como el juego *Pokémon GO* (2016) no solo popularizaron la tecnología para el entretenimiento, sino que también demostraron el potencial de la AR como una nueva forma de interacción con el mundo físico. La culminación de esta era se produjo con el lanzamiento de los kits de desarrollo de software (SDK) ARKit (Apple) y ARCore (Google), que proporcionaron a los desarrolladores herramientas estandarizadas y de alta calidad para crear experiencias de AR estables y fluidas, impulsando un crecimiento exponencial en el sector.

Componentes y tipos de AR

Un sistema de realidad aumentada se compone de varios elementos clave que trabajan en conjunto para integrar los mundos real y digital:

1. **Cámara y sensores:** la cámara captura el entorno físico, mientras que los sensores de movimiento (acelerómetro y giroscopio) rastrean la posición y orientación del dispositivo en el espacio tridimensional. El GPS, por su parte, añade una capa de geolocalización.

2. **Procesamiento e interpretación del entorno:** esta es la parte más avanzada. El sistema utiliza algoritmos de visión por computador para analizar la imagen de la cámara. Este análisis permite dos técnicas clave:
 - **Localización y mapeo simultáneo (SLAM):** Es la técnica fundamental para la AR “sin marcador”. El sistema utiliza puntos de referencia visuales en el entorno para construir un mapa 3D y, al mismo tiempo, determinar su propia posición y orientación dentro de ese mapa. Esto permite a los objetos virtuales permanecer anclados al mundo real de forma realista, incluso cuando el usuario se mueve [44].
 - **Detección de planos y objetos:** los algoritmos pueden identificar superficies planas (como mesas, suelos y paredes) para anclar objetos virtuales, o reconocer objetos específicos (como una verdura) para desencadenar una interacción. Esta capacidad eleva la AR de una simple superposición a un sistema consciente del contexto físico.
3. **Renderizado:** es el proceso de dibujar los objetos virtuales en la pantalla, superponiéndolos en la transmisión de la cámara. El renderizado debe ser lo suficientemente rápido para mantener la ilusión de que el objeto forma parte del entorno real.

Basándonos en estas técnicas de procesamiento, podemos distinguir dos tipos principales de AR:

- **AR basada en marcador (*Marker-based AR*):** requiere que la cámara detecte un marcador predefinido (una imagen específica o un patrón visual) para superponer el contenido digital. Es una técnica robusta y precisa, pero limita la experiencia a la presencia del marcador.
- **AR sin marcador (*Markerless AR*):** se basa en la tecnología SLAM y la detección de planos para proyectar contenido virtual en cualquier superficie del entorno del usuario, ofreciendo una experiencia mucho más flexible y fluida, esencial para la interacción implícita y ubicua.

2.5. Aplicaciones educativas inmersivas: panorama, brechas y mejoras

El panorama de las aplicaciones educativas inmersivas, que utilizan tecnologías como la Realidad Aumentada (AR) y la Realidad Virtual (VR), está experimentando un crecimiento exponencial. Estas herramientas van

más allá de los libros de texto y los videos para ofrecer experiencias de aprendizaje que son inherentemente interactivas y atractivas.

2.5.1. Panorama y tendencias actuales

Las aplicaciones inmersivas actuales se centran en el compromiso y la visualización de conceptos abstractos. En campos como las ciencias naturales y la ingeniería (STEM), la VR se utiliza para crear simulaciones de laboratorio seguras donde los estudiantes pueden manipular moléculas o explorar sistemas biológicos sin riesgos, lo que mejora la retención de la información [62]. La AR, por su parte, permite superponer modelos 3D del cuerpo humano en la mesa de un aula o visualizar conceptos de física en el entorno físico, haciendo que el aprendizaje sea más tangible y colaborativo [44]. La gamificación es otra tendencia, con aplicaciones que utilizan elementos de juego (recompensas, niveles, desafíos) para motivar a los estudiantes y aumentar las tasas de finalización de las actividades.

2.5.2. Brechas críticas y posibles mejoras

A pesar de estos avances, el campo todavía presenta brechas significativas, especialmente desde la perspectiva de la computación ubicua y la interacción implícita. La mayoría de las aplicaciones existentes, aunque “inmersivas”, siguen operando bajo un modelo de interacción explícita. El usuario debe emitir comandos claros (tocar un botón, deslizar el dedo, etc.) para que el sistema reaccione. La interacción es una secuencia de comandos y respuestas, en lugar de un flujo continuo y adaptativo.

Aquí es donde tecnologías como el reconocimiento facial, el NLP y los sensores inteligentes entran en juego como herramientas para abordar estas brechas:

De la interacción explícita a la adaptativa: las aplicaciones educativas podrían evolucionar de ser simplemente interactivas a ser verdaderamente adaptativas. Por ejemplo, un sistema de realidad aumentada podría utilizar el reconocimiento facial para detectar la confusión o el aburrimiento en un estudiante y ajustar automáticamente el nivel de dificultad o presentar una pista, en lugar de esperar a que el estudiante pida ayuda explícitamente.

De la detección de marcador a la consciencia del contexto físico: aunque muchas aplicaciones de AR utilizan marcadores, la verdadera inteligencia ambiental implica la capacidad de un sistema para interpretar el entorno en tiempo real. La integración de la detección de objetos permitiría a una aplicación reconocer el objeto físico que el niño tiene en la mano (por ejemplo, una verdura) y generar una respuesta digital en consecuencia, sin necesidad de un marcador, creando una conexión más fluida entre lo real y lo virtual.

2.5.3. Ética, regulación y buenas prácticas

La implementación de tecnologías que se basan en la percepción del usuario, especialmente en entornos educativos dirigidos a niños, plantea serias consideraciones éticas que deben ser abordadas de manera transparente y responsable.

El Panóptico Digital y los riesgos de la privacidad

El término “Panóptico Digital” evoca la prisión “Panóptico” de Jeremy Bentham, donde la vigilancia es omnipresente, haciendo que los individuos se auto-disciplinen bajo la percepción de que están siendo observados en todo momento. En el contexto de los sensores inteligentes y las tecnologías de IA, este concepto se materializa en la recopilación masiva de datos sobre el comportamiento, las emociones y la ubicación de los usuarios, a menudo sin su total consentimiento o conciencia. Para los niños, que son particularmente vulnerables, esto plantea riesgos significativos. La recopilación de datos biométricos (como los rasgos faciales o la voz) y de información de comportamiento puede generar perfiles detallados que, si no se protegen adecuadamente, podrían ser vulnerables a usos indebidos. Además, existe la preocupación de que una dependencia excesiva de la IA para el monitoreo pueda desplazar las interacciones humanas y limitar el desarrollo de la autonomía del niño.

Regulación y Normativa de Protección de Datos

Para mitigar estos riesgos, la legislación global ha introducido marcos regulatorios estrictos. En Europa, el Reglamento General de Protección de Datos (RGPD) es la normativa regulatoria. El artículo 8 del RGPD establece condiciones especiales para el consentimiento de los niños, exigiendo que se obtenga el consentimiento o la autorización de los titulares de la responsabilidad parental para el tratamiento de datos personales de menores de 16 años (una edad que los estados miembros pueden reducir hasta 13 años) [1]. Este marco legal subraya la necesidad de una protección especial para los datos de los niños, reconociendo su menor capacidad para comprender los riesgos del tratamiento de datos.

Buenas prácticas en el diseño de sistemas para niños

En consonancia con la regulación, se deben adoptar un conjunto de buenas prácticas en el diseño y la implementación de aplicaciones educativas inmersivas:

- **Principio de minimización de datos:** recopilar solo los datos estrictamente necesarios para el funcionamiento de la aplicación.

- **Transparencia y consentimiento parental:** informar de forma clara y accesible a los padres o tutores sobre qué datos se recopilan, cómo se utilizan y cómo se protegen.
- **Diseño centrado en la privacidad:** implementar la privacidad desde el diseño y por defecto. Los sistemas deben ser seguros por diseño, ofreciendo la máxima protección a la información personal.
- **Educación y alfabetización digital:** integrar, en la propia experiencia de la aplicación, lecciones que ayuden a los niños y a sus padres a comprender cómo la tecnología utiliza la información y la importancia de la privacidad.
- **Fomento de la interacción humana:** diseñar sistemas que complementen la interacción humana y el juego en el mundo físico, en lugar de reemplazarlos.

La integración de la interacción implícita y los sistemas conscientes del contexto representa una oportunidad para crear experiencias de aprendizaje más ricas y personalizadas. Sin embargo, para que esta evolución sea beneficiosa, debe estar guiada por un compromiso inquebrantable con la ética, la privacidad y el bienestar de los niños.

Capítulo 3

Estado del Arte

En este capítulo se analizarán aplicaciones y plataformas educativas orientadas a niños y niñas de entre 4 y 6 años, cuyo objetivo principal es favorecer el aprendizaje mediante experiencias interactivas y lúdicas. Este análisis comparativo permite identificar tanto sus fortalezas como sus limitaciones, lo cual sirve de base para poner en valor las aportaciones innovadoras de *Luminia*.

3.1. Aplicaciones educativas con gamificación y narrativa

Endless Alphabet / Endless Numbers (Originator Inc.). Estas aplicaciones se centran en la enseñanza de vocabulario y números a través de animaciones llamativas y personajes simpáticos [36]. Sin embargo, carecen de una narrativa progresiva y de una estructura de desafíos organizados en mundos o niveles, lo que limita la motivación sostenida.

Khan Academy Kids. Plataforma gratuita que ofrece ejercicios interactivos de lectura, matemáticas y lógica [4]. Incluye personajes guía y recompensas básicas, pero no emplea tecnologías inmersivas (como la realidad aumentada) ni una narrativa sólida que funcione como hilo conductor.

ABCmouse. Se trata de una de las plataformas más completas para la etapa infantil, con contenidos relacionados con colores, formas, matemáticas y lectoescritura [59]. Aunque ofrece progresión y recompensas, su interfaz es más tradicional y presenta poca innovación en términos de interacción inmersiva.

3.2. Aplicaciones con realidad aumentada

Quiver. Permite colorear dibujos que cobran vida en 3D mediante realidad aumentada [48]. No obstante, la experiencia resulta aislada, sin progresión narrativa ni gamificación avanzada.

AR Flashcards. Aplicación basada en tarjetas educativas con RA para aprender letras, números y animales [50]. El aprendizaje es limitado y repetitivo, sin misiones ni progresión estructurada.

Pokémon Smile. Motiva a los niños a cepillarse los dientes usando RA y recompensas [19]. Aunque es atractiva, se centra exclusivamente en un hábito de higiene y no en aprendizajes académicos amplios.

3.3. Juegos educativos con mundos temáticos

Toontastic 3D (Google). Facilita la creación de historias animadas, fomentando la creatividad y la narración [51]. Sin embargo, está más orientada a la expresión artística que a aprendizajes curriculares.

Osmo. Combina objetos físicos con interacción digital en actividades relacionadas con matemáticas, letras y lógica [37]. Aunque innovador, requiere hardware adicional y no presenta una narrativa global unificada.

Toca Boca (Toca Life World, Toca Nature, etc.). Ofrece experiencias sandbox educativas basadas en la exploración libre [3]. Su principal limitación es la ausencia de un sistema de progresión y de objetivos de aprendizaje definidos.

3.4. Comparativa

En la Tabla 3.1 se presenta una síntesis comparativa de las aplicaciones analizadas, considerando aspectos como el uso de tecnologías inmersivas, la presencia de narrativa, el sistema de progresión y recompensas, así como sus principales limitaciones.

3.5. Síntesis

Del análisis se concluye que muchas aplicaciones actuales carecen de una narrativa estructurada que guíe la experiencia de aprendizaje, combinando progresión y motivación a través de mundos temáticos. Asimismo, son escasas las propuestas que integran tecnologías inmersivas como la realidad aumentada junto con sistemas de recompensas robustos y adaptativos.

En este contexto, *Luminia* se presenta como una propuesta innovadora que combina narrativa, gamificación avanzada y tecnologías emergentes (realidad aumentada, interacción implícita y conciencia de contexto) para ofrecer una experiencia educativa más completa e inmersiva, capaz de motivar al alumnado de manera sostenida y adaptada a su edad.

Aplicación	Tecnologías usadas	Narrativa	Progresión y recompensas	Limitaciones
Endless Alphabet / Numbers	Animaciones 2D	No	Limitada	Sin narrativa global ni mundos temáticos
Khan Academy Kids	Interactividad básica	Parcial	Sí	Sin RA ni narrativa inmersiva
ABCmouse	Plataforma digital completa	Parcial	Sí	Interfaz tradicional, poca innovación
Quiver	Realidad aumentada	No	No	Experiencia aislada, sin progresión
AR Flashcards	Realidad aumentada	No	No	Aprendizaje repetitivo y limitado
Pokémon Smile	Realidad aumentada	Muy básica	Sí	Enfoque muy específico (higiene)
Toontastic 3D	Animación 3D	Sí	No	Orientada a creatividad, no a contenidos curriculares
Osmo	Interacción física + digital	Parcial	Sí	Requiere hardware adicional, sin narrativa global
Toca Boca	Exploración digital libre	No	No	Sin objetivos educativos estructurados

Cuadro 3.1: Comparativa de aplicaciones educativas existentes.

Capítulo 4

Planificación del proyecto

A lo largo de este capítulo se describe el proceso de planificación seguido en el desarrollo de *Luminia*, desde su metodología hasta la gestión técnica y temporal del proyecto. Se detalla la metodología de desarrollo, las fases metodológicas, el cronograma de trabajo, los recursos empleados y las estrategias adoptadas para mitigar los posibles riesgos a detectar durante la implementación.

4.1. Metodología de desarrollo

El desarrollo de *Luminia* se ha basado en un enfoque ágil. Esta elección responde a la necesidad de mantener una evolución iterativa, con revisiones frecuentes y una alta capacidad de adaptación.

4.1.1. Principios adoptados

- **Iteración continua:** el desarrollo se ha llevado a cabo en cinco iteraciones, con revisiones técnicas al final de cada ciclo.
- **Retroalimentación constante:** la validación con el tutor académico y la autoevaluación tras cada iteración han permitido refinar objetivos y priorizar tareas.
- **Integración incremental:** cada módulo funcional se ha desarrollado de forma independiente y se ha integrado progresivamente en el sistema global.
- **Control de versiones:** todo el código fuente se ha gestionado mediante **Git** y **GitHub**, permitiendo mantener un registro de cambios y revertir versiones en caso de error.

4.1.2. Etapas de desarrollo

El proyecto se ha estructurado en cinco etapas principales:

1. **Investigación:** Análisis de estudios sobre aprendizaje con realidad aumentada, gamificación y teorías del desarrollo infantil (Piaget [60]; Ausubel [6]). Esta etapa permitió definir las bases pedagógicas y psicológicas del sistema, asegurando que las mecánicas del juego fomenten el aprendizaje significativo y la motivación intrínseca.
2. **Definición de requisitos:** Identificación de funcionalidades (voz, RA, base de datos, gamificación) y de criterios de calidad (usabilidad y seguridad infantil).
3. **Diseño de arquitectura y experiencia de usuario (UX):** Diseño modular basado en hilos concurrentes y desarrollo de prototipos. Se definen los flujos de interacción naturales entre usuario, asistente virtual y entorno físico.
4. **Desarrollo e integración tecnológica:** Implementación de los módulos principales para los cambios de estado, reconocimiento de voz, reproducir audios, gestionar los juegos y renderizado. Configuración de base de datos y comunicación entre componentes mediante.
5. **Pruebas y validación:** Verificación técnica y validación pedagógica del sistema con usuarios reales. Evaluación de la fluidez de la voz, la precisión del reconocimiento y la experiencia inmersiva general.

4.2. Cronograma del proyecto

El desarrollo del proyecto se ha llevado a cabo durante un periodo aproximado de **tres meses**, distribuidos en cinco iteraciones consecutivas. Cada iteración concluye con una revisión técnica, retroalimentación con el tutor y ajustes.

La distribución de las actividades han sido representadas en el Cuadro 4.1. En dicho cuadro se observa que la primera iteración fue, sin duda, la que requirió una mayor inversión de tiempo. Esto se debe principalmente a la fase inicial de investigación, tanto de las herramientas tecnológicas como de los fundamentos pedagógicos y psicológicos que sustentan el proyecto. Quedando finalmente, un total de **327 horas**

Cuadro 4.1: Reparto de tareas y horas dedicadas por iteración

Tarea	Horas	Inicio	Fin
ITERACIÓN 1			
Investigación teórica	48	2025-08-18	2025-09-05
Redacción capítulo de introducción	9	2025-08-25	2025-08-28
Redacción capítulo del Marco teórico	11	2025-08-28	2025-09-01
Redacción capítulo del Estado del Arte	10	2025-09-01	2025-09-05
Configuración del entorno	4	2025-08-18	2025-08-19
Implementación del reconocimiento facial	9	2025-08-26	2025-08-29
Implementación del inicio de sesión	7	2025-08-29	2025-09-01
Implementación de la interacción por voz	11	2025-09-01	2025-09-05
ITERACIÓN 2			
Guía interactiva Tina (Diseño + Implementación)	12	2025-09-05	2025-09-09
Implementación módulo realidad aumentada	12	2025-09-09	2025-09-14
Implementación del gestor de mundos educativos	12	2025-09-12	2025-09-17
Redacción capítulo de Planificación	9	2025-09-16	2025-09-19
Redacción capítulo de Requisitos	11	2025-09-18	2025-09-23
Pruebas iniciales de comunicación	9	2025-09-22	2025-09-25
ITERACIÓN 3			
Integración gestor de mundos con narraciones de Tina	6	2025-09-25	2025-09-28
Desarrollo lógica de juegos	18	2025-09-26	2025-10-03
Desarrollo desafíos educativos	14	2025-10-02	2025-10-08
Redacción capítulo Requisitos	13	2025-10-08	2025-10-13
Redacción capítulo Análisis	10	2025-10-12	2025-10-16
ITERACIÓN 4			
Integración BD	5	2025-10-16	2025-10-17
Evaluación automática de los juegos	6	2025-10-17	2025-10-20
Retroalimentación emocional al terminar cada desafío	4	2025-10-20	2025-10-22
Integración de recompensas	6	2025-10-21	2025-10-23
Desarrollo para mostrar el progreso del jugador	4	2025-10-22	2025-10-24
Continuación redacción capítulo de Requisitos	11	2025-10-23	2025-10-28
Redacción capítulo Diseño	15	2025-10-24	2025-10-30
ITERACIÓN 5			
Desarrollo para editar el perfil del usuario	4	2025-10-30	2025-10-31
Desarrollo de cierre de sesión seguro	2	2025-10-31	2025-10-31
Validación final, pruebas de integración	9	2025-10-31	2025-11-04
Redacción capítulo Implementación	15	2025-11-03	2025-11-09
Redacción capítulo de Pruebas	6	2025-11-07	2025-11-10
Redacción capítulo de Conclusiones y trabajos futuros	5	2025-11-09	2025-11-10

Además, muestro el diagrama de Gantt obtenido en la Figura 4.1



Figura 4.1: Diagrama de Gantt.

4.3. Presupuesto

Todos los programas y herramientas empleados en el desarrollo del proyecto *Luminia* son de código abierto o disponen de versiones gratuitas para uso académico, por lo que no se han generado gastos por licencias de software.

Hardware

Para el desarrollo y las pruebas del sistema se ha utilizado el siguiente equipo informático y dispositivos complementarios:

■ Ordenador portátil para desarrollo

- Modelo: MacBook Pro 15" (2018).
- Procesador: Intel Core i7 de 6 núcleos a 2.2 GHz y 16 GB de memoria DDR4 a 2400 MHz.
- Memoria RAM: 16 GB.
- Almacenamiento: 512 GB SSD.
- Precio estimado: 500 €.

■ Webcam externa

- Resolución: 4 Megapíxeles, Full HD
 - Conexión: USB.
 - Precio estimado: 100 €.
- **Tablero físico con marcadores ArUco**
 - Material: Cartón pluma impreso.
 - Uso: Interacción física para la realidad aumentada.
 - Precio estimado: 1 €.

Software y entorno de ejecución

El desarrollo de *Luminia* se ha realizado íntegramente con herramientas y librerías de software libre:

- Sistema operativo: **macOS Sequoia 15.6**.
- Lenguaje de programación: **Python 3.13**.
- Librerías principales: **OpenCV, SpeechRecognition, PyAudio, PyMongo, Threading**.
- Base de datos: **MongoDB** ejecutada en un contenedor **Docker**.
- IDE: **Visual Studio Code**.
- Control de versiones: **GitHub**.

El proyecto se ejecuta en un entorno local virtualizado dentro del propio ordenador, sin requerir despliegue en servidores externos ni servicios de alojamiento en la nube. Esto reduce los costes operativos a cero.

Coste total estimado

Dado que el desarrollo se ha realizado con medios personales y software libre, los únicos costes asociados corresponden al hardware utilizado para el desarrollo y pruebas del sistema:

Concepto	Coste (€)
MacBook Pro 15" (2018)	500
Webcam externa	100
Tablero con marcadores ArUco	1
Total estimado	601 €

4.4. Gestión de riesgos

Durante la planificación de *Luminia* se ha realizado un análisis de riesgos con el objetivo de anticipar posibles incidencias técnicas, organizativas y éticas, definiendo estrategias preventivas para minimizar su impacto.

Los principales riesgos identificados y sus medidas de mitigación fueron los siguientes:

- **Integración de la realidad aumentada:** posibles incompatibilidades entre librerías de visión por computador y módulos de RA. **Mitigación:** uso de versiones estables de *OpenCV* y pruebas unitarias desde etapas tempranas.
- **Retrasos en la planificación:** riesgo de desajustes por dificultades técnicas. **Mitigación:** aplicación de enfoque ágil y reajuste flexible de tareas según prioridad.
- **Dependencia de hardware externo:** funcionamiento condicionado al uso de cámara y micrófono. **Mitigación:** detección automática de dispositivos y manejo de errores mediante excepciones controladas.
- **Privacidad y protección infantil:** al dirigirse a menores, se evita el tratamiento de datos sensibles. **Mitigación:** almacenamiento únicamente del vector facial para reconocimiento, ejecutando la base de datos **MongoDB** en un contenedor **Docker** aislado y sin comunicación con servicios externos.

Evaluación y cumplimiento ético-legal

El enfoque preventivo ha permitido garantizar la conformidad del sistema con las **directrices internacionales de software educativo seguro**, en línea con las recomendaciones de la *Agencia Española de Protección de Datos* [20], la *OCDE* [58] y diversos autores especializados [25, 49, 81].

El proyecto incorpora principios de **Privacy by Design**, garantizando la minimización de datos, la transparencia del tratamiento y la supervisión adulta durante el uso. Además, la interacción por voz y los contenidos visuales se han diseñado conforme a criterios éticos y pedagógicos adecuados a la edad preescolar, asegurando una experiencia educativa segura y emocionalmente positiva.

En conclusión, **Luminia** cumple plenamente con las recomendaciones internacionales sobre privacidad, ética y seguridad en software educativo infantil. Su arquitectura modular, el uso controlado de reconocimiento facial, el entorno local aislado de la base de datos y el diseño cognitivo adaptado a la edad, garantizan un entorno **seguro y pedagógicamente responsable**, coherente con las directrices propuestas por organismos internacionales y la literatura científica reciente.

Capítulo 5

Ingeniería de requisitos

El presente capítulo establece los requisitos funcionales, no funcionales, de información, del sistema, pedagógicos y de denominaciones que definen el comportamiento, las condiciones y las limitaciones de **Luminia**. El objetivo es asegurar que el sistema cumpla sus metas pedagógicas, tecnológicas y de usabilidad bajo un marco formal de ingeniería del software.

5.1. Descripción de los implicados

Los implicados o actores del sistema representan a todas las entidades humanas y técnicas que intervienen directa o indirectamente en el funcionamiento de *Luminia*. A continuación, se detallan los actores identificados, sus responsabilidades e implicación dentro del sistema.

Usuario (niño)

Descripción	Niños y niñas de entre 4 y 6 años que utilizan <i>Luminia</i> como herramienta educativa interactiva, empleando voz y marcadores físicos para participar en actividades de realidad aumentada.
Tipo	Usuario final del sistema.
Responsabilidad	Participar activamente en las actividades, responder a las indicaciones del asistente virtual y manipular los elementos físicos durante el aprendizaje.
Criterios de éxito	Que el sistema sea fácil de usar, visualmente atractivo y proporcione una experiencia de aprendizaje positiva y fluida.
Implicación	Alta; utiliza el sistema de forma directa y continua.
Comentarios/Cuestiones	Se prioriza la sencillez de interacción y la seguridad de los datos; no se almacena información personal identificable.

Asistente virtual (Tina)

Descripción	Agente conversacional inteligente que guía al usuario, ofrece instrucciones y proporciona retroalimentación emocional y cognitiva durante las actividades.
Tipo	Entidad virtual integrada en el sistema.
Responsabilidad	Dirigir el flujo de interacción, motivar al usuario, adaptar las respuestas según el contexto y asegurar una experiencia educativa segura.
Criterios de éxito	Que las respuestas sean comprensibles, coherentes y adecuadas.
Implicación	Permanente; participa en todas las interacciones del sistema.
Comentarios/Cuestiones	Su comportamiento se rige por principios de comunicación empática y lenguaje adaptado a la edad preescolar.

5.2. Especificación de requisitos

5.2.1. Requisitos funcionales

Los requisitos funcionales especifican los comportamientos observables del sistema, las acciones que ejecuta y las respuestas que produce ante la interacción de los actores.

- **RF-1.** El sistema deberá realizar el **reconocimiento facial** del usuario al inicio de la sesión para identificar su perfil en la base de datos.
- **RF-2.** El sistema deberá permitir la **interacción mediante comandos de voz naturales**, sin necesidad de interfaces gráficas complejas.
- **RF-3.** La asistente virtual Tina deberá guiar al usuario mediante mensajes sonoros y visuales durante todo el proceso educativo.
- **RF-4.** El sistema deberá **evaluar las respuestas del usuario** y emitir retroalimentación positiva o correctiva.
- **RF-5.** El sistema deberá renderizar **objetos tridimensionales en realidad aumentada** sobre marcadores ArUco detectados por cámara.
- **RF-6.** El sistema deberá permitir la **selección de mundos educativos** (letras, animales, frutas, números) mediante comandos de voz.

- **RF-7.** El sistema deberá registrar el **progreso y puntuaciones** de cada usuario en la base de datos.
- **RF-8.** El sistema deberá permitir la **personalización de la asistente virtual** mediante recompensas simbólicas.
- **RF-9.** El sistema deberá permitir la **edición de datos del perfil** por parte del usuario.
- **RF-10.** El sistema deberá **guardar automáticamente** los datos de progreso y cerrar la sesión de forma segura.

5.2.2. Requisitos no funcionales

Los requisitos no funcionales definen las propiedades de calidad y restricciones que debe cumplir el sistema para garantizar una experiencia eficiente, segura y mantenible.

- **RNF-1.** La interfaz de usuario deberá ser **intuitiva y simple** para niños sin habilidades avanzadas de lectura.
- **RNF-2.** El reconocimiento facial y de voz deberá completarse con una latencia máxima de **4 s**.
- **RNF-3.** El renderizado de objetos de RA deberá ejecutarse a una tasa mínima de **30 FPS**.
- **RNF-4.** El código fuente deberá seguir una **arquitectura modular**, permitiendo la integración de nuevos módulos sin alterar el núcleo.
- **RNF-5.** Los datos del usuario deberán almacenarse en un contenedor **Docker** protegido frente a accesos no autorizados.
- **RNF-6.** El sistema deberá garantizar la **integridad de la sesión** ante fallos de hardware o interrupciones de energía.

5.2.3. Requisitos de información

La base de datos del sistema *Luminia* se gestiona mediante **MongoDB**, donde se almacenan los datos del usuario, el progreso educativo y los elementos de personalización del entorno virtual. A continuación se describen los requisitos de información identificados:

- **RI-1. Usuario.** Información básica del jugador registrada en el sistema. **Contenido:** nombre de usuario, vector facial, fecha de registro.

- **RI-2. Progreso educativo.** Datos relativos a los logros obtenidos por el jugador en cada mundo temático. **Contenido:** destellos conseguidos en el mundo de las letras, mundo de los animales, mundo de las frutas y verduras, mundo de los números y mundo final.
- **RI-3. Estadísticas globales.** Información consolidada del rendimiento del jugador. **Contenido:** destellos totales acumulados y número de *lunios* obtenidos.
- **RI-4. Personalización.** Información asociada al aspecto visual y objetos del asistente. **Contenido:** disfraces disponibles y disfraz equipado Tina.
- **RI-5. Estado de juego.** Datos sobre el punto actual del progreso en la sesión. **Contenido:** mundo actual, minijuego actual, fase.
- **RI-6. Desbloques y avance.** Información relativa a la evolución del jugador en los diferentes mundos. **Contenido:** mundos desbloqueados.

5.2.4. Requisitos del sistema

Estos requisitos definen las dependencias técnicas y condiciones necesarias de hardware y software para el correcto funcionamiento del sistema.

Hardware

- **RS-1.** El sistema deberá ejecutarse en un equipo con procesador de al menos 2.2 GHz y 8 GB de RAM.
- **RS-2.** Deberá disponer de cámara HD (mínimo 720p) y micrófono.
- **RS-3.** La GPU deberá ser compatible con OpenGL 3.0 o superior para soportar renderizado RA.

Software

- **RS-4.** El sistema deberá ser compatible con Windows 10, Ubuntu 22.04 LTS o macOS 13.
- **RS-5.** Deberá ejecutarse sobre Python 3.13 con las librerías `OpenCV`, `SpeechRecognition`, `PyAudio`, `PyMongo`, `Threading` y `NumPy`.
- **RS-6.** Deberá integrarse con una base de datos MongoDB gestionada en un contenedor Docker.

5.2.5. Requisitos pedagógicos

Estos requisitos establecen las condiciones educativas que debe cumplir Luminia para alinearse con los principios del aprendizaje constructivista y significativo.

- **RP-1.** El sistema deberá fomentar la **participación activa** del niño a través de la manipulación física y la interacción por voz.
- **RP-2.** El sistema deberá ofrecer **refuerzo positivo inmediato** mediante recompensas simbólicas y mensajes motivadores.
- **RP-3.** El sistema deberá integrar **estímulos multisensoriales** (visuales, auditivos y kinestésicos) en sus actividades.
- **RP-4.** El sistema deberá promover la **autonomía guiada** mediante la mediación de la asistente virtual, sin sustituir la figura del docente.

5.2.6. Requisitos de denominaciones

Estos requisitos establecen las condiciones lingüísticas, cognitivas y afectivas que deben cumplir las denominaciones empleadas en *Luminia* (personajes, aplicación y elementos del entorno) para garantizar su accesibilidad, pronunciabilidad y conexión emocional con el público infantil.

Requisitos lingüísticos y cognitivos

- **RD-1.** Las denominaciones deberán ser **cortas**, preferiblemente **bi-silábicas o trisílabas**, para facilitar su pronunciación y memorización en niños de 4 a 6 años [28, 67].
- **RD-2.** Deberán emplear **fonemas simples** (p. ej., /t/, /m/, /n/, /a/, /i/), evitando grupos consonánticos complejos o sonidos difíciles de articular a esa edad [67, 74].
- **RD-3.** Las denominaciones deberán estar asociadas a **conceptos concretos y familiares**, ya que los niños en edad preescolar comprenden y recuerdan mejor palabras que refieren a objetos o ideas tangibles [54, 70].
- **RD-4.** Los nombres deberán mantener **coherencia entre pronunciación y grafía**, evitando discrepancias ortográficas que puedan dificultar su aprendizaje o reconocimiento [74].
- **RD-5.** Las denominaciones deberán poseer una **carga emocional positiva** o evocativa, que favorezca el vínculo afectivo con el personaje o entorno educativo [54, 70].

5.3. Casos de uso

Los casos de uso describen las interacciones entre los actores y el sistema, garantizando la cobertura de todos los requisitos definidos en las secciones anteriores. El diagrama general se muestra en la Figura 5.1.

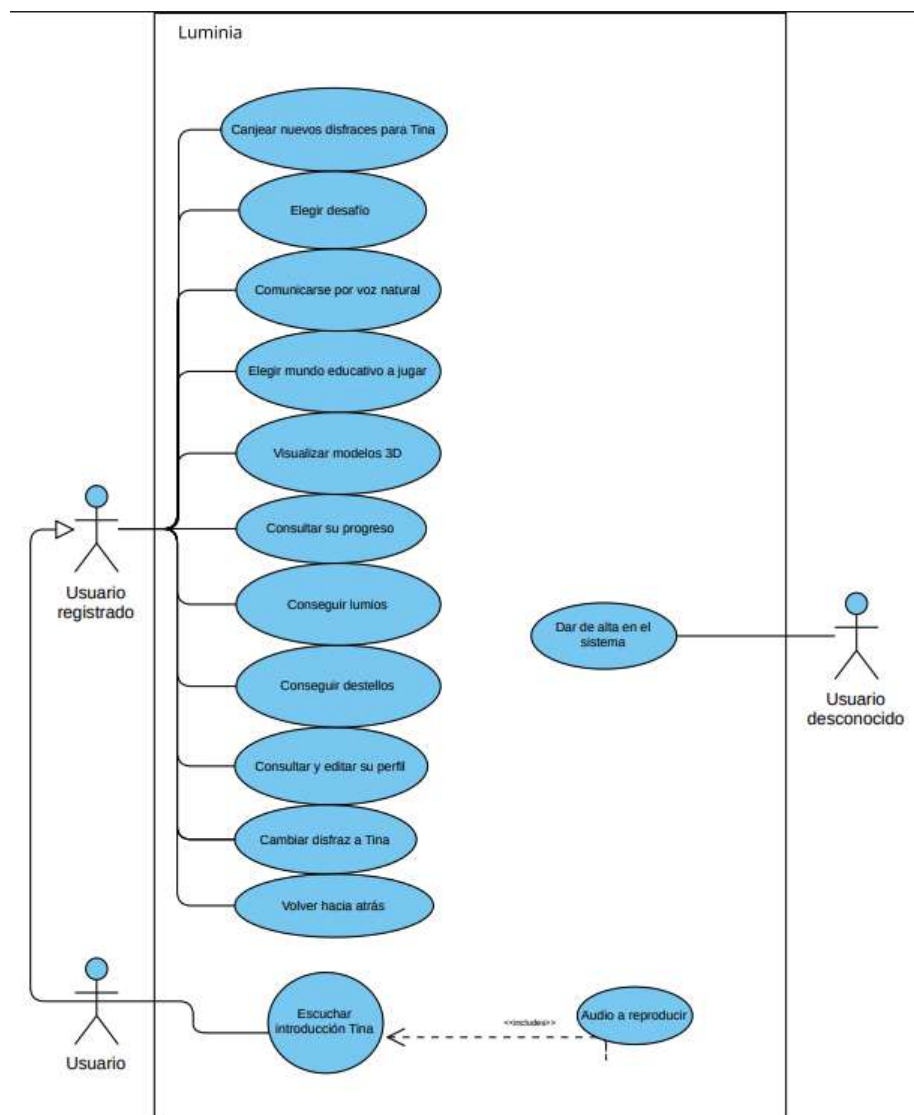


Figura 5.1: Diagrama general de casos de uso del sistema Luminia.

5.4. Descripción detallada de casos de uso

En esta sección se desarrollan en más profundidad los casos de uso que describen las distintas interacciones posibles entre los actores del sistema *Luminia* y la aplicación.

He optado por elaborar casos de uso de dos niveles: algunos de carácter más general, centrados en representar la interacción del usuario infantil con el sistema de forma abstracta y pedagógica; y otros más concretos, próximos al diseño técnico, donde se detallan los flujos de ejecución internos y los mecanismos de reconocimiento, comunicación o almacenamiento.

El objetivo es mantener un equilibrio entre la perspectiva conceptual (que refleja la experiencia educativa y la narrativa interactiva guiada por *Tina*) y la perspectiva técnica, que recoge el comportamiento del sistema y la gestión de los datos en cada interacción.

Cuadro 5.1: Caso de uso CU-1: Registrarse / Reconocimiento facial

Actor principal	Usuario (niño)
Requisitos asociados	RF-1, RNF-2, RS-2, RS-3, RNF-5, RI-1, RI-5
Precondición	El sistema está encendido y la cámara activa. La fase actual tiene que ser reconocimiento facial”.
Curso normal	1. El sistema solicita al usuario que mire a la cámara y diga registrarme”. 2. Se genera un vector facial. 3. El vector se compara con los registros de la base de datos. 4. Si no hay coincidencia, el sistema solicita datos al usuario y carga el perfil.
Curso alternativo	a) Si hay coincidencia, el sistema solicita que se inicie sesión diciendo “iniciar sesión”. b) Si falla la cámara, el sistema muestra error de dispositivo.
Postcondición	El usuario queda autenticado y su perfil cargado en memoria.

Cuadro 5.2: Caso de uso CU-2: Iniciar sesión / Reconocimiento facial

Actor principal	Usuario (niño)
Requisitos asociados	RF-1, RNF-2, RS-2, RS-3, RNF-5, RI-1, RI-5
Precondición	El sistema está encendido y la cámara activa. La fase actual tiene que ser reconocimiento facial”.
Curso normal	1. El sistema solicita al usuario que mire a la cámara y diga “iniciar sesión”. 2. Se genera un vector facial. 3. El vector se compara con los registros de la base de datos. 4. Si hay coincidencia, el sistema carga el perfil.
Curso alternativo	a) Si no hay coincidencia, el sistema solicita que se registre diciendo registrarme”. b) Si falla la cámara, el sistema muestra error de dispositivo.
Postcondición	El usuario queda autenticado y su perfil cargado en memoria.

Cuadro 5.3: Caso de uso CU-3: Comunicación natural por voz

Actor principal	Usuario (niño)
Requisitos asociados	RF-2, RNF-1, RNF-2, RP-3, RI-5
Precondición	El módulo de reconocimiento de voz está activo.
Curso normal	1. El usuario pronuncia un comando. 2. El sistema procesa la señal y la transcribe. 3. El comando se interpreta y ejecuta.
Curso alternativo	a) Si no se reconoce el comando, se pide repetir.
Postcondición	La orden se procesa correctamente y se ejecuta la acción.

Cuadro 5.4: Caso de uso CU-4: Guía interactiva y narración contextual

Actor principal	Asistente virtual (Tina)
Requisitos asociados	RF-3, RNF-1, RP-4, RD-1, RD-2, RD-3, RD-4, RD-5
Precondición	Usuario autenticado.
Curso normal	1. Tina explica reglas y objetivos del juego. 2. Ajusta su tono según desempeño del usuario.
Curso alternativo	Si el usuario no responde, Tina anima al jugador.
Postcondición	El usuario comprende la dinámica de la actividad.

Cuadro 5.5: Caso de uso CU-5: Evaluación y retroalimentación

Actor principal	Sistema Luminia
Requisitos asociados	RF-4, RP-2, RS-6, RI-2, RI-3
Precondición	El usuario ha emitido una respuesta.
Curso normal	1. El sistema analiza la respuesta. 2. Determina si es correcta o incorrecta. 3. Registra el resultado y actualiza la puntuación. 4. Tina ofrece refuerzo positivo o corrección.
Curso alternativo	Si no se entiende la respuesta, Tina solicita repetir.
Postcondición	Se almacena la evaluación y se actualiza el progreso.

Cuadro 5.6: Caso de uso CU-6: Visualización en realidad aumentada

Actor principal	Usuario (niño)
Requisitos asociados	RF-5, RNF-3, RP-3, RS-3
Precondición	Módulo de RA activo y cámara operativa.
Curso normal	1. Se detectan marcadores ArUco. 2. El sistema genera objetos 3D sobre el tablero físico. 3. Tina describe los elementos mostrados.
Curso alternativo	Si el marcador no se reconoce, se vuelve a intentar incorporar.
Postcondición	Se visualiza correctamente el contenido educativo.

Cuadro 5.7: Caso de uso CU-7: Selección de mundo educativo

Actor principal	Usuario (niño)
Requisitos asociados	RF-6, RP-1, RNF-1, RI-5
Precondición	Usuario autenticado y módulo de voz activo.
Curso normal	1. Tina enumera los mundos disponibles. 2. El usuario elige uno mediante comando de voz. 3. El sistema carga los recursos asociados.
Curso alternativo	Si no se entiende la orden, en la interfaz aparecerán indicaciones.
Postcondición	Mundo educativo seleccionado y activo.

Cuadro 5.8: Caso de uso CU-8: Juegos y desafíos educativos

Actor principal	Usuario (niño)
Requisitos asociados	RF-4, RF-7, RP-1, RP-2, RI-2, RI-6
Precondición	Mundo educativo activo.
Curso normal	1. El sistema presenta una actividad. 2. El usuario responde verbalmente. 3. Se evalúa la respuesta y se actualiza la puntuación.
Curso alternativo	Si el usuario no responde, se espera hasta que responda.
Postcondición	Se registra el resultado en la base de datos.

Cuadro 5.9: Caso de uso CU-9: Personalización de Tina y recompensas

Actor principal	Usuario (niño)
Requisitos asociados	RF-8, RP-2, RNF-1, RI-4
Precondición	Usuario autenticado con puntos disponibles.
Curso normal	1. Tina informa de las opciones de personalización. 2. El usuario elige un accesorio o disfraz. 3. El sistema aplica el cambio.
Curso alternativo	Si no hay puntos suficientes, se notifica la limitación.
Postcondición	Personalización registrada en el perfil del usuario.

Cuadro 5.10: Caso de uso CU-10: Consulta y gestión del progreso

Actor principal	Usuario (niño)
Requisitos asociados	RF-7, RNF-4, RS-6, RI-2, RI-3
Precondición	Usuario autenticado.
Curso normal	El sistema muestra estadísticas del usuario.
Postcondición	Configuración guardada en la base de datos.

Cuadro 5.11: Caso de uso CU-11: Edición de datos del perfil

Actor principal	Usuario (niño)
Requisitos asociados	RF-9, RNF-4, RS-6, RI-1
Precondición	Usuario autenticado.
Curso normal	1. Se accede al perfil. 2. Se modifican datos. 3. El sistema actualiza la base de datos.
Postcondición	Cambios almacenados correctamente.

Cuadro 5.12: Caso de uso CU-12: Guardado automático y cierre seguro

Actor principal	Sistema Luminia
Requisitos asociados	RF-10, RNF-6, RS-6, RI-5
Precondición	Sesión activa.
Curso normal	1. El sistema guarda el estado actual. 2. Cierra los módulos abiertos. 3. Sistema despide al usuario.
Postcondición	Sesión finalizada y datos preservados.

Cuadro 5.13: Caso de uso CU-13: Comando de salida rápida

Actor principal	Usuario (niño)
Requisitos asociados	RF-10, RNF-6, RS-6
Precondición	Sistema en ejecución en fase "menu principal" o "reconocimiento facial".
Curso normal	1. El usuario dice "salir". 2. El sistema guarda el progreso y cierra procesos. 3. Sistema confirma el cierre.
Postcondición	Aplicación cerrada correctamente.

Cuadro 5.14: Caso de uso CU-14: Comando pantalla anterior

Actor principal	Usuario (niño)
Requisitos asociados	RF-10, RNF-6, RS-6
Precondición	Sistema en ejecución en cualquier fase que no sea "menu principal".
Curso normal	1. El usuario dice "volver" o "atrás". 2. El sistema guarda el progreso y cierra procesos. 3. El sistema vuelve al estado anterior.
Postcondición	Pantalla anterior cargada correctamente.

5.5. Diagrama de paquetes

El diagrama de paquetes mostrado representa la estructura modular del sistema *Luminia*. Cada paquete agrupa componentes con responsabilidades similares. El paquete **modules** constituye el núcleo funcional, mientras que los paquetes **ar**, **config**, **utils**, **media** y **models** ofrecen servicios de soporte a la lógica de juego. Las dependencias se indican mediante flechas discontinuas, indicando relaciones de uso entre paquetes.

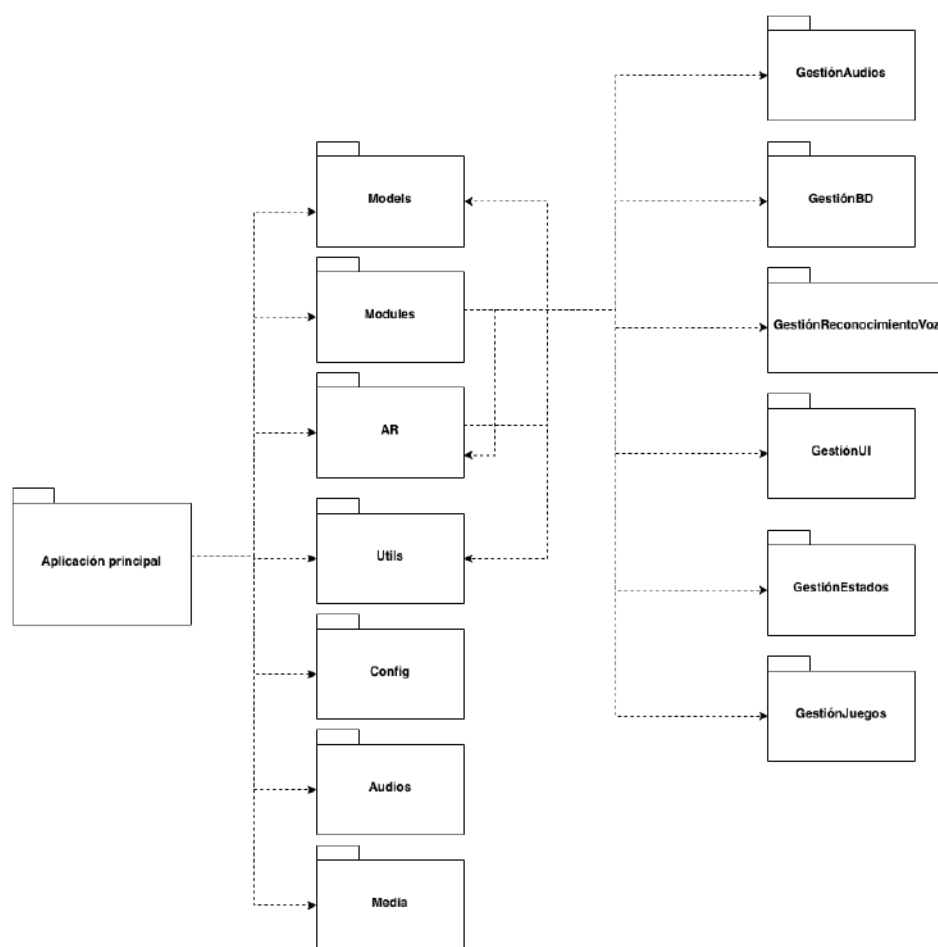


Figura 5.2: Diagrama de paquetes del sistema Luminia.

5.6. Diagramas de actividad

En este apartado se muestran los diagramas de actividad para representar el flujo de los procesos contenidos en los casos de uso.

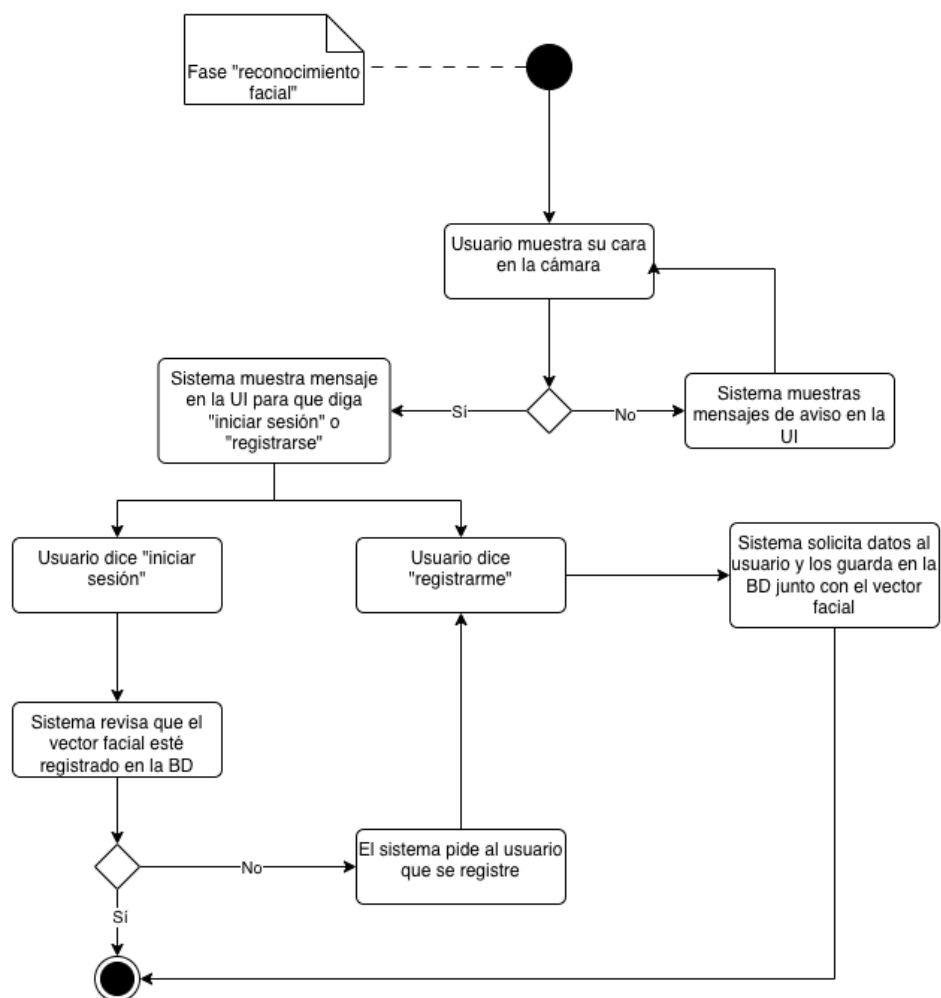


Figura 5.3: Diagrama de actividad CU-1 y CU-2: Reconocimiento facial.

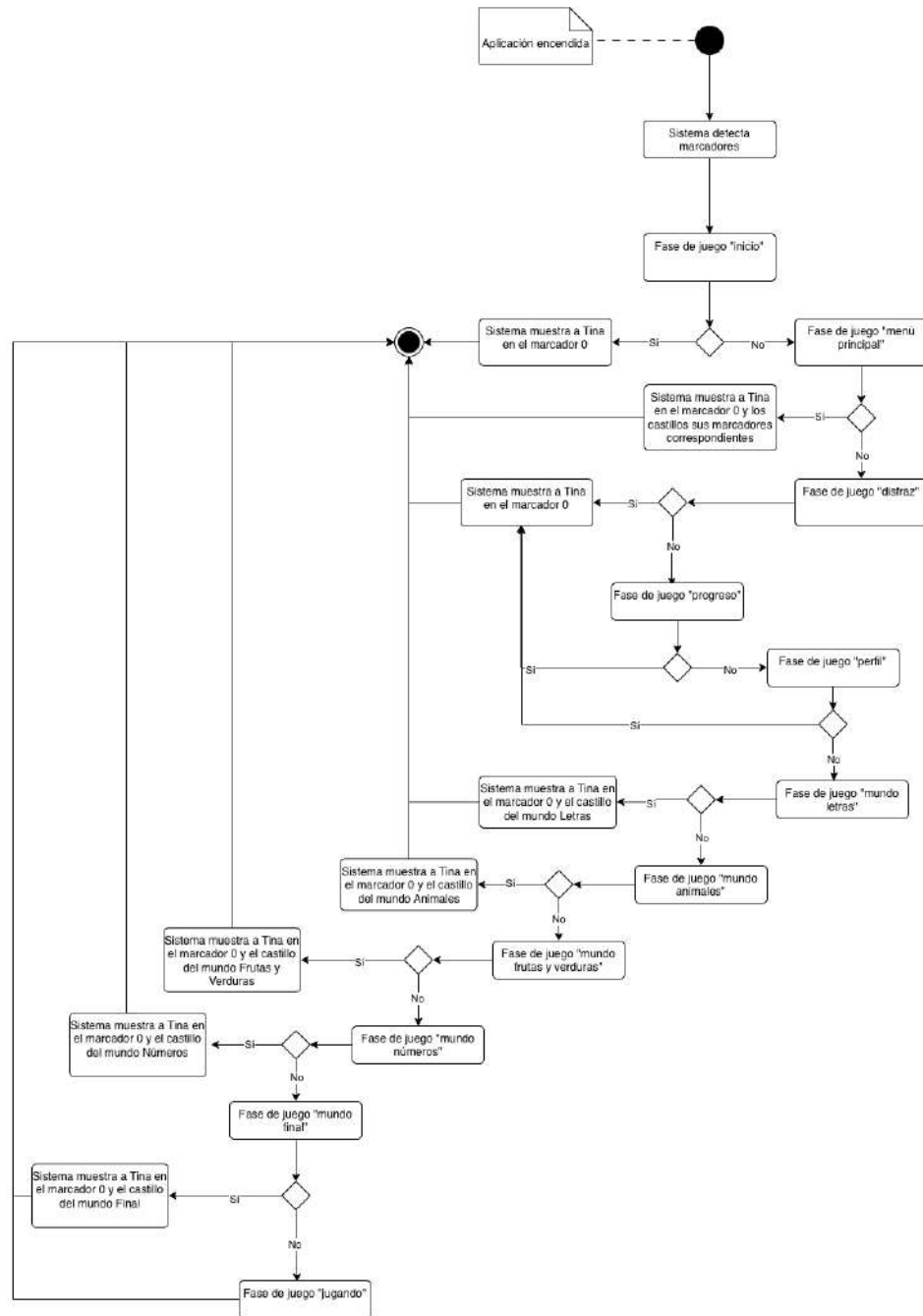


Figura 5.4: Diagrama de actividad CU-6: Visualización RA.

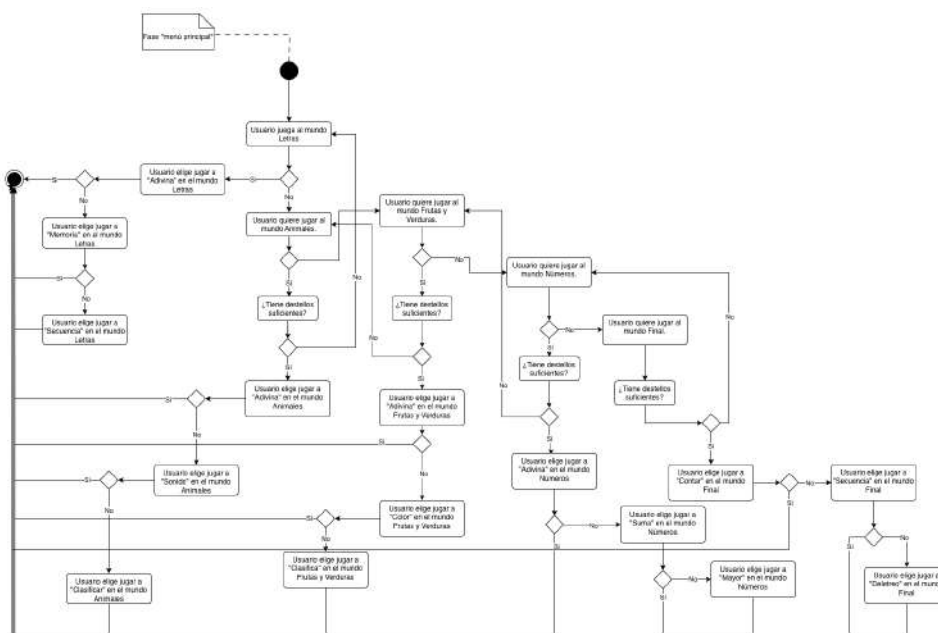


Figura 5.5: Diagrama de actividad CU-8: Juegos y desafíos educativos.

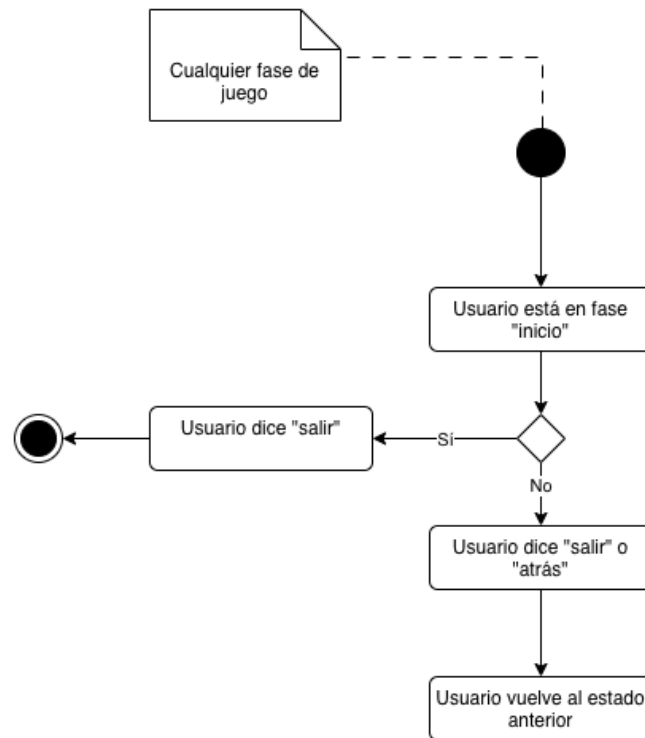


Figura 5.6: Diagrama de actividad CU-13 y CU-14: Comando de salida.

Matriz de trazabilidad de requisitos

La matriz de trazabilidad asegura que todos los requisitos definidos estén cubiertos por al menos un caso de uso del sistema *Luminia*. Esto garantiza la coherencia entre el análisis de requisitos, el diseño y la validación del sistema.

Cuadro 5.15: Matriz de trazabilidad de requisitos y casos de uso

Caso de uso	Requisitos asociados
CU-1: Registrarse / Reconocimiento facial	RF-1, RNF-2, RS-2, RS-3, RNF-5, RI-1, RI-5
CU-2: Iniciar sesión / Reconocimiento facial	RF-1, RNF-2, RS-2, RS-3, RNF-5, RI-1, RI-5
CU-3: Comunicación natural por voz	RF-2, RNF-1, RNF-2, RP-3, RI-5
CU-4: Guía interactiva y narración contextual	RF-3, RNF-1, RP-4, RD-1, RD-2, RD-3, RD-4, RD-5
CU-5: Evaluación y retroalimentación	RF-4, RP-2, RS-6, RI-2, RI-3
CU-6: Visualización en realidad aumentada	RF-5, RNF-3, RP-3, RS-3
CU-7: Selección de mundo educativo	RF-6, RP-1, RNF-1, RI-5
CU-8: Juegos y desafíos educativos	RF-4, RF-7, RP-1, RP-2, RI-2, RI-6
CU-9: Personalización de Tina y recompensas	RF-8, RP-2, RNF-1, RI-4
CU-10: Consulta y gestión del progreso	RF-7, RNF-4, RS-6, RI-2, RI-3
CU-11: Edición de datos del perfil	RF-9, RNF-4, RS-6, RI-1
CU-12: Guardado automático y cierre seguro	RF-10, RNF-6, RS-6, RI-5
CU-13: Comando de salida rápida	RF-10, RNF-6, RS-6
CU-14: Comando pantalla anterior	RF-10, RNF-6, RS-6

Capítulo 6

Análisis de requisitos

En este capítulo se presenta el modelo conceptual del sistema, seguido de diagramas de secuencia de los casos más relevantes. Finalmente, se definen historias de usuario; cada una se relaciona con uno o varios casos de uso del capítulo anterior y se definen criterios de aceptación claros para su validación.

6.1. Modelo conceptual

El sistema **Luminia** se ha diseñado como una aplicación educativa inmersiva.

En el proyecto, no se crean ni destruyen marcadores durante el juego, el tablero físico ya incorpora los identificadores ArUco necesarios, y el sistema se encarga de mostrar y ocultar modelos 3D sobre ellos en función de la fase activa del juego. La arquitectura modular permite que los componentes de voz, visión, lógica y datos trabajen de forma concurrente, coordinados por el módulo principal.

Cada módulo implementa una funcionalidad específica: detección de marcadores, renderizado de modelos 3D, reconocimiento de voz, control de flujo del juego, gestión de base de datos y reproducción de audio. La comunicación entre estos módulos se realiza a través de un objeto compartido de tipo **GameState**, que actúa como repositorio central del estado global del sistema.

6.2. Diagramas secuencia

Esta sección se ha planteado con el objetivo de mostrar las interacciones entre los objetos que provocan algunos de los casos de uso más importantes por parte de los actores.

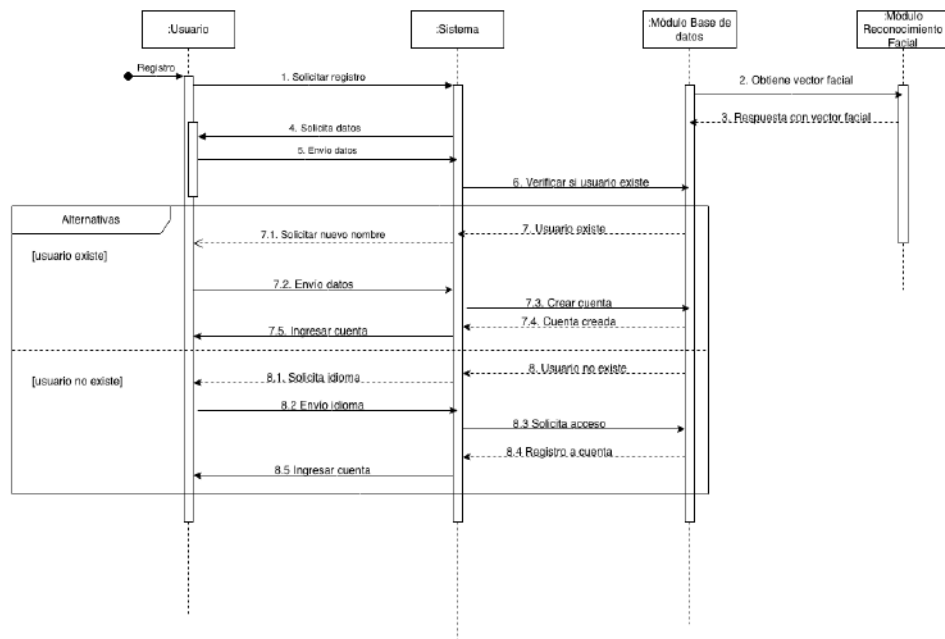


Figura 6.1: Diagrama de secuencia del registro de sesión.

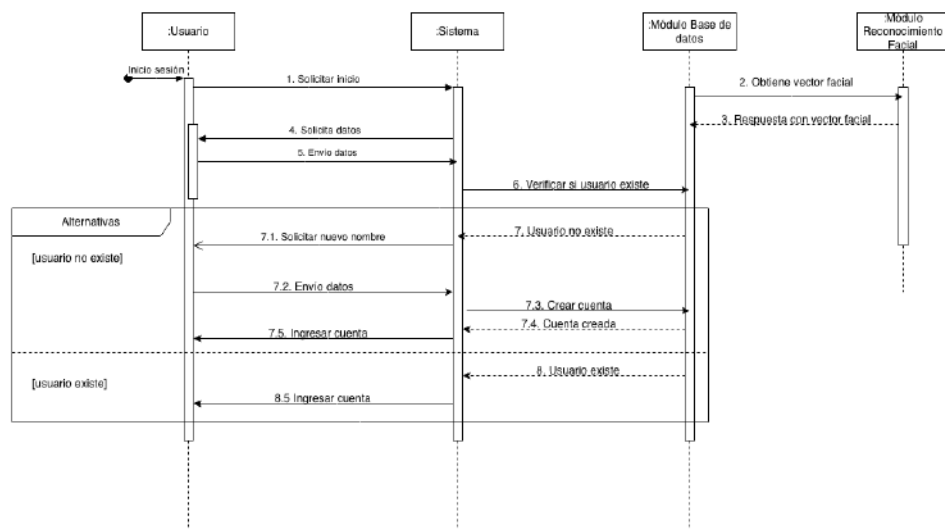


Figura 6.2: Diagrama de secuencia del inicio de sesión.

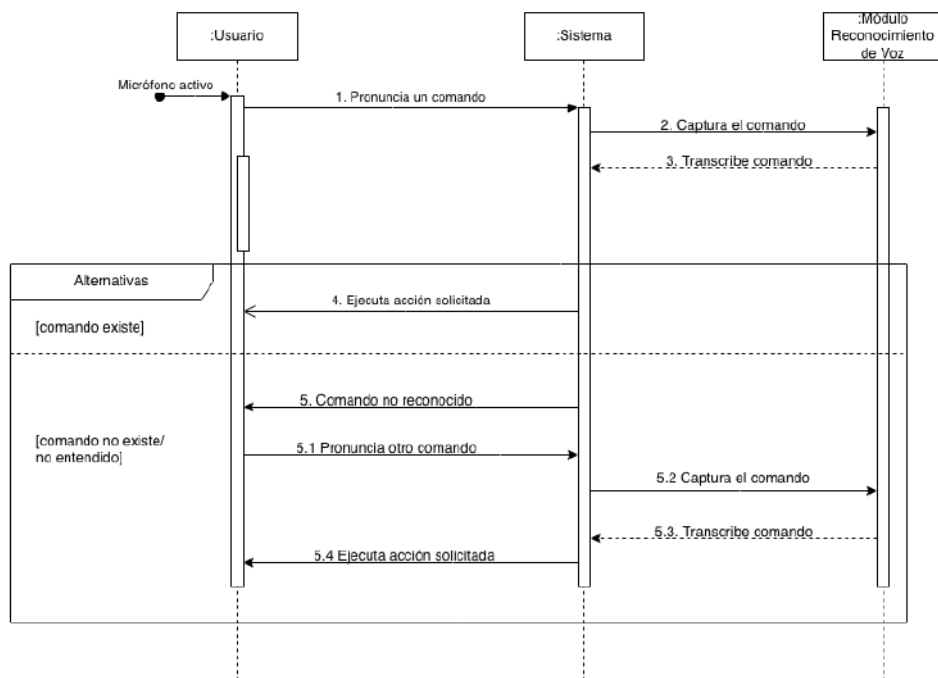


Figura 6.3: Diagrama de secuencia de la comunicación por voz.

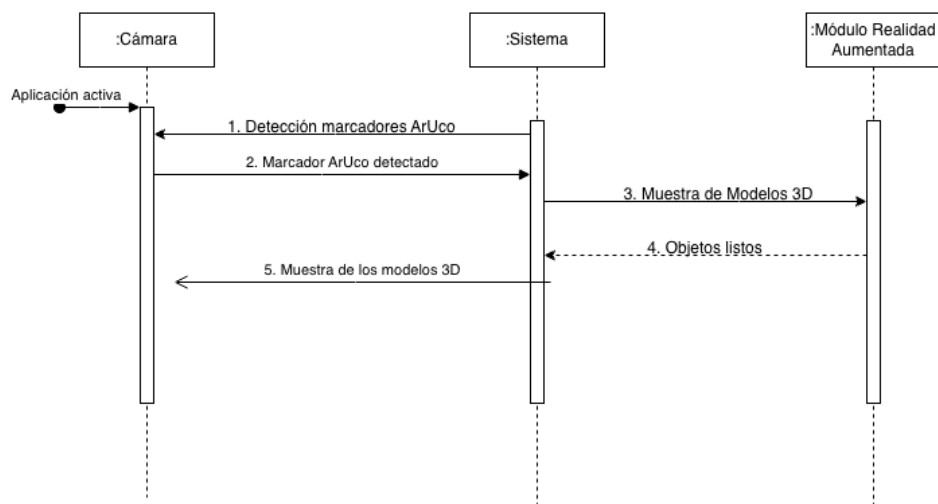


Figura 6.4: Diagrama de secuencia de la visualización de la Realidad Aumentada.

6.3. Historias de usuario

6.3.1. Product backlog

El *Product Backlog* agrupa las funcionalidades previstas para el desarrollo de Luminia. Los ítems se han ordenado según su prioridad, valor educativo y complejidad técnica.

Cuadro 6.1: Product backlog del sistema Luminia

ID HU	Descripción resumida	Prioridad	Iteración
HU1	Identificación del jugador mediante reconocimiento facial.	Alta	1
HU2	Comunicación natural por voz con Tina.	Alta	2
HU3	Guía interactiva y narración contextual.	Alta	2
HU4	Visualización en realidad aumentada.	Alta	2
HU5	Selección del mundo educativo.	Media-Alta	2
HU6	Juegos y desafíos educativos.	Alta	3
HU7	Evaluación y retroalimentación inmediata.	Alta	4
HU8	Recompensas y personalización.	Media	4
HU9	Consulta del progreso y estadísticas.	Media	5
HU10	Edición del perfil del usuario.	Media	5
HU11	Guardado automático y cierre seguro.	Alta	5
HU12	Salida rápida por comando de voz.	Alta	5

6.3.2. Tarjetas de Historias de Usuario

Las siguientes tarjetas detallan las historias de usuario del sistema Luminia, siguiendo la estructura: descripción, criterios de aceptación, prioridad, iteración, estado y relación con los casos de uso.

Cuadro 6.2: HU1 – Identificación del jugador

ID	HU1
Nombre	Identificación del jugador mediante reconocimiento facial
Descripción	Como usuario infantil, quiero que el sistema me reconozca por mi rostro para acceder automáticamente a mi cuenta.
Criterios de aceptación	▪ Precisión de reconocimiento facial superior al 90 %. ▪ Asociación correcta del vector facial con el perfil en la base de datos.
Prioridad	Alta
Iteración	1 – Implementación del módulo de reconocimiento facial
Estado	Completado
Casos de uso relacionados	CU-1, CU-2

Cuadro 6.3: HU2 – Comunicación natural por voz

ID	HU2
Nombre	Comunicación natural por voz con Tina
Descripción	Como usuario, quiero comunicarme con Tina mediante voz para controlar el sistema sin botones ni menús.
Criterios de aceptación	▪ Reconocimiento correcto de comandos básicos (“iniciar”, “salir”, “letras”) en menos de 4 s. ▪ Respuesta coherente por voz mediante audios.
Prioridad	Alta
Iteración	2 – Integración del reconocimiento y audios
Estado	Completado
Casos de uso relacionados	CU-1, CU-2, CU-3

Cuadro 6.4: HU3 – Guía interactiva y narraciones

ID	HU3
Nombre	Guía interactiva y narraciones contextuales
Descripción	Como jugador, quiero que Tina me oriente con explicaciones y mensajes positivos durante las actividades educativas.
Criterios de aceptación	■ Narraciones sincronizadas con las fases del juego. ■ Tono de voz afectivo y adaptado al contexto. ■ Reconocimiento de voz pausado durante la locución.
Prioridad	Alta
Iteración	2 – Desarrollo del módulo <code>audio_manager</code>
Estado	Completado
Casos de uso relacionados	CU-4, CU-5

Cuadro 6.5: HU4 – Visualización en realidad aumentada

ID	HU4
Nombre	Visualización en realidad aumentada
Descripción	Como jugador, quiero ver objetos 3D sobre el tablero físico para aprender jugando.
Criterios de aceptación	■ Detección estable de marcadores ArUco. ■ Renderizado fluido a 30 FPS. ■ Correspondencia entre marcador y modelo educativo.
Prioridad	Alta
Iteración	2 – Implementación del motor RA <code>ui_renderer</code>
Estado	Completado
Casos de uso relacionados	CU-6

Cuadro 6.6: HU5 – Selección del mundo educativo

ID	HU5
Nombre	Selección del mundo educativo
Descripción	Como jugador, quiero elegir el mundo a explorar (letras, frutas, animales, números).
Criterios de aceptación	▪ Comando de voz reconocido correctamente. ▪ Carga sin errores de modelos y sonidos del mundo elegido.
Prioridad	Media-Alta
Iteración	2 – Integración del módulo GestorJuegosAR
Estado	Completado
Casos de uso relacionados	CU-7

Cuadro 6.7: HU6 – Juegos y desafíos educativos

ID	HU6
Nombre	Juegos y desafíos educativos
Descripción	Como jugador, quiero participar en juegos interactivos para aprender y ganar puntos.
Criterios de aceptación	▪ Evaluación inmediata de respuestas. ▪ Retroalimentación sonora y visual coherente.
Prioridad	Alta
Iteración	3 – Implementación del módulo GestorJuegosAR
Estado	Completado
Casos de uso relacionados	CU-8

Cuadro 6.8: HU7 – Evaluación y retroalimentación

ID	HU7
Nombre	Evaluación y retroalimentación inmediata
Descripción	Como jugador, quiero saber si mis respuestas son correctas para poder mejorar durante el juego.
Criterios de aceptación	■ Análisis automático de las respuestas del usuario (voz o interacción). ■ Retroalimentación visual y sonora inmediata. ■ Registro de resultados en la base de datos.
Prioridad	Alta
Iteración	4 – Integración del módulo de evaluación y feedback
Estado	Completado
Casos de uso relacionados	CU-5, CU-8

Cuadro 6.9: HU8 – Recompensas y personalización

ID	HU8
Nombre	Recompensas y personalización de la asistente virtual
Descripción	Como jugador, quiero ganar Lumios y canjearlos por disfraces o accesorios para Tina.
Criterios de aceptación	■ Los puntos acumulados se registran correctamente en la base de datos. ■ Personalizaciones disponibles se cargan dinámicamente. ■ Confirmación por voz tras cada canje exitoso.
Prioridad	Media
Iteración	4 – Desarrollo del módulo de gestión de recompensas
Estado	Completado
Casos de uso relacionados	CU-9

Cuadro 6.10: HU9 – Consulta del progreso

ID	HU9
Nombre	Consulta del progreso y estadísticas
Descripción	Como jugador, quiero ver mis avances y logros dentro del sistema.
Criterios de aceptación	▪ Los datos se obtienen desde la base de datos en tiempo real. ▪ Presentación gráfica del progreso del jugador. ▪ Acceso mediante comando de voz simple (“progreso”).
Prioridad	Media
Iteración	5 – Integración del panel de progreso
Estado	Completado
Casos de uso relacionados	CU-10

Cuadro 6.11: HU10 – Edición del perfil de usuario

ID	HU10
Nombre	Edición del perfil de usuario
Descripción	Como jugador o tutor, quiero modificar mis datos personales para mantener mi información actualizada.
Criterios de aceptación	▪ Validación de sesión activa antes de la modificación. ▪ Actualización persistente en la base de datos. ▪ Confirmación de los cambios aplicados.
Prioridad	Media
Iteración	5 – Implementación de la edición de perfil
Estado	Completado
Casos de uso relacionados	CU-11

Cuadro 6.12: HU11 – Guardado automático y cierre seguro

ID	HU11
Nombre	Guardado automático y cierre seguro
Descripción	Como jugador, quiero que el sistema guarde mi progreso antes de cerrar la aplicación.
Criterios de aceptación	▪ Guardado automático antes del cierre de sesión. ▪ Cierre controlado de procesos y recursos abiertos. ▪ Confirmación de cierre.
Prioridad	Alta
Iteración	5 – Implementación del flujo de persistencia y cierre
Estado	Completado
Casos de uso relacionados	CU-12

Cuadro 6.13: HU12 – Salida rápida por comando de voz

ID	HU12
Nombre	Salida rápida por comando de voz
Descripción	Como jugador, quiero decir “volver” o “atrás” para salir de la fase donde estoy.
Criterios de aceptación	▪ Comando de salida reconocido sin errores. ▪ Guardado previo del progreso del usuario. ▪ Liberación de recursos antes del cambio.
Prioridad	Alta
Iteración	5 – Implementación del cierre global por voz
Estado	Completado
Casos de uso relacionados	CU-13, CU-14

Capítulo 7

Diseño

Hasta el momento, la interacción usuario-sistema ha sido algo abstracta, por lo que abordaremos en primer lugar su interacción. Seguidamente se tratará el diseño de los diferentes módulos tecnológicos.

7.1. Diseño de la interacción usuario–sistema

Principio de interacción

El diseño de interacción de **Luminia** se rige por el principio de **interacción natural y multimodal**. El usuario no debe enfrentarse a una interfaz tradicional con menús o botones, sino comunicarse mediante comandos de voz. El tablero físico con marcadores ArUco actúa como interfaz tangible, permitiendo que la interacción se produzca mediante la observación y manipulación de elementos físicos.

Flujo de interacción

El flujo de interacción define las etapas principales de comunicación entre el usuario y el sistema, organizadas de forma secuencial.

1. **Inicio del sistema:** inicialización de los módulos de cámara, micrófono y conexión con la base de datos. Tina da la bienvenida mediante audios de voz.
2. **Reconocimiento facial:** el jugador se coloca frente a la cámara; el sistema identifica su rostro y cuando el usuario indica, el sistema carga su perfil.
3. **Selección de mundo:** el usuario elige verbalmente la temática (“frutas”, “animales”) y el sistema carga los recursos correspondientes.

4. **Ejecución del juego:** el tablero actúa como superficie aumentada. Los modelos 3D se actualizan dinámicamente según la fase del juego y la respuesta del usuario.
5. **Feedback interactivo:** Tina evalúa las respuestas y proporciona retroalimentación inmediata (voz y mensajes en la interfaz).
6. **Finalización:** el sistema guarda el progreso.

7.2. Diseño de la experiencia inmersiva y la interfaz educativa

El diseño de la experiencia inmersiva en *Luminia* se fundamenta en la creación de un entorno emocionalmente positivo, visualmente atractivo y cognitivamente estimulante para el público infantil. El sistema combina estímulos visuales, auditivos y de interacción natural para fomentar la atención, la comprensión y la motivación del niño durante el aprendizaje.

Principios de diseño visual y sensorial

La composición visual y sonora ha sido desarrollada siguiendo criterios de accesibilidad y usabilidad infantil:

- **Dimensión visual:** uso de modelos tridimensionales con texturas coloridas, formas redondeadas y fondos neutros que reducen la sobrecarga perceptiva y favorecen la concentración en los elementos educativos principales.
- **Dimensión auditiva:** implementación de una voz femenina afectiva que acompaña al niño con mensajes breves, refuerzos positivos y un tono amable, generando cercanía emocional (“¡Excelente trabajo!”, “Vamos a intentarlo de nuevo”).
- **Accesibilidad sensorial:** se priorizan contrastes altos e instrucciones auditivas claras. Los tiempos de espera y respuesta se adaptan al ritmo individual del usuario, garantizando una interacción inclusiva.

Diseño de la interacción educativa

La interfaz se basa en un enfoque de **diseño centrado en el niño**, eliminando barreras cognitivas y priorizando la simplicidad y la respuesta inmediata. La navegación no depende de menús ni botones tradicionales, sino de comandos de voz y elementos visuales reactivos que refuerzan la comprensión.

- **Simplicidad visual:** disposición de elementos grandes y fácilmente identificables, con paletas de colores cálidos y alto contraste para maximizar la legibilidad y la atención.
- **Continuidad narrativa:** el personaje virtual Tina mantiene un tono coherente y constante durante toda la experiencia, favoreciendo la inmersión y la sensación de acompañamiento.
- **Retroalimentación emocional:** los mensajes de voz y mensajes en la interfaz, responden al desempeño del jugador, proporcionando estímulos positivos tanto ante los aciertos como ante los errores.
- **Accesibilidad cognitiva:** comandos orales simples, ausencia de texto denso y tiempos de respuesta ajustados al contexto educativo.
- **Indicadores de estado:** iconos dinámicos (por ejemplo, el micrófono activo) y variaciones cromáticas informan al usuario sobre las fases de escucha o habla del sistema.

En conjunto, estos principios de diseño permiten una experiencia coherente, afectiva y adaptada a las necesidades del aprendizaje infantil, reforzando el vínculo entre tecnología, emoción y educación.

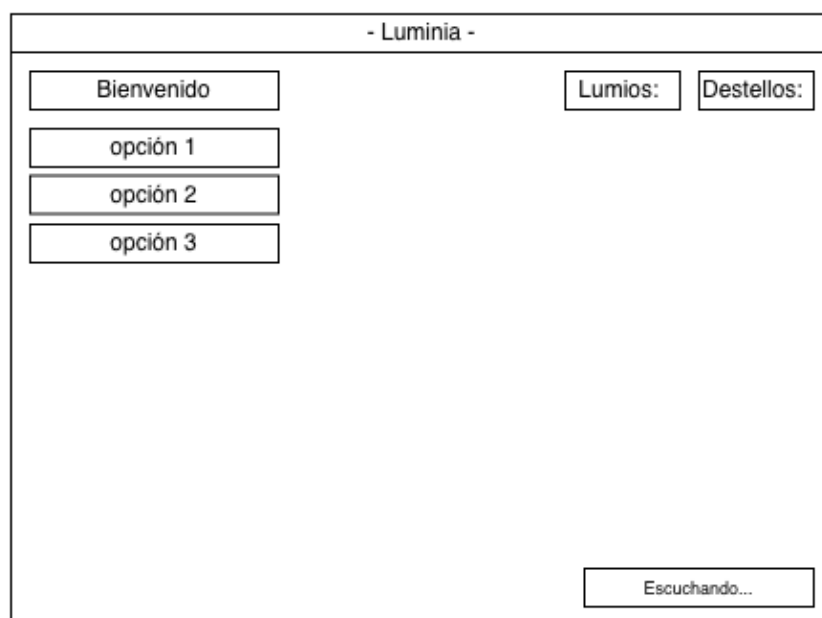


Figura 7.1: Boceto de la interfaz educativa de Luminia durante la fase de menú principal.

60 7.2. Diseño de la experiencia inmersiva y la interfaz educativa

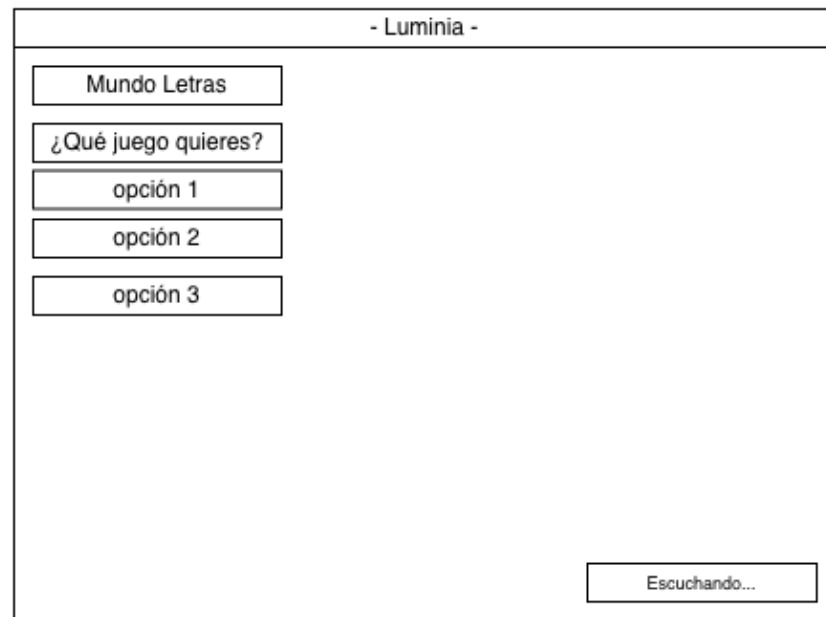


Figura 7.2: Boceto de la interfaz educativa de Luminia durante la fase de mundo elegido.

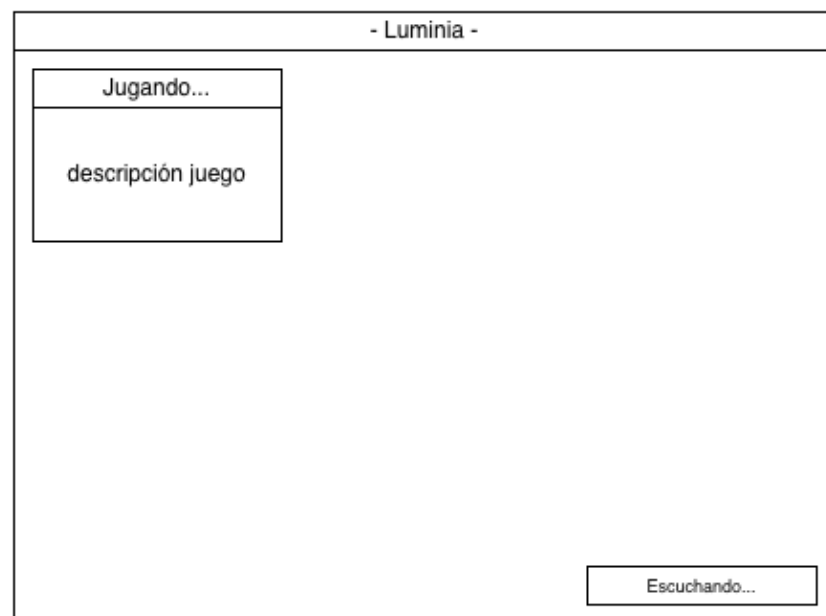


Figura 7.3: Boceto de la interfaz educativa de Luminia durante la fase de juego.

7.3. Diseño de los módulos tecnológicos

El sistema se compone de seis módulos principales. Todos se comunican mediante el objeto compartido `GameState`, que gestiona el estado global del juego, las señales de control y la sincronización de hilos. De este modo, cada módulo puede operar en hilos independientes manteniendo coherencia y evitando condiciones de carrera.

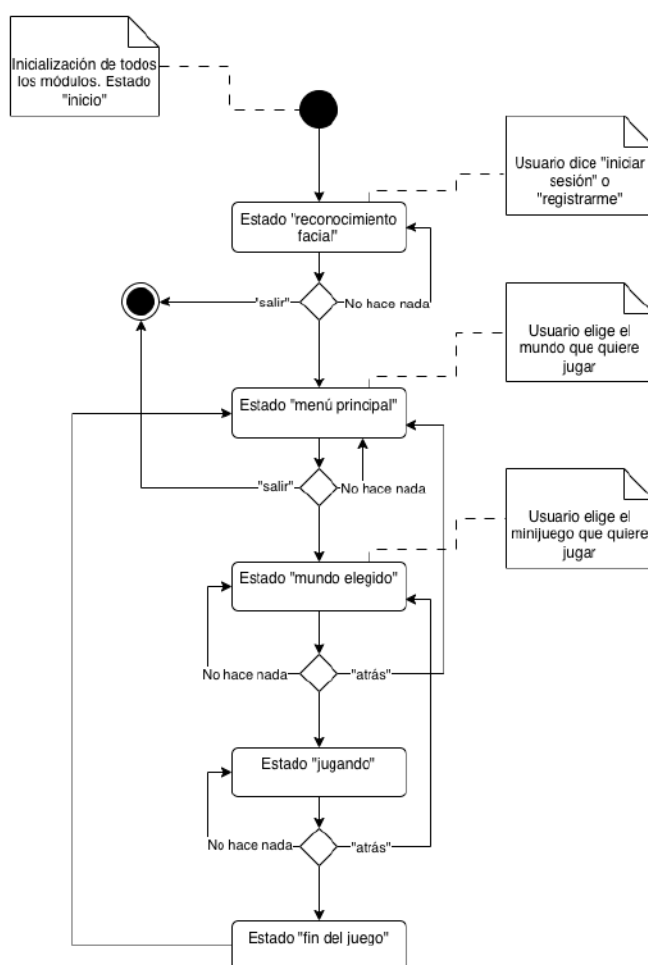


Figura 7.4: Diagrama de estados principal del sistema *Luminia*.

Módulo `data_manager` (`MongoDBManager`)

Responsable de la conexión con la base de datos y de la gestión de operaciones CRUD (Create, Read, Update, Delete). Debe de operar dentro de un contenedor Docker, garantizando el aislamiento de datos.

Funciones principales:

- `conectar()`: establece una conexión segura con la base de datos.
- `desconectar()`: cierra la conexión activa, liberando recursos.
- `encontrar_usuario(nombre)`: recupera información de perfil y progreso del jugador.
- `actualizar_usuario()`: modifica los datos del usuario o registra un evento de sesión.

Módulo `game_state`

Actúa como núcleo lógico del sistema y coordinador de procesos. Mantiene los estados principales del ciclo de ejecución como (`inicio`, `fase_juego`, `salir`) y sincroniza la comunicación entre los módulos concurrentes.

Su diseño multihilo reduce la latencia y evita bloqueos durante la ejecución de voz y renderizado simultáneos.

Módulo `audio_manager`

Gestiona la generación y reproducción de voz de Tina mediante reproducción de audios. Su objetivo es ofrecer una respuesta vocal natural y evitar solapamientos con el módulo de reconocimiento de voz.

El módulo emplea la librería `pygame.mixer` para la reproducción de archivos de audio, integrando una cola de reproducción que asegura la ejecución secuencial de los mensajes. Durante la reproducción, el micrófono se desactiva temporalmente mediante los callbacks `on_talk_start()` y `on_talk_end()`, coordinados con el `GameState`, de forma que controla cuándo el sistema puede hablar o escuchar.

Características técnicas:

- Sistema de colas concurrentes para gestionar la prioridad de mensajes.
- Prevención de repeticiones mediante intervalos mínimos configurables.
- Control de tiempo máximo de reproducción (`timeout`) para evitar bloqueos.
- Callbacks sincronizados que actualizan el estado de escucha y habla del sistema.

La clase `AudioManager` ejecuta un hilo interno encargado de procesar la cola de audios, garantizando la reproducción sin interrupciones. En caso de error durante la carga o ejecución del audio, se invoca el callback de finalización para restaurar el estado del micrófono y evitar inconsistencias en el sistema.

Módulo `voice_recognition`

Encargado de la transcripción de comandos hablados mediante la librería **SpeechRecognition**. Este módulo monitoriza el micrófono, convierte la señal acústica a texto y envía los comandos reconocidos al **GameState**.

Antes de iniciar la escucha, el módulo comprueba el estado global para asegurarse de que el sistema no esté hablando, evitando así interferencias acústicas.

Funcion principal: `reconocimiento_voz()`: inicializa el hilo de reconocimiento de voz.

Módulo `ui_renderer`

Responsable del renderizado de la escena aumentada mediante **OpenCV** y detección de marcadores **ArUco**. Este módulo combina información visual del entorno con elementos gráficos 3D superpuestos, generando una experiencia de realidad aumentada fluida.

Su ejecución en un hilo independiente permite mantener una tasa estable de al menos 30 FPS, incluso durante la reproducción de audio o la ejecución de lógica de juego. El renderizador controla la cámara, aplica transformaciones de pose y proyecta los modelos tridimensionales sobre la imagen real.

Funciones principales:

- `detectar_marcadores()`: localiza patrones ArUco y calcula sus transformaciones.
- `renderizar_escena()`: dibuja los objetos virtuales sobre el entorno físico.
- `dibujar_texto_escena()`: dibuja mensajes de texto sobre el entorno físico.

Módulo `gestorJuegosAR`

Encargado de la gestión pedagógica del sistema, coordinando los minijuegos y evaluando el desempeño del jugador. Su función principal es reproducir las actividades comunicándose con cada mundo y comunicar los resultados al **GameState** y al **MongoDBManager**.

Cada sesión se estructura como un conjunto de actividades interactivas, donde las respuestas del jugador son interpretadas por el `voice_recognition` y procesadas en función del contexto del **GameState**. Los resultados se almacenan en la base de datos.

Funciones principales:

- `procesar_comando()`: reconoce comando y cambia de fase en el juego.

7.4. Diseño de la base de datos

El diseño de la base de datos se centra en la simplicidad, la eficiencia y la trazabilidad del progreso del jugador. Para ello se emplea una estructura no relacional basada en **MongoDB**, donde toda la información relevante de cada usuario se almacena en un único documento dentro de la colección **users**. Esta decisión de diseño permite acceder y actualizar los datos del jugador de forma directa, minimizando la latencia y evitando la necesidad de múltiples consultas o relaciones entre colecciones.

Estructura de almacenamiento en MongoDB

Cada documento de la colección **users** representa de manera completa el perfil de un jugador, su configuración, progreso educativo y estado actual dentro del juego. La organización jerárquica del documento facilita la agrupación lógica de los diferentes mundos, minijuegos y recursos desbloqueados, como se muestra en el siguiente esquema:

Listing 7.1: Estructura del documento de usuario en la colección **users**

```
1 {
2   "_id": "nuria",
3   "nombre": "nuria",
4   "idioma": "es",
5   "vector_facial": [0.12, 0.45, 0.78, ...],
6   "fecha_registro": "2025-10-22T18:12:05.708441",
7   "mundos": {
8     "letras": { "adivina": 3, "memoria": 3, "secuencia": 3, "
9       total_estrellas": 9 },
10    "animales": {},
11    "frutas_verduras": {},
12    "numeros": {},
13    "final": {}
14  },
15  "estrellas_totales": 35,
16  "ludios": 90,
17  "disfraces": {
18    "disponibles": ["tina_unicornio"],
19    "equipado": "tina_unicornio"
20  },
21  "mundo_actual": "animales",
22  "minijuego_actual": "adivina",
23  "mundos_desbloqueados": {
24    "letras": true,
25    "animales": true,
26    "frutas_verduras": false,
27    "numeros": false,
28    "final": false
29  },
30  "fase": "menu_principal"
31 }
```

Finalmente, para tener una idea más concreta de cuales van a ser sus componentes, muestro a continuación el diagrama de componentes obtenido.

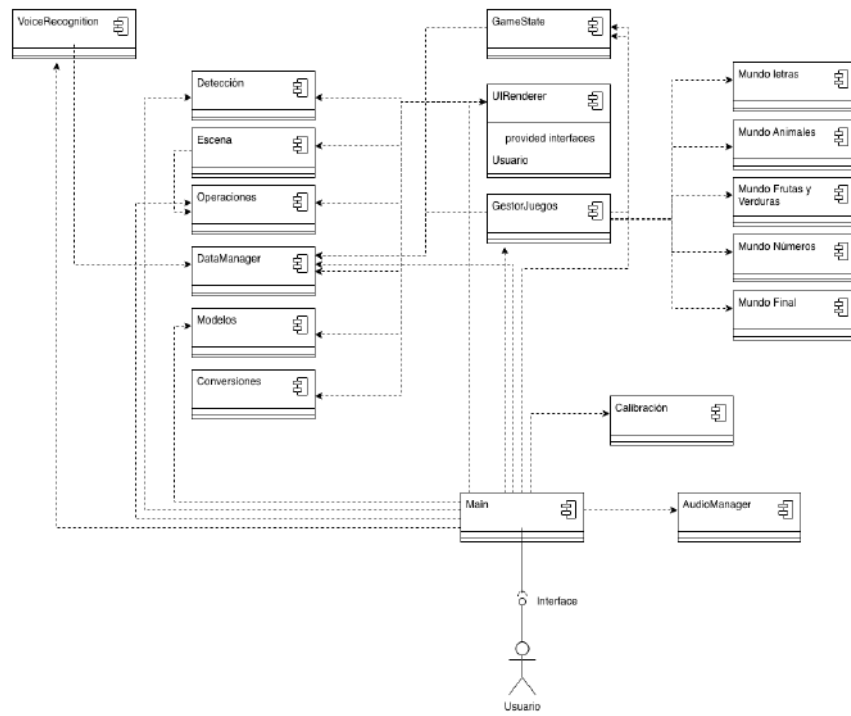


Figura 7.6: Diagrama de componentes y dependencias del sistema *Luminia*.

Capítulo 8

Implementación

8.1. Entorno y herramientas utilizadas

El desarrollo del sistema **Luminia** se ha llevado a cabo en un entorno modular y distribuido que combina tecnologías de visión por computador, reconocimiento de voz, renderizado 3D en tiempo real y almacenamiento persistente de datos. El objetivo principal de esta configuración es garantizar la escalabilidad, la estabilidad y la capacidad de análisis del comportamiento del usuario durante las sesiones educativas.

Lenguaje y frameworks principales

El sistema se implementa en **Python 3.13**, un lenguaje interpretado y multiplataforma ampliamente utilizado en proyectos de investigación, inteligencia artificial y educación interactiva. Su sintaxis y la disponibilidad de librerías específicas permiten el desarrollo rápido de prototipos interactivos sin comprometer la calidad técnica.

Las principales bibliotecas empleadas son:

- **OpenCV:** detección y calibración de marcadores ArUco, procesamiento de imagen y control del flujo de cámara.
- **SpeechRecognition** y **Pygame:** captura, preprocesamiento y transcripción de comandos de voz.
- **PyMongo:** conexión con la base de datos **MongoDB** para la lectura, escritura y actualización de registros.
- **Threading:** ejecución concurrente de hilos para gestionar de forma simultánea el reconocimiento de voz, la cámara y el renderizado.
- **Pygame mixer:** reproducción de audios y efectos sonoros de la narradora “Tina”.

- **NumPy**: cálculos matriciales y proyecciones 3D necesarias para la visualización aumentada.

Entorno de desarrollo

El desarrollo se ha realizado en el entorno **Visual Studio Code**, utilizando un entorno virtual de Python configurado con **venv**. El sistema de control de versiones empleado fue **Git**, con un repositorio público en **GitHub** para gestionar la trazabilidad y la evolución del código. Se puede encontrar en <https://github.com/mmnuria/luminia>

Infraestructura de base de datos

La base de datos **MongoDB** se despliega en un contenedor **Docker**, proporcionando un entorno aislado, reproducible y fácilmente desplegable en distintos equipos. El contenedor expone el puerto estándar 27017, permitiendo su conexión desde la aplicación y su monitorización mediante **MongoDB Compass**, herramienta empleada para la validación visual de registros durante las pruebas.

Esta configuración garantiza la independencia del entorno de desarrollo respecto al sistema operativo, mejorando la portabilidad y reduciendo los conflictos de dependencias.

Listing 8.1: Conexión con MongoDB desde el contenedor Docker

```
1 def __init__(self, uri="mongodb://root:example@localhost:27017/?
  authSource=admin", db_name="luminia_db", collection_name="users"):
2     if not hasattr(self, '_initialized'):
3         self.uri = uri
4         self.db_name = db_name
5         self.collection_name = collection_name
6         self.client = None
7         self.db = None
8         self.collection = None
9         self._initialized = True
10
11 def conectar(self):
12     if self.client is not None:
13         print("Ya est  conectado a MongoDB")
14         return
15     try:
16         self.client = MongoClient(self.uri, serverSelectionTimeoutMS
            =5000)
17         self.client.admin.command('ping')
18         print("Conexi n exitosa a MongoDB")
19         self.db = self.client[self.db_name]
20         self.collection = self.db[self.collection_name]
21     except ConnectionFailure as e:
22         print(f"Error al conectar a MongoDB: {e}")
23         raise
24     except Exception as e:
25         print(f"Error inesperado: {e}")
26         raise
```

El correcto funcionamiento de la conexión se verifica a través de **MongoDB Compass**, permitiendo observar las operaciones CRUD durante las pruebas.

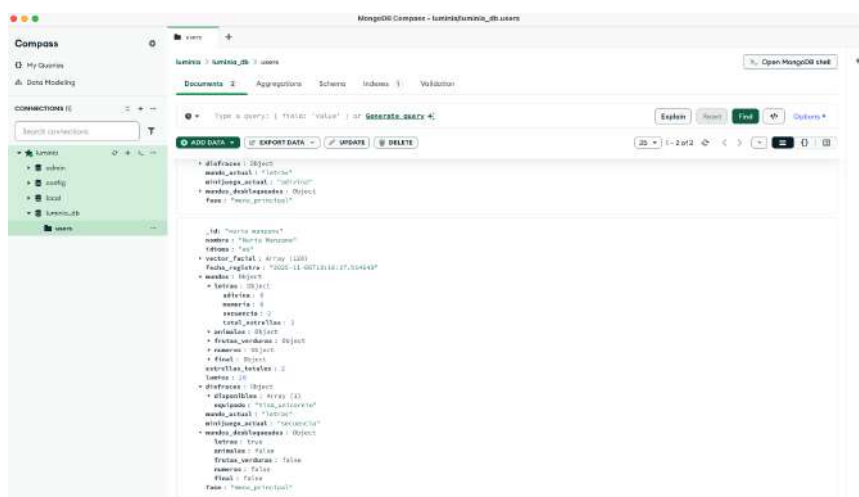


Figura 8.1: Muestra de un ejemplo de colección en MogoDB Compass.

8.2. Integración de tecnologías

La aplicación integra las distintas tecnologías mediante el módulo principal `main.py`, que coordina los subsistemas de entrada (voz y cámara), procesamiento (motor de juego y renderizado) y salida (audios, gráficos y base de datos). El diseño modular sigue un patrón concurrente, con hilos independientes sincronizados a través del objeto `GameState`.

La integración se organiza en tres niveles:

1. **Nivel de adquisición:** el módulo `ar.deteccion` captura el flujo de vídeo y detecta los marcadores ArUco mediante OpenCV (diccionario 5x5 de tamaño 18 mm). La calibración de cámara se obtiene desde `config.calibracion.json` que previamente se ha tenido que calibrar con la ejecución del archivo `config.calibracion.json`.
2. **Nivel de interacción:** el reconocimiento y los audios de Tina son gestionados por los módulos `voice_recognition` y `audio_manager`, respectivamente, los cuales trabajan en hilos separados y sincronizados mediante eventos.
3. **Nivel de persistencia y lógica:** la lógica del juego se maneja con `gestorJuegosAR`, que interactúa con `data_manager` para almacenar el progreso y los logros del jugador en la base de datos MongoDB.

Esta estructura permite que los hilos trabajen de forma simultánea sin interferir entre sí, garantizando un rendimiento fluido y tiempos de respuesta adecuados para la interacción infantil.

8.3. Desarrollo de los módulos principales

El sistema utiliza marcadores físicos fijos colocados sobre un tablero real Figura 8.2. Mediante la detección de estos marcadores con la cámara, los modelos 3D se muestran u ocultan dinámicamente según la fase activa del juego, la cual está gestionada en todo momento por el objeto `GameState`. La arquitectura modular permite una comunicación fluida entre los componentes de reconocimiento visual, gestión de voz, renderizado y persistencia de datos, garantizando la ejecución coordinada de todas las tareas concurrentes.

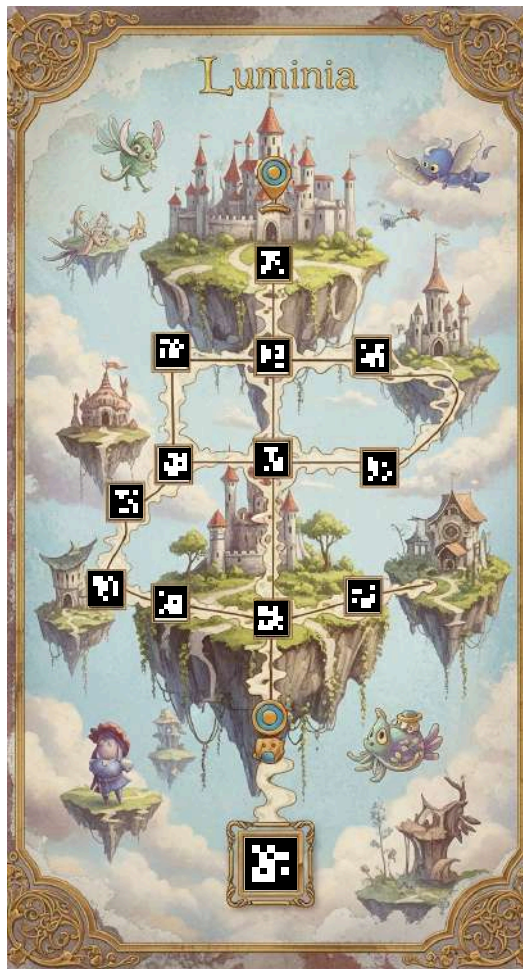


Figura 8.2: Tablero de juego.

8.3.1. Módulo principal main.py

El archivo `main.py` constituye el núcleo funcional del sistema y el punto de entrada de la aplicación. Su propósito es inicializar todos los componentes principales (la cámara, la base de datos, el motor de voz, el renderizador de realidad aumentada y la lógica de juego), garantizando su coordinación a lo largo de la ejecución. Actúa como un orquestador que sincroniza los hilos de procesamiento, el ciclo de renderizado y la comunicación entre los distintos subsistemas.

El fragmento siguiente resume las operaciones más importantes realizadas durante la inicialización:

Listing 8.2: Inicialización del sistema y componentes principales

```
1 mongo = MongoDBManager()
2 mongo.conectar()
3
4 state = GameState()
5 audio_manager = AudioManager(
6     on_talk_start=lambda: setattr(state, "microfono_listo", False),
7     on_talk_end=lambda: setattr(state, "microfono_listo", True)
8 )
9
10 cameraMatrix, distCoeffs = cargar_calibracion(ancho, alto)
11 detector = crear_detector()
12
13 ar = myVideo(cam, backend)
14 ar.process = lambda frame: realidad_mixta(frame.copy(),
15                                         detector, cameraMatrix,
16                                         distCoeffs, state, escenas)
```

Aunque más adelante se tratarán con más detalle, cabe mencionar que `main.py` también depende de varios módulos de configuración y utilidades que no intervienen directamente en la lógica principal pero, resultan esenciales para el correcto funcionamiento del sistema. Entre ellos se incluyen:

- `utils.operaciones.py`: backend encargado del renderizado 3D en tiempo real, de la manipulación de modelos GLTF e inicializar la escena.
- `config.calibracion`: encargado de recuperar o generar los parámetros de la cámara.
- `ar.deteccion`: responsable de la detección de marcadores ArUco y del cálculo de sus poses tridimensionales mediante la función `detectar_pose()`.
- `modules.ui_renderer`: módulo que integra los frames de vídeo con los modelos 3D según la fase activa del juego.
- `modules.gestorJuegosAR`: encargado de definir las reglas de progresión y coordinar las acciones entre voz, base de datos y renderizado.

- `modules.voice_recognition`: gestiona los procesos de escucha y análisis de audio en hilos separados.
- `models.modelos`: contiene las rutas y funciones de carga de los modelos 3D utilizados por las diferentes categorías (animales, frutas, letras, números, etc.).

En conjunto, el módulo actúa como un orquestador que mantiene la coherencia del sistema completo, asegura la comunicación entre los distintos subsistemas (voz, visión, persistencia y renderizado), gestiona el ciclo de vida de los hilos de ejecución y define el flujo principal de interacción del usuario con el entorno de realidad aumentada.

8.3.2. Módulo de gestión de datos `data_manager.py`

El módulo `data_manager.py` contiene la clase `MongoDBManager`, que implementa la capa de persistencia utilizada por la aplicación. Esta clase encapsula la conexión y las operaciones sobre MongoDB y está implementada como *singleton* para asegurar que exista una única instancia compartida durante la ejecución de la aplicación.

Descripción general La clase se inicializa con los parámetros de conexión: `uri`, `db_name` y `collection_name`. Por defecto utiliza una URI¹, la base de datos "luminia_db" y la colección `users`. El diseño se centra en garantizar una reconexión automática en caso de pérdida de la conexión y en ofrecer un conjunto de operaciones CRUD orientadas al manejo de usuarios y estadísticas básicas.

Patrón singleton La implementación usa el método especial `__new__` para mantener una única instancia (`_instance`) y un atributo `_initialized` para evitar re-inicializaciones posteriores en `__init__`.

Operaciones disponibles El módulo expone operaciones concretas centradas en la gestión de usuarios y algunas métricas asociadas:

- `crear_usuario(datos_usuario)`: inserta un nuevo documento de usuario. Usa `nombre.lower()` como `_id` para forzar búsquedas case-insensitive por nombre. Si el usuario ya existe, no inserta y devuelve `None`.
- `encontrar_usuario(nombre)`: devuelve el documento del usuario cuyo `_id` coincide con `nombre.lower()` o `None` si no existe.

¹"mongodb://root:example@localhost:27017/?authSource=admin"

- **actualizar_usuario(nombre, datos_actualizados)**: actualiza campos del usuario especificado (operación `update_one` con `$set`); devuelve `True` si hubo coincidencia y actualización, `False` en caso contrario.
- **eliminar_usuario(nombre)**: elimina el documento del usuario y devuelve `True` si se borró algún documento, `False` si no existía.
- **listar_usuarios()**: lista todos los usuarios devolviendo un listado con campos limitados (`_id`, `nombre` y `estrellas_totales`).
- **mundos_desbloqueados_usuario(nombre)**: obtiene la lista de mundos desbloqueados de un usuario si existe el campo correspondiente, o una lista vacía en caso contrario.
- **obtener_nombre(nombre), obtener_lumios(nombre)**: funciones auxiliares que recuperan valores puntuales del documento de usuario (`nombre`, `estrellas_totales` y `lumios`), devolviendo valores por defecto cuando el campo no existe.

Manejo de errores Cada método relevante invoca `asegurar_conexion()` al inicio para intentar garantizar que la operación se realiza sobre una conexión válida. Además, los métodos envuelven las llamadas a la base de datos en bloques `try/except` y, en caso de fallo, registran el error (mediante `print`) y devuelven un valor apropiado (`None`, `False` o listas vacías) para no propagar excepciones inesperadas a la lógica de alto nivel.

Fragmento representativo A continuación se incluye un extracto del código real que muestra la estructura básica y algunos métodos importantes:

Listing 8.3: Implementación representativa de `MongoDBManager`

```
1 from pymongo import MongoClient
2 from pymongo.errors import ConnectionFailure
3 import time
4
5 class MongoDBManager:
6     _instance = None
7
8     def __new__(cls, *args, **kwargs):
9         if cls._instance is None:
10             cls._instance = super(MongoDBManager, cls).__new__(cls)
11             return cls._instance
12
13     def __init__(self, uri="mongodb://root:example@localhost:27017/?
14         authSource=admin",
15         db_name="luminia_db", collection_name="users"):
16         if not hasattr(self, '_initialized'):
17             self.uri = uri
18             self.db_name = db_name
19             self.collection_name = collection_name
20             self.client = None
21             self.db = None
```

```

21         self.collection = None
22         self._initialized = True
23
24     def conectar(self):
25         if self.client is not None:
26             print("Ya esta conectado a MongoDB")
27             return
28         try:
29             self.client = MongoClient(self.uri,
30                                     serverSelectionTimeoutMS=5000)
31             self.client.admin.command('ping')
32             print("Conexion exitosa a MongoDB")
33             self.db = self.client[self.db_name]
34             self.collection = self.db[self.collection_name]
35         except ConnectionFailure as e:
36             print(f"Error al conectar a MongoDB: {e}")
37             raise
38         except Exception as e:
39             print(f"Error inesperado: {e}")
40             raise

```

8.3.3. Módulos: voice_recognition.py y audio_manager.py

El sistema de voz de la aplicación se divide en dos componentes principales, el módulo de reconocimiento de voz (`voice_recognition.py`) y el módulo para reproducir los audios (`audio_manager.py`). Ambos trabajan de forma coordinada para permitir la interacción natural con el asistente “Tina”, garantizando que no se produzcan interferencias entre la reproducción de audio y la escucha activa del micrófono.

Reconocimiento de voz (`voice_recognition.py`) Este módulo es el encargado de capturar y procesar las órdenes vocales del usuario. Su diseño se basa en el uso de hilos independientes (`threading.Thread`) para mantener una escucha continua sin bloquear el hilo principal del renderizado ni las tareas del juego.

Durante la inicialización, la función `inicializar_microfono()` configura el dispositivo de entrada y calibra el nivel de ruido ambiente utilizando las librerías de `SpeechRecognition`. Esto permite ajustar dinámicamente la sensibilidad del micrófono a las condiciones del entorno, mejorando la precisión del reconocimiento.

Una vez calibrado, el sistema lanza un bucle de escucha continua mediante la función `reconocimiento_voz()`, que utiliza un reconocedor asíncronico para detectar y procesar comandos. Cuando el usuario pronuncia una palabra o una orden reconocible, el módulo la envía al gestor de juegos (`GestorJuegosAR`) para su interpretación y ejecución. Este enfoque modular permite extender fácilmente el vocabulario o añadir nuevas acciones sin alterar la arquitectura general.

El siguiente fragmento muestra la estructura simplificada del bucle principal de escucha:

Listing 8.4: Bucle principal de escucha de comandos de voz

```
1 def reconocimiento_voz(state, gestor):
2     while True:
3         if state.microfono_listo:
4             comando = escuchar()
5             if comando:
6                 gestor.procesar_comando(comando)
```

Reproductor de audios (audio_manager.py) El módulo de reproductor de audios, gestiona la generación y reproducción de audios asociados a la narradora “Tina”. Se implementa en la clase **AudioManager**, la cual encapsula la lógica de reproducción de los audios y la sincronización con el micrófono.

El constructor de **AudioManager** recibe dos funciones de retorno (callbacks) denominadas **on_talk_start** y **on_talk_end**. Estas funciones se ejecutan antes y después de reproducir un audio, modificando el estado global (**state.microfono_listo**) para desactivar temporalmente el reconocimiento de voz mientras “Tina” habla. Este mecanismo evita que el sistema se escuche a sí mismo, garantizando una interacción fluida y libre de errores.

La reproducción de los audios se realiza de forma síncrona mediante la función **play_audio()**, que recibe la ruta del archivo y activa los eventos correspondientes antes y después de la reproducción. El siguiente fragmento muestra su implementación simplificada:

Listing 8.5: Gestión de sincronización entre voz sintética y micrófono

```
1 def play_audio(self, ruta_audio):
2     if self.on_talk_start:
3         self.on_talk_start()
4     playsound(ruta_audio)
5     if self.on_talk_end:
6         self.on_talk_end()
```

Integración y flujo de comunicación El flujo de interacción entre ambos módulos se basa en una comunicación indirecta a través del estado global (**GameState**) y del gestor de juegos. El módulo de reconocimiento de voz genera eventos de entrada que son interpretados por el **GestorJuegosAR**, mientras que el módulo de síntesis produce las salidas auditivas correspondientes. Gracias a la sincronización basada en los atributos del estado, ambos procesos funcionan en paralelo sin conflictos, incluso durante los cambios de fase o la reproducción simultánea de modelos 3D.

En conjunto, el subsistema de voz dota al entorno de una interfaz natural y accesible, donde el usuario puede comunicarse con el sistema utilizando lenguaje oral, sin necesidad de interacción manual con el dispositivo.

8.3.4. Módulo `ui_renderer`

El módulo `ui_renderer` constituye el componente de **realidad aumentada (RA)** del sistema. Su función principal es capturar el entorno real mediante la cámara y superponer sobre él los elementos visuales del juego, integrando así la interacción física y digital en tiempo real.

El módulo utiliza la librería `OpenCV` para acceder al flujo de vídeo y realizar la detección de marcadores **ArUco**, que actúan como puntos de anclaje para posicionar los modelos virtuales dentro de la escena. Cada marcador corresponde a elementos dentro del mundo de *Luminia*, lo que permite activar minijuegos o mostrar modelos específicos según la detección realizada.

Además, el módulo mantiene una comunicación constante con el objeto `GameState`, del cual obtiene información sobre el estado actual del juego, el mundo activo y el minijuego en ejecución. Esto permite actualizar la interfaz aumentada de forma coherente con las acciones del jugador y con las respuestas del sistema de voz.

Funcionamiento interno

El ciclo de ejecución del módulo se basa en un bucle continuo de renderizado que opera en un hilo independiente, asegurando una experiencia fluida sin afectar el rendimiento del resto de componentes. El flujo básico de procesamiento puede resumirse en los siguientes pasos:

1. Captura de imagen en tiempo real mediante la cámara integrada.
2. Conversión del fotograma al espacio de color apropiado para el análisis.
3. Detección de marcadores **ArUco** utilizando `cv2.aruco.detectMarkers()`.
4. Cálculo de la posición y orientación de cada marcador detectado.
5. Superposición de elementos visuales, textos o modelos 3D asociados a cada marcador.
6. Visualización del resultado final en una ventana de salida gestionada por `OpenCV`.

La estructura modular del código permite que las funciones de detección y renderizado puedan ampliarse o sustituirse en el futuro por librerías más avanzadas.

Sincronización con el resto del sistema

El módulo `ui_renderer` está diseñado para trabajar de forma sincronizada con el resto de componentes a través del `GameState`. Cuando un marcador

es detectado, se actualizan las variables globales que describen la escena, permitiendo que otros módulos (como el `gestorJuegosAR` o el `audio_manager`) reaccionen adecuadamente, por ejemplo, iniciando una narración o evaluando la interacción del usuario.

De esta forma, la interfaz aumentada no actúa como un componente visual aislado, sino como un *módulo interactivo* que refleja el estado interno del sistema y responde dinámicamente al progreso del jugador.

En conjunto, este módulo constituye el núcleo visual del sistema, permitiendo la fusión entre el mundo físico y los elementos digitales que caracterizan la experiencia educativa de *Luminia*.

8.3.5. Módulo gestorJuegosAR

El módulo `gestorJuegosAR` constituye el componente central de gestión pedagógica y lúdica del sistema. Su principal responsabilidad es coordinar los minijuegos de realidad aumentada, controlar la progresión del jugador y comunicar los resultados obtenidos con los módulos de voz y persistencia de datos.

Descripción general

Este módulo implementa la lógica que define el comportamiento de los distintos mundos y minijuegos del entorno educativo. Cada mundo (por ejemplo, *letras*, *animales* o *frutas y verduras*) contiene varios juegos asociados, cuyos resultados influyen en el progreso global del usuario dentro del sistema.

El `gestorJuegosAR` interactúa de forma continua con el `GameState`, accediendo a las variables que indican el estado actual del juego (fase, mundo activo, minijuego seleccionado, etc.) y modificándolas según la evolución de la sesión. De esta manera, el flujo de juego se mantiene sincronizado entre la interfaz visual y la respuesta auditiva de Tina.

Funciones principales

El módulo organiza su funcionamiento en torno a una serie de métodos y rutinas que estructuran el ciclo de vida de cada minijuego:

- `iniciar_minijuego()`: prepara el entorno y los recursos asociados al minijuego activo.
- `evaluar_respuesta()`: valida la interacción del jugador (por ejemplo, una selección o comando verbal) y determina si es correcta.
- `actualizar_progreso()`: incrementa la puntuación y actualiza las estrellas o recompensas obtenidas.

- `gestionar_resultado()`: comunica los resultados al `AudioManager` para emitir la retroalimentación auditiva correspondiente.
- `sincronizar_datos()`: envía los cambios al `MongoDBManager` para mantener la persistencia del progreso en la base de datos.

Estas funciones se ejecutan de forma dinámica, adaptándose a la fase del juego y al desempeño del usuario.

Interacción con otros módulos

El `gestorJuegosAR` actúa como puente entre los módulos de entrada (reconocimiento de voz y cámara) y los módulos de salida (reproductor de audios y renderizado). Cuando el jugador completa una acción o responde a un estímulo, este módulo interpreta el resultado, actualiza el estado global mediante el `GameState` y genera las señales necesarias para desencadenar respuestas visuales o auditivas.

En síntesis, el `gestorJuegosAR` es el encargado de unir los aspectos pedagógicos, lógicos y sensoriales del sistema. Su diseño modular permite añadir nuevos minijuegos o mundos sin alterar la arquitectura existente, manteniendo la coherencia del flujo de interacción entre el jugador y el entorno aumentado.

8.3.6. Clases de soporte para la realidad aumentada `myVideo`, `modeloGLTF` y `escenaPYGFX`

La correcta integración entre el entorno real y los modelos virtuales en el sistema **Luminia** se logra gracias a tres clases principales: `myVideo`, `modeloGLTF` y `escenaPYGFX`, definidas en el módulo `utils.operaciones.py`. Estas clases constituyen el núcleo técnico de la representación visual aumentada, encargándose respectivamente de la captura y procesamiento del flujo de vídeo, la gestión de modelos tridimensionales y la composición de escenas 3D sobre la imagen real de la cámara.

Clase `myVideo`

La clase `myVideo` abstrae la gestión del flujo de vídeo, permitiendo trabajar indistintamente con archivos locales o cámaras en tiempo real (USB o IP). Su estructura encapsula un objeto de `cv2.VideoCapture` y ofrece métodos para leer, reproducir y procesar cada fotograma. Una de sus características más relevantes es el atributo `process`, que permite asociar dinámicamente una función de procesamiento a cada frame capturado, lo que posibilita integrar la función `realidad_mixta()` del renderizador principal. De esta

forma, cada imagen capturada por la cámara se transforma en un lienzo ² sobre el que se proyectan los elementos virtuales del juego.

La clase también soporta vídeos pregrabados con sincronización temporal precisa mediante la tasa de fotogramas (**fps**) y un control del índice de cuadro (**CAP_PROP_POS_FRAMES**), lo que permite reproducir animaciones o escenarios de prueba en bucle.

Clase `modeloGLTF`

La clase `modeloGLTF` gestiona la carga, manipulación y animación de modelos tridimensionales en formato `.gltf`, un estándar ampliamente utilizado por su eficiencia y compatibilidad con texturas, materiales y esqueletos de animación. Internamente utiliza las funciones de `pygfx` y `pylinalg` para manejar las transformaciones geométricas y la interpolación de animaciones. Entre sus métodos principales destacan:

- `cargar(ruta_modelo)`: carga un modelo GLTF desde disco y selecciona su escena principal.
- `escalar()`, `rotar()` y `trasladar()`: aplican transformaciones locales sobre el modelo, adaptando su tamaño y orientación en función del marcador detectado.
- `flotar()`: ajusta la posición vertical del modelo para situarlo sobre el plano del marcador, corrigiendo desplazamientos por el centroide del objeto.
- `animar(nombre)`: activa una animación interna del modelo GLTF mediante el `AnimationMixer`.

Gracias a esta clase, los modelos 3D (animales, letras, frutas, etc.) pueden manipularse y animarse dinámicamente durante la experiencia de juego, sincronizados con las acciones de Tina o las respuestas del jugador.

Clase `escenaPYGFX`

La clase `escenaPYGFX` constituye el motor de renderizado de la aplicación. Crea una escena 3D a través del módulo `pygfx`, gestionando la cámara, la iluminación y la composición del modelo virtual sobre el flujo de vídeo real. Durante la inicialización, genera un lienzo offscreen mediante `WgpuCanvas` y un renderizador `WgpuRenderer`, lo que permite obtener cada frame ya combinado y con fondo transparente.

Entre sus métodos más destacados se encuentran:

²una copia del frame original

- `agregar_modelo()`: inserta un objeto GLTF en la escena y, si posee animaciones, las activa mediante un `AnimationMixer`.
- `iluminar()` e `ilumina_modelo()`: configuran múltiples fuentes de luz direccional y ambiental, mejorando el realismo de las sombras y la percepción volumétrica.
- `actualizar_camara(matriz)`: ajusta la posición y orientación de la cámara virtual según la pose detectada del marcador ArUco (calculada por `detectar_pose()`).
- `render()`: actualiza las animaciones y genera un frame renderizado que se combina con el vídeo de fondo.

Integración con las funciones de realidad aumentada

Estas tres clases trabajan en conjunto con las funciones auxiliares de realidad aumentada definidas en módulos complementarios:

- `crear_detector()`: inicializa el detector ArUco responsable de identificar los marcadores físicos en la imagen.
- `detectar_pose()`: calcula la posición y orientación (vectores `rvec` y `tvec`) de cada marcador, necesarios para posicionar correctamente el modelo 3D en la escena.
- `from_opencv_to_pygfx()`: convierte la matriz de pose de OpenCV a una matriz de transformación compatible con `pygfx`.
- `crear_escena()`: instancia una `escenaPYGFX`, asigna un modelo GLTF y ajusta el campo de visión de la cámara virtual en función de la calibración óptica.
- `mezclar_con_alpha()`: combina el frame del renderizador 3D con el vídeo real usando *alpha blending*, logrando la fusión visual característica de la realidad aumentada.

En conjunto, el flujo de procesamiento visual puede resumirse como:

1. Captura del frame de cámara mediante `myVideo`.
2. Detección del marcador y cálculo de pose con `detectar_pose()`.
3. Actualización de la cámara virtual de `escenaPYGFX` según la pose obtenida.
4. Renderizado del modelo GLTF con iluminación dinámica.
5. Composición del resultado sobre el vídeo real mediante `mezclar_con_alpha()`.

Este conjunto de clases y funciones constituye el **núcleo técnico de la experiencia aumentada de Luminia**, complementando a los módulos anteriores y garantizando que los modelos 3D aparezcan correctamente integrados en el entorno físico, alineados con los marcadores y sincronizados con las acciones narrativas y lúdicas del sistema.

Capítulo 9

Pruebas

9.1. Estrategia de pruebas

El proceso de validación del sistema **Luminia** se ha basado en una estrategia de pruebas iterativa, combinando distintas tipologías de evaluación con el objetivo de garantizar la estabilidad, fiabilidad y experiencia inmersiva de la aplicación.

Se definieron los siguientes tipos de pruebas:

- **Pruebas funcionales:** verifican que los módulos reconocimiento de voz y de realidad aumentada cumple con los requisitos establecidos.
- **Pruebas unitarias:** evalúan el comportamiento de los módulos `data_manager`, `game_state` y `gestorJuegosAR` de forma aislada, comprobando que las funciones internas principales devuelvan los resultados esperados.
- **Pruebas de integración:** garantizan que los módulos funcionan correctamente al comunicarse entre sí, especialmente en procesos que combinan voz, renderizado y lógica educativa.
- **Pruebas de usabilidad:** analizan la facilidad de interacción, la claridad de las instrucciones y la respuesta emocional de los usuarios. En concreto se evalúan con usuarios reales.
- **Pruebas de rendimiento:** miden el tiempo de respuesta de los módulos concurrentes, la latencia del reconocimiento de voz y la fluidez visual.

El conjunto de pruebas se realizó sobre la versión final desplegada en entorno local, utilizando los mismos recursos hardware que se emplearán en el uso real de la aplicación.

9.2. Pruebas funcionales

Se comprueba que los módulos cumplan los requisitos funcionales para los que fueron diseñados.

Módulo voice_recognition

Las pruebas se centraron en la detección y transcripción de comandos de voz mediante la biblioteca `SpeechRecognition`. Se comprobaron los comandos principales y la respuesta del sistema ante ausencia de entrada o ruido ambiental.

La validación se ha llevado a cabo observando en la consola los resultados generados por el hilo de escucha. En las imágenes siguientes, puede apreciarse a través de la etiqueta **[voz]** cómo el micrófono capta el audio mientras está activo. Asimismo, se observa que, al detectar un comando válido, el sistema responde adecuadamente, y cuando el comando no se entiende correctamente, también lo notifica.

```
[voz] escuchando en fase: reconocimiento_facial
[voz] detectado: 'iniciar sesión'
[voz] petición iniciar sesión por voz
[voz] escuchando en fase: reconocimiento_facial
[voz] detectado: 'ah registrarme'
- Transición: reconocimiento_facial -> esperando_nombre_registro
[voz] escuchando en fase: esperando_nombre_registro
[voz] detectado: 'nuria'
[registro] nombre detectado: 'Nuria'
Usuario encontrado: {'_id': 'nuria', 'nombre': 'nuria', 'idioma': 'es', 'vector_facial': [], 'fecha_registro': None, 'mundos': {'letras': {'adivina': 3, 'memoria': 3, 'secuencia': 3, 'total_estrellas': 9}, 'animales': {'adivina': 2, 'sonido': 3, 'clasificar': 3, 'total_estrellas': 8}, 'frutas_verduras': {'adivina': 3, 'color': 0, 'clasifica': 3, 'total_estrellas': 6}, 'numeros': {'adivina': 3, 'suma': 3, 'mayor': 3, 'total_estrellas': 9}, 'final': {'contar': 2, 'secuencia': 0, 'deletreo': 1, 'total_estrellas': 3}}, 'estrellas_totales': 35, 'ludios': 150, 'disfraces': {'disponibles': ['tina unicornio'], 'equipado': 'tina unicornio'}, 'mundo_actual': 'letras', 'mini juego actual': 'adivina', 'mundos_desbloqueados': {'letras': True, 'animales': True, 'fruta_y_verdura': True, 'numeros': True, 'final': True}, 'fase': 'menu_principal'})
[voz] escuchando en fase: esperando_nombre_registro
[voz] detectado: 'nuria manzano'
[registro] nombre detectado: 'Nuria Manzano'
[registro] nombre ok -> pedir idioma para Nuria Manzano
[voz] escuchando en fase: esperando_idioma_registro
[voz] detectado: 'español'
Usuario Nuria Manzano creado con _id: nuria manzano
```

Figura 9.1: Primera evidencia de reconocimiento de voz en ejecución.

```
[Gestor] Sincronizando datos de Nuria Manzano
[registro] usuario 'Nuria Manzano' registrado -> menu_principal
[voz] escuchando en fase: menu_principal
[voz] detectado: 'letras'
[menu] petición elección mundo: letras
Usuario Nuria Manzano actualizado
■ Estado COMPLETO guardado correctamente para Nuria Manzano
[Gestor] Entrando al Mundo de las Letras...
■ Bienvenido al Mundo de las Letras
Aquí aprenderás jugando con las letras mágicas del alfabeto.
Puedes decir: 'adivina', 'memoria' o 'secuencia' para comenzar un minijuego.
O di 'salir' para regresar al menú principal.
[GestorJuegosAR] Mundo 'letras' cargado y guardado en GameState.
[voz] escuchando en fase: mundo_letras
[voz] WaitTimeoutError (no se detectó audio)
[voz] escuchando en fase: mundo_letras
[voz] detectado: 'atrás'
[Gestor] Ya regresando al menú principal...
Usuario Nuria Manzano actualizado
■ Estado COMPLETO guardado correctamente para Nuria Manzano
mundo: salir -> menu_principal
[voz] escuchando en fase: menu_principal
[voz] detectado: 'animales'
[menu] petición elección mundo: animales
[Gestor] Todavía no has desbloqueado este mundo. ¡Consigue más estrellas para avanzar!
[voz] escuchando en fase: menu_principal
[voz] detectado: 'salir'
[menu] comando salir -> inicio
[voz] escuchando en fase: inicio
[voz] detectado: 'salir'
[voz] escuchando en fase: inicio
Cerrando aplicación...
Conexión cerrada
```

Figura 9.2: Segunda evidencia de reconocimiento de voz en ejecución.

Estos resultados confirman el correcto funcionamiento del micrófono, la estabilidad del hilo y la adecuada interpretación de comandos.

Módulo `ui_renderer`

Las pruebas de `ui_renderer` se centraron en verificar que los modelos 3D y elementos gráficos se renderizan correctamente dentro del entorno de realidad aumentada. Se ejecutaron pruebas de visualización en distintos escenarios, comprobando que las posiciones corresponden al estado del juego actual.

Para su verificación se ha usado un Patrón de Calibrado de marcadores ArUco que muestro en la siguiente Figura 9.3

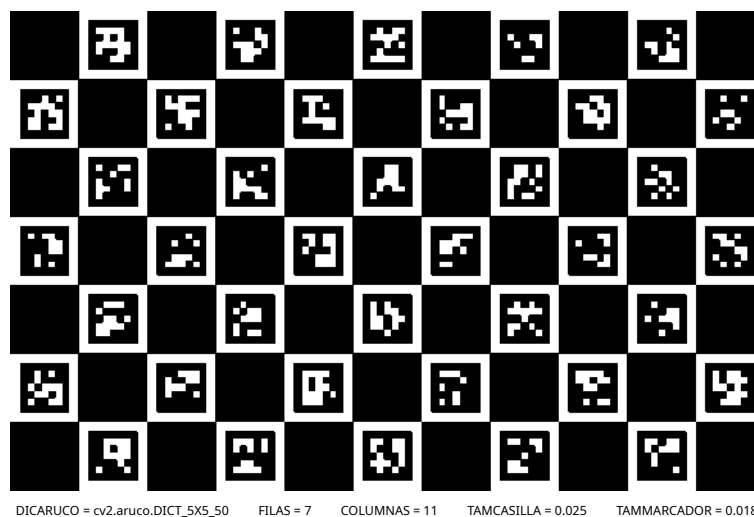


Figura 9.3: Patrón de Calibrado.

A través de esos marcadores se ha mostrado los diferentes elementos en 3D para corroborar su correcto funcionamiento:

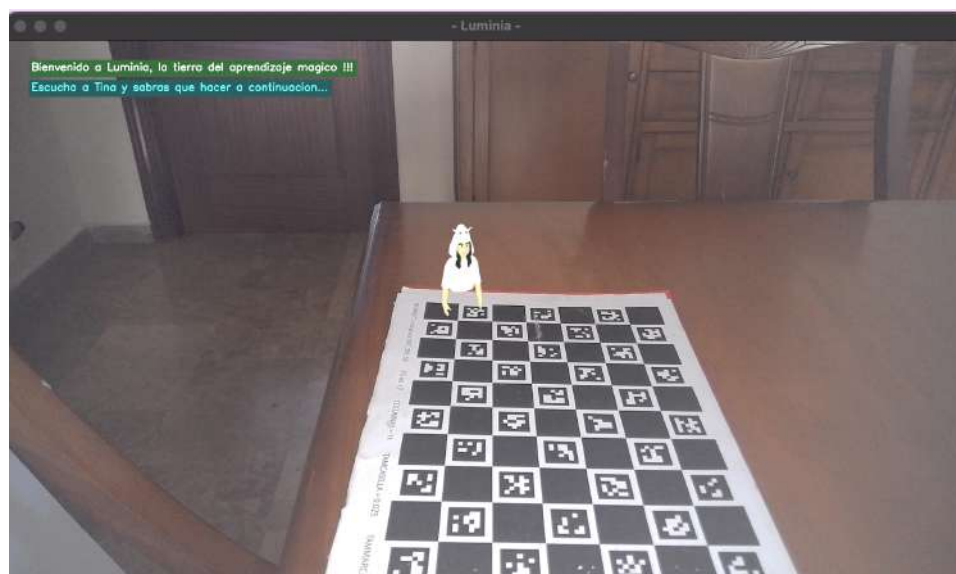


Figura 9.4: Visualización correcta del modelo 3D de la mascota Tina.

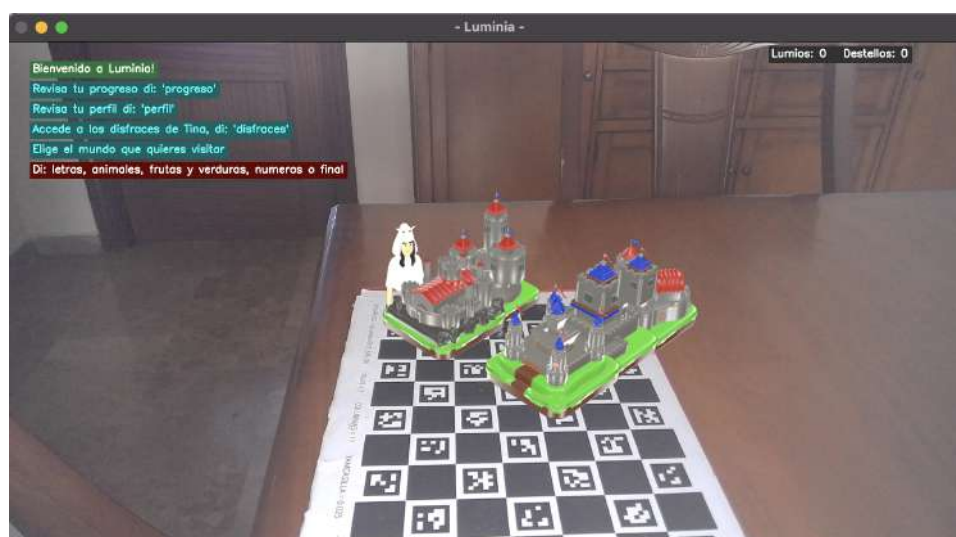


Figura 9.5: Visualización correcta de los modelos 3D en el menú principal.

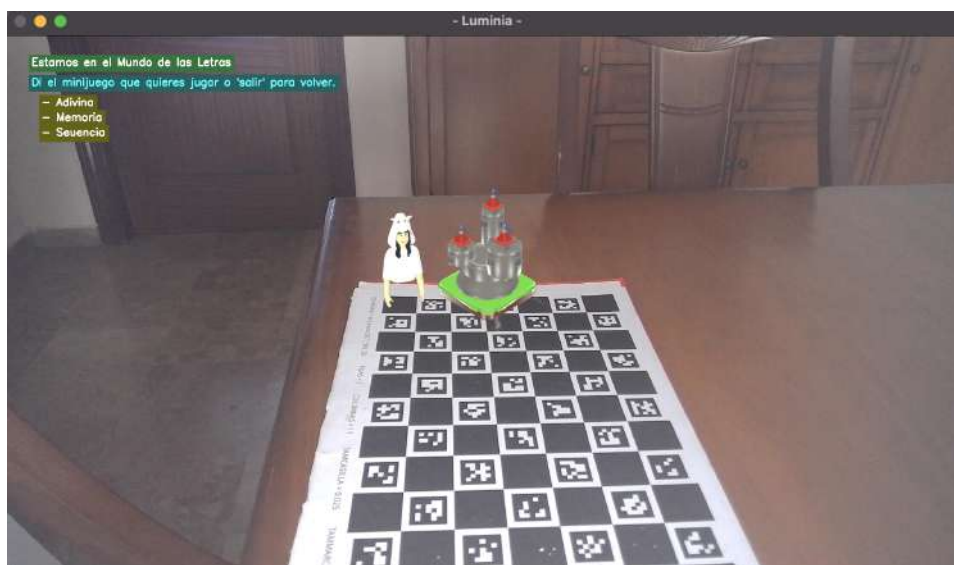


Figura 9.6: Visualización correcta de los modelos 3D en el mundo de las letras.

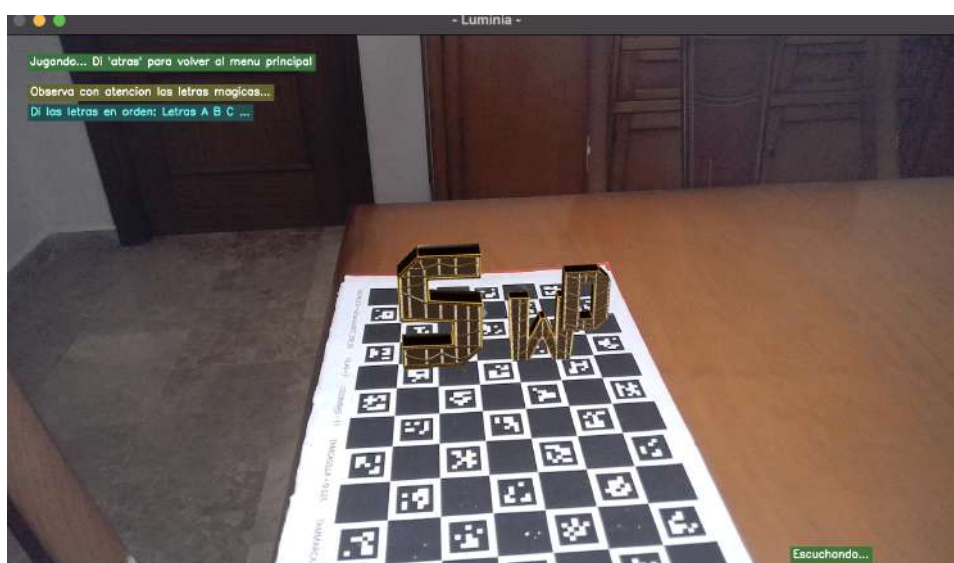


Figura 9.7: Visualización correcta de los modelos 3D en el juego de memoria del mundo de las letras.

En todos los casos, la representación visual ha sido coherente con las instrucciones del sistema, confirmando la correcta integración entre el reconocimiento de voz y el renderizado gráfico.

9.3. Pruebas unitarias

Las pruebas unitarias han tenido como objetivo garantizar que los módulos del sistema funcionaran correctamente de forma independiente, comprobando la fiabilidad de las funciones principales y la correcta comunicación entre los componentes del sistema.

Módulo `data_manager`

Se comprobaron las operaciones básicas de lectura, escritura y actualización sobre la colección `users`, que almacena toda la información del usuario (progreso, mundos desbloqueados, disfraces y estadísticas). Las pruebas confirmaron Figura 9.8 la persistencia correcta de los datos y la actualización coherente de los campos internos tras cada operación.

Listing 9.1: Ejemplo de prueba unitaria sobre el módulo de datos

```
1 import pytest
2 from modules.data_manager import MongoDBManager
3
4 @pytest.fixture
5 def manager():
6     return MongoDBManager()
7
8 def test_conexion_y_desconexion(manager):
9     assert manager.conectar() is True
10    assert manager.desconectar() is True
11
12 def test_lectura_usuario(manager):
13     usuario = manager.encontrar_usuario("nuria")
14     assert usuario is not None
15     assert "lumios" in usuario
16
17 def test_actualizacion_datos(manager):
18     resultado = manager.actualizar_usuario("nuria", {"lumios": 120})
19     assert resultado is True
```

Los resultados obtenidos mediante `pytest` fueron satisfactorios Figura 9.8, confirmando que las operaciones de conexión, lectura y escritura se completan sin errores de integridad.

Módulo `game_state`

Se ha verificado que los estados `inicio`, `menu_principal` y `fase_juego` se gestionan correctamente, así como la comunicación con otros módulos mediante variables globales y atributos compartidos.

Las pruebas unitarias comprobaron el estado inicial del juego y las transiciones de fase:

Listing 9.2: Pruebas sobre el control de estados del juego


```
1 from modules.game_state import GameState
2
3 def test_estado_inicial():
4     gs = GameState()
5     assert gs.mundo_actual == None
6
7 def test_transicion_fase_juego():
8     gs = GameState()
9     gs.establecer_fase("fase_juego")
10    assert gs.fase == "fase_juego"
```

Los resultados confirman una transición estable entre fases del juego sin bloqueos ni pérdidas de datos del usuario Figura 9.8 .

Módulo gestorJuegosAR

El módulo `gestorJuegosAR` centraliza el flujo general del juego, gestionando el cambio de mundos, la ejecución de minijuegos y la actualización de recompensas. Las pruebas unitarias validaron la correcta inicialización del gestor, la detección de comandos de voz, la carga de mundos y la transición a los minijuegos.

Listing 9.3: Pruebas unitarias del gestor de mundos y minijuegos

```
1 from modules.gestorJuegos import GestorJuegosAR
2
3 class DummyUI:
4     def render(self, *args, **kwargs):
5         pass
6
7 class DummyVoice:
8     def hablar(self, texto):
9         print(f"[Voz] {texto}")
10
11 class DummyGameState:
12     def __init__(self):
13         self.fase = "menu_principal"
14         self.mensaje_actual = ""
15         self.usuario_actual = "test_user"
16         self.usuario_data = {"estrellas_totales": 5}
17         self._progreso = {}
18
19     def establecer_fase(self, fase, **kwargs):
20         self.fase = fase
21
22     def registrar_resultado(self, mundo, minijuego, estrellas):
23         self._progreso[minijuego] = estrellas
24
25     def obtener_progreso(self, mundo):
26         return {"total_estrellas": 3, "adivina": 2}
27
28     def _verificar_desbloques(self):
29         pass
30
31
32 @pytest.fixture
33 def gestor():
34     ui = DummyUI()
```

```

35     voice = DummyVoice()
36     state = DummyGameState()
37     return GestorJuegosAR(ui, voice, state)
38
39 def test_inicia_minijuego(gestor):
40     """Comprueba que _iniciar_minijuego cambia la fase y registra el
41     juego."""
42     # Simulamos un mundo con un m todo iniciar_juego()
43     class DummyMundo:
44         def iniciar_juego(self, tipo):
45             print(f"[DummyMundo] Juego {tipo} iniciado.")
46
47     gestor.mundo_actual = DummyMundo()
48     gestor._iniciar_minijuego("adivina")
49
50     assert gestor.state.fase == "jugando"
51     assert gestor.juego_actual == "adivina"
52
53 def test_mostrar_mensaje(gestor):
54     """Verifica que el mensaje se guarda correctamente en el estado.
55     """
56
57     gestor._mostrar("Mensaje de prueba")
58     assert gestor.state.mensaje_actual == "Mensaje de prueba"
59
60 def test_salir_mundo(gestor):
61     """Verifica que al salir del mundo se vuelve al men principal.
62     """
63
64     gestor.mundo_actual = "dummy"
65     gestor._salir_mundo()
66     assert gestor.mundo_actual is None
67     assert gestor.state.fase == "menu_principal"
68
69 def test_registrar_resultado(gestor):
70     """Comprueba que se registran los resultados correctamente."""
71
72     gestor.registrar_resultado("letras", "adivina", 3)
73     assert gestor.state._progreso["adivina"] == 3

```

Los resultados de las pruebas confirmaron que el módulo maneja correctamente los cambios de fase, la carga dinámica de mundos y la gestión de puntuaciones. Figura 9.8 muestra la ejecución global de todas las pruebas unitarias.

```

===== test session starts =====
platform darwin -- Python 3.13.0, pytest-8.4.2, pluggy-1.6.0 -- /Users/mmurria/Desktop/TFG- Luminia -/tfg_env/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/mmurria/Desktop/TFG- Luminia -/tfg_env/Luminia
plugins: anyio=4.9.0
collected 9 items

test/test_data_manager.py::test_conexion_y_desconexion PASSED [ 11%]
test/test_data_manager.py::test_lectura_usuario PASSED [ 22%]
test/test_data_manager.py::test_actualizacion_datos PASSED [ 33%]
test/test_game_state.py::test_estado_inicial PASSED [ 44%]
test/test_game_state.py::test_transicion_fase_juego PASSED [ 55%]
test/test_gestorJuegos.py::test_inicia_minijuego PASSED [ 66%]
test/test_gestorJuegos.py::test_mostrar_mensaje PASSED [ 77%]
test/test_gestorJuegos.py::test_salir_mundo PASSED [ 88%]
test/test_gestorJuegos.py::test_registrar_resultado PASSED [100%]

===== warnings summary =====
../lib/python3.13/site-packages/face_recognition_models/__init__.py:7
/Users/mmurria/Desktop/TFG- Luminia -/tfg_env/lib/python3.13/site-packages/face_recognition_models/__init__.py:7: DeprecationWarning: pkg_resou
rces is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import resource_filename

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 9 passed, 1 warning in 4.91s =====

```

Figura 9.8: Ejecución de las pruebas unitarias en los módulos principales.

9.4. Pruebas de integración

Una vez validados los módulos de forma individual, se realizaron pruebas de integración para garantizar la correcta comunicación entre ellos. Estas pruebas fueron especialmente relevantes debido a la naturaleza concurrente del sistema y la necesidad de sincronizar la entrada de voz con la respuesta visual.

El flujo principal probado fue el siguiente:

1. El sistema inicia sesión con un usuario almacenado en la base de datos.
2. El módulo `voice_recognition` detecta un comando de voz (por ejemplo: “letras” para entrar en el Mundo de las Letras).
3. El `gestorJuegosAR` activa el mundo correspondiente y actualiza el estado global en `GameState`.
4. La interfaz `ui_renderer` muestra los elementos aumentados según la acción del jugador.
5. El módulo `voice_recognition` detecta otro comando de voz (por ejemplo: “adivina” para jugar a ese minijuego).
6. El `gestorJuegosAR` activa el minijuego correspondiente y actualiza el estado global en `GameState`.
7. La interfaz `ui_renderer` muestra los elementos aumentados según la acción del jugador.
8. Una vez terminado el juego, `data_manager` guarda el progreso alcanzado en la base de datos.

En la fase automatizada, se emplearon objetos simulados (*DummyUI* y *DummyVoice*) para reproducir el flujo completo de interacción entre los módulos. Estas pruebas permitieron verificar la correcta comunicación entre los componentes principales del sistema (`GestorJuegosAR`, `GameState` y `DataManager`) asegurando la transición adecuada entre fases del juego y la gestión coherente del estado global.

Listing 9.4: Prueba de integración con `pytest`

```
1 import pytest
2 from modules.gestorJuegos import GestorJuegosAR
3 from modules.game_state import GameState
4 from modules.data_manager import MongoDBManager
5
6
7 class DummyUI:
8     def render(self, msg):
9         print(f"[UI] {msg}")
```

```

10
11 class DummyVoice:
12     def escuchar(self):
13         comando = "letras"
14         print(f"[DummyVoice] Simulando comando: {comando}")
15         return comando
16
17 @pytest.mark.integration_db
18 def test_flujo_integracion_con_db():
19     """
20     Prueba el flujo completo usando una base de datos MongoDB real.
21     Requiere que el contenedor o servicio est  activo:
22
23         docker run -d -p 27017:27017 \
24             -e MONGO_INITDB_ROOT_USERNAME=root \
25             -e MONGO_INITDB_ROOT_PASSWORD=example \
26             mongo
27
28     """
29     # Conectar con MongoDB
30     db = MongoDBManager()
31     assert db.conectar() is True
32
33     # Crear o cargar usuario
34     nombre_usuario = "nuria"
35     usuario = db.encontrar_usuario(nombre_usuario)
36     if usuario is None:
37         usuario = {
38             "nombre": nombre_usuario,
39             "lunios": 0,
40             "estrellas_totales": 0,
41             "mundos": {
42                 "letras": {"adivina": 0, "memoria": 0, "
43                             total_estrellas": 0},
44                 "animales": {"adivina": 0, "memoria": 0, "
45                             total_estrellas": 0}
46             }
47         }
48         db.crear_usuario(usuario)
49
50     # Inicializar gestor y estado
51     gs = GameState()
52     gestor = GestorJuegosAR(DummyUI(), DummyVoice(), gs)
53     gs.usuario_actual = nombre_usuario
54     gs.usuario_data = usuario
55     gs.fase = "menu_principal"
56
57     # Simular flujo de interaccion
58     comando = "letras"
59     gestor.procesar_comando_voz(comando)
60     assert gs.fase.startswith("mundo_")
61
62     gestor._iniciar_minijuego("adivina")
63     assert gs.fase == "jugando"
64
65     # Registrar resultado y actualizar en BD
66     gestor.registrar_resultado("letras", "adivina", 3)
67     actualizado = db.actualizar_usuario(gs.usuario_actual, gs.
68         usuario_data)
69     assert actualizado is True
70
71     # Verificar que se guarda correctamente

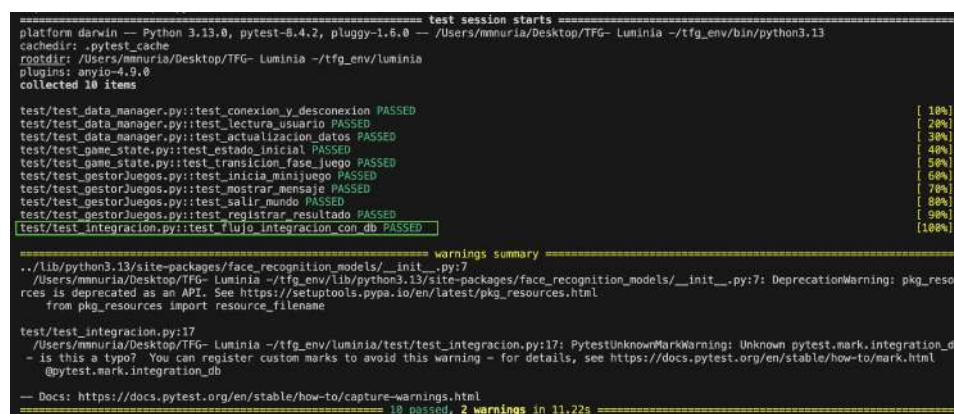
```

```

69     usuario_final = db.encontrar_usuario(nombre_usuario)
70     assert usuario_final is not None
71     assert usuario_final["estrellas_totales"] >= 3
72
73     # Desconectar
74     assert db.desconectar() is True

```

Los resultados demostraron una comunicación estable entre módulos y una respuesta sincronizada entre la entrada auditiva y la representación visual del entorno (Figura 9.9).



```

===== test session starts =====
platform darwin -- Python 3.13.0, pytest-8.4.2, pluggy-1.6.0 -- /Users/mmurria/Desktop/TFG- Luminia ~/tfg_env/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/mmurria/Desktop/TFG- Luminia ~/tfg_env/luminia
plugins: anyio-4.9.0
collected 10 items

test/test_data_manager.py::test_conexion_y_desconexion PASSED [ 10%]
test/test_data_manager.py::test_lectura_usuario PASSED [ 20%]
test/test_data_manager.py::test_actualizacion_datos PASSED [ 30%]
test/test_game_state.py::test_estado_inicial PASSED [ 40%]
test/test_game_state.py::test_transicion_fase_juego PASSED [ 50%]
test/test_gestorJuegos.py::test_inicia_minijuego PASSED [ 60%]
test/test_gestorJuegos.py::test_mostrar_mensaje PASSED [ 70%]
test/test_gestorJuegos.py::test_salir_mundo PASSED [ 80%]
test/test_gestorJuegos.py::test_registrar_resultado PASSED [ 90%]
test/test_integracion.py::test_flujo_integracion_con_db PASSED [100%]

===== warnings summary =====
../lib/python3.13/site-packages/face_recognition_models/__init__.py:7
/Users/mmurria/Desktop/TFG- Luminia ~/tfg_env/lib/python3.13/site-packages/face_recognition_models/__init__.py:7: DeprecationWarning: pkg_resou
rces is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import resource_filename

test/test_integracion.py:17
/Users/mmurria/Desktop/TFG- Luminia ~/tfg_env/luminia/test/test_integracion.py:17: PytestUnknownMarkWarning: Unknown pytest.mark.integration_db
- is this a typo? You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/how-to/mark.html
@pytest.mark.integration_db

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 10 passed, 2 warnings in 11.22s =====

```

Figura 9.9: Pruebas de integración entre módulos principales de Luminia.

9.5. Pruebas de usabilidad

Con el fin de evaluar la efectividad educativa y la calidad de la experiencia inmersiva, se realizaron pruebas con un pequeño grupo de niños de entre 4 y 6 años, en un entorno doméstico controlado. Las sesiones se llevaron a cabo con hijos de familiares y amigos cercanos, a quienes se les explicó el funcionamiento básico del juego y la dinámica de interacción con el asistente virtual Tina. Estas pruebas tuvieron un carácter exploratorio, observacional y no invasivo.

- Se presentó brevemente el sistema y se explicó el objetivo de los mini-juegos.
- Los niños tuvieron libertad para explorar el tablero y observar los modelos 3D a través del dispositivo.
- Posteriormente, se guió una partida siguiendo el flujo completo del juego, incluyendo el reconocimiento de voz y las fases de aprendizaje.
- Se observaron sus reacciones, nivel de atención y grado de interacción espontánea.

- Finalmente, se recogieron comentarios cualitativos de los padres sobre la experiencia general y posibles mejoras.

Resultados y observaciones

La acogida general fue muy positiva. Los niños mostraron entusiasmo desde el primer momento, mantuvieron la atención durante toda la sesión y demostraron curiosidad natural por explorar el entorno aumentado. Muchos de ellos giraban el tablero para ver cómo los modelos 3D se visualizaban desde diferentes ángulos, descubriendo detalles con gran interés. Tras una fase inicial de exploración libre, se logró completar el flujo del juego siguiendo el orden correcto, observándose altos niveles de motivación y disfrute durante las actividades propuestas.

Puntos de mejora identificados

A partir de la observación directa y los comentarios de los padres, se identificaron varias oportunidades de mejora:

- **Rendimiento gráfico:** durante la fase de ejecución, se detectaron momentos puntuales de congelación de la pantalla debido a la carga de renderizado en tiempo real, lo que afectó a la continuidad de la experiencia y la atención de los niños.
- **Expresividad del asistente virtual:** varios padres coincidieron en que la experiencia podría resultar más atractiva si el personaje Tina incorporase animaciones sencillas o microgestos, aportando mayor sensación de dinamismo y “vida” a la interacción.

En conjunto, los resultados indican que la propuesta cumple su objetivo de captar la atención infantil y fomentar el aprendizaje lúdico, ofreciendo una base sólida para futuras mejoras centradas en la optimización técnica y la interacción emocional con el asistente.

9.6. Pruebas de rendimiento

Las pruebas de rendimiento tuvieron como objetivo medir la eficiencia del sistema *Luminia* en condiciones reales de ejecución, verificando la latencia de respuesta, la estabilidad de los procesos concurrentes y la tasa de fotogramas en el renderizado de realidad aumentada.

Entorno de pruebas

Todas las pruebas se ejecutaron en un equipo **MacBook Pro (15 pulgadas, 2018)** con las siguientes especificaciones técnicas:

- **Procesador:** Intel Core i7 de 6 núcleos a 2.2 GHz y 16 GB de memoria DDR4 a 2400 MHz.
- **Gráficos:** Radeon Pro 555X (4 GB) e Intel UHD Graphics 630 (1536 MB), con soporte para OpenGL 4.6.
- **Cámara externa:** Razer Kiyo HD (1080p a 30 FPS) y micrófono de doble canal integrado.
- **Sistema operativo:** macOS Sequoia, versión 15.6.

Durante la ejecución se observaron pausas ocasionales de entre **1 y 2 segundos** al cargar simultáneamente varios modelos tridimensionales.¹

Metodología

Para la evaluación del rendimiento del sistema se implementó un script de diagnóstico que automatiza la captura de métricas de desempeño durante la ejecución real de la aplicación. El procedimiento combina mediciones internas del código (mediante el uso de `time.perf_counter()` y los módulos `psutil` y `OpenCV`) con observaciones del monitor de actividad del sistema operativo (*Activity Monitor* de macOS).

Durante la sesión, el script solicitó comandos de voz concretos (“letras”, “adivina” y “animales”) y, tras cada iteración, midió automáticamente los tiempos de respuesta y el rendimiento gráfico en tiempo real.

Se registraron tres indicadores principales:

1. **Latencia de reconocimiento de voz:** tiempo transcurrido desde que el usuario pronuncia el comando hasta que el sistema devuelve la transcripción o respuesta asociada. Esta métrica se obtuvo mediante diferencias de tiempo calculadas con `time.perf_counter()`, abarcando tanto la fase de escucha como la de procesamiento en el servicio de reconocimiento.
2. **Tasa de fotogramas por segundo (FPS):** promedio de cuadros renderizados por segundo durante la ejecución de escenas de realidad aumentada, capturado a través de `OpenCV`. Este valor refleja la fluidez visual de los modelos 3D.
3. **Uso de CPU y memoria RAM:** carga promedio del sistema durante las pruebas, medida mediante la biblioteca `psutil` y validada con las lecturas paralelas de *Activity Monitor*.

¹Estas interrupciones se consideran aceptables dentro del rango operativo, ya que no afectan la interacción general ni la respuesta del sistema a los comandos de voz.

Listing 9.5: Pruebas de rendimiento

```

1 import time
2 import csv
3 import psutil
4 import cv2
5 import numpy as np
6 import speech_recognition as sr
7
8 CSV_FILE = "resultados_rendimiento.csv"
9 DURACION_TEST = 30 # segundos para medir FPS
10 MUESTRAS_VOZ = ["letras", "adivina", "animales"]
11
12 def medir_latencia_voz(comando_esperado="letras"):
13     """Mide el tiempo desde el inicio de escucha hasta la respuesta
14     procesada."""
15     recognizer = sr.Recognizer()
16     with sr.Microphone() as source:
17         print("\n Diga el comando:", comando_esperado)
18         inicio = time.time()
19         audio = recognizer.listen(source)
20         fin_escucha = time.time()
21         try:
22             texto = recognizer.recognize_google(audio, language="es-ES")
23         except sr.UnknownValueError:
24             texto = "No reconocido"
25         fin_total = time.time()
26
27     latencia_escucha = fin_escucha - inicio
28     latencia_total = fin_total - inicio
29
30     print(f"Reconocido: {texto}")
31     print(f"Latencia escucha: {latencia_escucha:.2f}s | Latencia total
32           : {latencia_total:.2f}s")
33
34     return latencia_escucha, latencia_total, texto
35
36 def medir_fps(duracion=DURACION_TEST):
37     """Simula renderizado y mide FPS promedio usando OpenCV."""
38     print("\n Iniciando prueba de FPS...")
39     start = time.time()
40     frame_count = 0
41
42     while time.time() - start < duracion:
43         frame = np.zeros((480, 640, 3), dtype=np.uint8)
44         cv2.putText(frame, f"Frame {frame_count}", (50, 240),
45                     cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
46         cv2.imshow("Luminia - Prueba FPS", frame)
47         frame_count += 1
48
49         if cv2.waitKey(1) == ord('q'):
50             break
51
52     end = time.time()
53     fps = frame_count / (end - start)
54     cv2.destroyAllWindows()
55     print(f"FPS promedio: {fps:.2f}")
56     return fps
57
58 def medir_recursos():

```



```

59     """Obtiene uso actual de CPU y memoria."""
60     cpu = psutil.cpu_percent(interval=1)
61     memoria = psutil.virtual_memory().percent
62     print(f"CPU: {cpu:.1f}% | RAM: {memoria:.1f}%")
63     return cpu, memoria
64
65 if __name__ == "__main__":
66     print("\n Iniciando pruebas de rendimiento del sistema Luminia")
67
68     # Cabecera CSV
69     with open(CSV_FILE, "w", newline="") as f:
70         writer = csv.writer(f)
71         writer.writerow(["Comando", "Latencia_escucha_s", "
72                         "Latencia_total_s",
73                         "Texto_reconocido", "FPS_promedio", "CPU_%",
74                         "Memoria_%"])
75
76     for comando in MUESTRAS_VOZ:
77         lat_escucha, lat_total, texto = medir_latencia_voz(comando)
78         fps = medir_fps()
79         cpu, mem = medir_recursos()
80
81         with open(CSV_FILE, "a", newline="") as f:
82             writer = csv.writer(f)
83             writer.writerow([comando, lat_escucha, lat_total, texto,
84                             fps, cpu, mem])
85
86     print(f"\n Resultados guardados en: {CSV_FILE}")
87     print(" Pruebas finalizadas con exito.")

```

Cada ciclo de prueba generó un archivo de resultados con las métricas obtenidas, garantizando la reproducibilidad y comparación entre ejecuciones.

Comando	Latencia_escucha_s	Latencia_total_s	Texto_reconocido	FPS_promedio	CPU_%	Memoria_%
letras	2.158270835876465	2.789444923400879	letras	59.971041834109634	2.1	67.4
adivina	2.175213098526001	2.768488883972168	adivina	60.996834521854495	2.6	67.0
animales	2.0268139839172363	2.7055718898773193	animales	61.1276242268887	22.1	67.3

Figura 9.10: Pruebas de rendimiento entre módulos principales de Luminia.

Resultados obtenidos

- **Latencia promedio de reconocimiento de voz:** 2.75s (muy por debajo del límite máximo de 4s establecido en el requisito **RNF-2**).
- **Frecuencia de renderizado (FPS):** entre 59.9 y 61.1 FPS, manteniendo una fluidez constante y cumpliendo el requisito **RNF-3**.
- **Uso de CPU:** entre 2 % y 22 % según la complejidad de la escena, alcanzando los valores más altos durante la carga de los modelos 3D del mundo de los animales.

- **Uso de memoria:** estable alrededor del 67 % del total disponible (aproximadamente 2.3 GB utilizados por el proceso principal).

Los resultados confirman que el sistema *Luminia* mantiene un **rendimiento estable y eficiente** en un entorno de hardware de gama media como el utilizado (MacBook Pro 15" de 2018 con GPU Radeon Pro 555X y cámara Razer Kiyo HD). La **latencia media de 2.75 s** demuestra una respuesta ágil del módulo de reconocimiento de voz, dentro de los márgenes definidos por los requisitos no funcionales. Asimismo, el renderizado de los modelos 3D se mantiene en torno a **60 FPS**, garantizando una experiencia fluida en realidad aumentada.

Durante las pruebas se observaron pausas ocasionales en la carga inicial de los modelos 3D, atribuibles al procesamiento concurrente entre hilos de renderizado y voz.² En conjunto, se concluye que el sistema cumple con los objetivos de rendimiento establecidos, ofreciendo una experiencia inmersiva y estable en tiempo real.

²Las pausas fueron esporádicas, de entre 0.5 y 1 s.

Capítulo 10

Conclusiones y trabajos futuros

En este capítulo se presentan las conclusiones derivadas del desarrollo de la aplicación *Luminia*, así como las líneas de trabajo futuras que podrían mejorar o ampliar su alcance educativo y tecnológico. Las reflexiones que se recogen, parten de la evaluación de los resultados obtenidos durante el desarrollo, la validación técnica del sistema y el cumplimiento de los objetivos planteados al inicio del proyecto.

10.1. Cumplimiento de los objetivos

El objetivo general de este trabajo consistía en desarrollar una plataforma educativa inmersiva dirigida a niños y niñas de entre 4 y 6 años, que integrara tecnologías de realidad aumentada, interacción implícita, conciencia de contexto y gamificación, con el fin de ofrecer experiencias de aprendizaje significativas, activas y motivadoras.

A lo largo del desarrollo, este objetivo se ha alcanzado de forma satisfactoria. El sistema implementado permite a los usuarios interactuar con el entorno físico a través de la cámara y el micrófono, fomentando una participación multisensorial que combina exploración, movimiento y comunicación oral. La integración de los distintos módulos (motor de realidad aumentada, reconocimiento de voz, renderizado 3D, gestión de usuarios y base de datos) ha dado lugar a un prototipo funcional, estable y coherente con la fundamentación pedagógica inicial.

En cuanto a los objetivos específicos, se pueden destacar los siguientes logros:

- **Potenciar la atención y motivación infantil:** se ha logrado mediante la incorporación de estímulos visuales y auditivos dinámicos, el uso de colores vivos, retroalimentación inmediata y recompensas

simbólicas (destellos y lumios) que refuerzan la implicación y el progreso.

- **Facilitar el aprendizaje significativo:** el diseño de los mundos temáticos (números, animales, colores, etc.) y sus desafíos progresivos permite la construcción activa del conocimiento a través del juego y la exploración.
- **Modularización del sistema:** se ha estructurado la arquitectura del software en módulos independientes (voz, renderizado, base de datos, interfaz, lógica de juego), incorporando escalabilidad y mantenibilidad.
- **Diseño e implementación de base de datos:** el sistema permite gestionar usuarios, guardar su progreso y realizar seguimiento individualizado, manteniendo coherencia y persistencia entre sesiones.
- **Desarrollo de mundos temáticos educativos:** se han creado distintos módulos temáticos que se gestionan mediante un módulo central denominado "Gestor de Juegos", permitiendo incorporar tantos módulos de juego como se desee sin comprometer el funcionamiento del sistema.
- **Integración de gamificación y recompensas:** las mecánicas de lumios y destellos fomentan la perseverancia y el afán de superación.
- **Diseño de la interfaz y experiencia de usuario:** la interfaz final es sencilla, atractiva e intuitiva, con una navegación clara y adaptada al nivel cognitivo de la franja etaria objetivo.
- **Implementación de funcionalidades avanzadas:** se ha integrado un módulo de reconocimiento de voz que permite la interacción natural con el asistente virtual *Tina*, así como un sistema de detección de rostros para enriquecer la experiencia inmersiva, y la detección de marcadores para la visualización de modelos 3D.
- **Garantizar seguridad y privacidad:** no se almacena ningún dato sensible de los usuarios, además se almacenan de forma local y controlada, sin conexión con redes externas, lo que garantiza un entorno seguro para el público infantil.

Por tanto, puede afirmarse que el proyecto ha cumplido con los objetivos propuestos, tanto desde el punto de vista técnico como pedagógico, validando la hipótesis inicial de que las tecnologías inmersivas pueden emplearse de forma segura y educativa en el aprendizaje infantil.

10.2. Conclusiones generales

El desarrollo de *Luminia* ha demostrado que es posible integrar tecnologías de vanguardia (como la realidad aumentada y el reconocimiento de voz) en un entorno educativo infantil, sin perder de vista los principios del aprendizaje activo y significativo. El sistema ofrece una experiencia lúdica y educativa equilibrada, que favorece la motivación intrínseca, la exploración autónoma y el refuerzo positivo, al tiempo que contribuye al desarrollo de habilidades cognitivas (atención, memoria, clasificación) y motoras finas.

La validación funcional ha evidenciado que el reconocimiento de voz responde adecuadamente a comandos simples y que el sistema es capaz de proporcionar retroalimentación clara y comprensible, incluso cuando las órdenes no son reconocidas con precisión. Asimismo, el uso de la cámara y los modelos 3D en realidad aumentada genera un alto nivel de inmersión, manteniendo la atención de los niños y niñas durante la interacción.

A nivel técnico, la arquitectura modular desarrollada garantiza la escalabilidad del sistema y facilita la integración de nuevas funcionalidades, como la incorporación de nuevos mundos o modos de juego. A nivel educativo, *Luminia* se posiciona como una herramienta complementaria al aprendizaje tradicional, reforzando los contenidos curriculares de forma motivadora y segura.

En síntesis, este proyecto demuestra que la combinación de tecnología, narrativa y diseño pedagógico puede dar lugar a experiencias digitales que promueven la curiosidad, la autonomía y la participación activa, aportando un valor real a la educación infantil.

10.3. Limitaciones

Durante el desarrollo y las pruebas del sistema se identificaron algunas limitaciones que es necesario considerar:

- **Reconocimiento de voz sensible al entorno:** la precisión del módulo depende del nivel de ruido ambiental y de la pronunciación infantil, lo que puede generar errores puntuales en la detección de comandos.
- **Dependencia del hardware:** el rendimiento visual y la fluidez del renderizado 3D varían según la potencia del dispositivo utilizado.
- **Ausencia de multilingüismo:** el sistema se encuentra actualmente disponible solo en español.
- **Gestión de datos local:** aunque garantiza la privacidad, limita la posibilidad de monitorizar el progreso desde plataformas externas (por ejemplo, por parte de docentes o tutores).

- **Expresividad limitada del personaje:** aunque el asistente virtual *Tina* cumple una función comunicativa efectiva, su expresividad podría enriquecerse mediante animaciones faciales simples.

10.4. Propuestas de mejora

A partir de las limitaciones observadas, se plantean varias líneas de mejora a corto plazo:

- **Optimización del reconocimiento de voz:** emplear modelos acústicos más robustos y entrenados con voces infantiles para mejorar la precisión.
- **Optimización del rendimiento gráfico:** aplicar técnicas de reducción de polígonos y renderizado progresivo para mantener una tasa de frames estable en dispositivos de gama media.
- **Mayor expresividad del asistente virtual:** incorporar gestos, animaciones sencillas en *Tina* para aumentar la empatía y la conexión con los niños.

10.5. Líneas de trabajo futuras

Finalmente, se proponen varias líneas de desarrollo a medio y largo plazo que podrían expandir significativamente el alcance educativo y técnico del sistema:

1. **Implementación multilingüe:** extender la aplicación a otros idiomas (como inglés o francés), ajustando tanto los comandos de voz como los contenidos pedagógicos.
2. **Plataforma web de seguimiento:** desarrollar una interfaz para docentes o tutores que permita visualizar el progreso de los alumnos y personalizar la dificultad de los juegos de forma remota.
3. **Incorporación de nuevos mundos educativos:** crear entornos temáticos centrados en otras áreas (emociones, naturaleza, música, etc.) que amplíen el universo de aprendizaje de *Luminia*.
4. **Sistema de aprendizaje adaptativo:** emplear inteligencia artificial para ajustar dinámicamente el nivel de dificultad en función del desempeño y la respuesta emocional del usuario.
5. **Compatibilidad multiplataforma:** adaptar la aplicación a entornos móviles (Android, iOS) y de realidad mixta, aumentando su accesibilidad y alcance.

En conclusión, *Luminia* constituye una aportación relevante al campo de la educación infantil mediada por tecnología, al demostrar que la realidad aumentada, interacción implícita, conciencia de contexto y la gamificación pueden combinarse de manera efectiva y segura para promover aprendizajes activos y significativos. El proyecto sienta las bases para futuras investigaciones y desarrollos orientados a la creación de entornos educativos digitales más personalizados, expresivos y emocionalmente enriquecedores.

Capítulo 11

Glosario de términos

Este capítulo recoge los principales conceptos y acrónimos utilizados a lo largo del Trabajo Fin de Grado, junto con una breve descripción y sus referencias bibliográficas cuando corresponde.

Interacción hombre-máquina: disciplina que estudia cómo las personas interactúan con los sistemas computacionales a través de interfaces. Se centra en la usabilidad, accesibilidad y experiencia de usuario [24].

Computación ubicua (Ubicomp): paradigma que propone la integración de la computación en el entorno cotidiano de forma invisible y constante [78].

Computación pervasiva: concepto estrechamente relacionado con la computación ubicua, enfocado en la integración constante de la tecnología en la vida diaria [65].

Sistemas ciberfísicos (CPS): sistemas que combinan componentes computacionales con procesos físicos mediante redes y sensores [47].

GPS: sistema de posicionamiento global que permite determinar la ubicación geográfica mediante señales satelitales.

RFID: tecnología de identificación por radiofrecuencia utilizada para reconocer objetos mediante etiquetas electrónicas.

Interacción explícita: forma de comunicación en la que el usuario realiza acciones deliberadas para interactuar con un sistema.

Interacción implícita: interacción que ocurre cuando el sistema interpreta acciones del usuario sin instrucciones directas.

Biometría: conjunto de técnicas de reconocimiento basadas en características físicas o de comportamiento humano.

PCA (Análisis de Componentes Principales): método estadístico para reducir la dimensionalidad de los datos preservando la mayor cantidad de información posible [38].

Eigenfaces: técnica de reconocimiento facial basada en PCA, que representa rostros mediante componentes principales [75].

Análisis Discriminante Lineal (LDA): técnica de reducción de dimensionalidad que maximiza la separación entre clases [26].

Redes Neuronales Convolucionales (CNN): arquitectura de redes neuronales especialmente eficaz para el procesamiento de imágenes y visión por computador [45].

Redes Neuronales Profundas (DNN): modelos de aprendizaje profundo con múltiples capas para extraer representaciones jerárquicas de datos [30].

Redes Neuronales Recurrentes (RNN): redes diseñadas para procesar secuencias temporales, ampliamente utilizadas en procesamiento del lenguaje y voz [35].

Reconocimiento Automático del Habla (ASR): tecnología que convierte el habla humana en texto mediante modelos acústicos y de lenguaje.

Procesamiento del Lenguaje Natural (NLP): área de la inteligencia artificial que estudia la interacción entre computadoras y lenguaje humano [42].

Modelos Ocultos de Markov (HMM): modelos estadísticos que describen secuencias de estados ocultos observables indirectamente [63].

Deep Learning: subcampo del aprendizaje automático basado en redes neuronales profundas [46].

Modelos de N-gramas: modelos de lenguaje que predicen palabras basándose en secuencias previas [53].

Named Entity Recognition (NER): tarea del NLP que identifica y clasifica entidades nombradas en texto [56].

Realidad Aumentada (AR): tecnología que superpone elementos digitales sobre el entorno físico en tiempo real [7].

Realidad Virtual (VR): entorno generado por computadora que ofrece una experiencia inmersiva tridimensional [11].

Pokémon Go: videojuego basado en realidad aumentada que combina el mundo real con elementos virtuales.

SDK (Software Development Kit): conjunto de herramientas que permiten crear aplicaciones para una plataforma determinada.

ARKit (Apple) y ARCore (Google): kits de desarrollo para la creación de aplicaciones de realidad aumentada en iOS y Android, respectivamente.

SLAM (Simultaneous Localization and Mapping): técnica que permite a un dispositivo construir un mapa del entorno mientras estima su ubicación [12].

Marker-based AR: realidad aumentada que utiliza marcadores visuales para posicionar objetos virtuales.

Markerless AR: realidad aumentada sin marcadores físicos, basada en el reconocimiento del entorno.

Ingeniería STEM: acrónimo de Ciencia, Tecnología, Ingeniería y Matemáticas (Science, Technology, Engineering and Mathematics).

Interacción adaptativa: sistemas que ajustan su comportamiento según el contexto o el usuario.

Panóptico digital: concepto que describe la vigilancia y recopilación masiva de datos en entornos digitales [80].

Product backlog: lista priorizada de tareas o requisitos en metodologías ágiles de desarrollo [73].

UI (User Interface): interfaz de usuario, conjunto de elementos visuales y funcionales para interactuar con un sistema [57].

Eventos asíncronos: eventos que ocurren de manera independiente del flujo principal del programa.

Patrón de diseño: solución reutilizable a un problema común de diseño de software [27].

Singleton: patrón de diseño que restringe la instanciación de una clase a un único objeto.

Búsquedas case-insensitive: búsquedas que no distinguen entre mayúsculas y minúsculas [53].

Bibliografía

- [1] Art. 8 gdpr – conditions applicable to child’s consent in relation to information society services. General Data Protection Regulation (GDPR).
- [2] Ley orgánica 3/2020, de 29 de diciembre, por la que se modifica la ley orgánica 2/2006, de 3 de mayo, de educación (loe). <https://www.boe.es/eli/es/lo/2020/12/29/3/con>, 2020. Boletín Oficial del Estado, España.
- [3] Toca Boca AB. Toca life world, 2020.
- [4] Khan Academy. Khan academy kids, 2018. Aplicación educativa gratuita.
- [5] Khalid Alharbi and Sandeep Semwal. Face recognition using deep convolutional neural networks: A survey. In *IADIS International Conferences on Interfaces and Human Computer Interaction (IHCI)*, 2023.
- [6] David P. Ausubel, Joseph D. Novak, and Helen Hanesian. *Psicología educativa: Un punto de vista cognoscitivo*. Trillas, México D.F., segunda edición edition, 1976.
- [7] Ronald T Azuma. A survey of augmented reality. *Presence*, 6(4):355–385, 1997.
- [8] Jorge Bacca, Silvia Baldiris, Ramón Fabregat, Sabine Graf, and Kinshuk. Augmented reality trends in education: A systematic review of research and applications. *Educational Technology & Society*, 17(4):133–149, 2014.
- [9] Peter N. Belhumeur, Joao P. Hespanha, and David J. Kriegman. Eigenfaces vs. fisherfaces: Recognition using class specific linear projection. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 19, pages 711–720, 1997.
- [10] David F. Bjorklund and Kayla B. Causey. *Children’s Thinking: Cognitive Development and Individual Differences*. SAGE Publications, Los Angeles, 6th edition, 2017.

- [11] Grigore Burdea and Philippe Coiffet. *Virtual Reality Technology*. John Wiley & Sons, 2003.
- [12] Cesar Cadena et al. Past, present, and future of slam. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016.
- [13] John M. Carroll. Human-computer interaction: Psychology as a science of design. *Annual Review of Psychology*, 48:61–83, 1997.
- [14] Bernard Casey. *Early Childhood Education: Principles and Practice*. Routledge, London, 2005.
- [15] Thomas P. Caudell and David W. Mizell. Augmented reality: An application of heads-up display technology to manual manufacturing processes. *Proceedings of the Hawaii International Conference on System Sciences*, 30(2):659–669, 1992.
- [16] Shilin Chen and Yung-Sheng Chen. Context-aware writing in ubiquitous learning environments. *International Journal of Advances in Engineering and Technology*, 3(1):53–61, 2011.
- [17] Children and Screens Organization. The children and screens guide for early child development and media use: Infants and children ages 0-5. 2024. Online resource, Children Screens.
- [18] G. G. Chowdhury. Natural language processing. *Annual Review of Information Science and Technology*, 37:183–228, 2003.
- [19] The Pokémon Company. Pokémon smile, 2020.
- [20] Agencia Española de Protección de Datos (AEPD). Nota técnica: Internet seguro por defecto para la infancia. Technical report, AEPD, 2023.
- [21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.
- [22] Abowd G. D. Salber D. Dey, A. K. A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications. *Human-Computer Interaction*, (16):97–166, 2001.
- [23] Anind K. Dey and Gregory D. Abowd. Understanding and using context. *Personal and Ubiquitous Computing*, 5(1):4–7, 2001.

- [24] Alan Dix, Janet Finlay, Gregory Abowd, and Russell Beale. *Human-computer interaction*. Pearson Education, 2003.
- [25] David Finkelhor. Teaching privacy: A flawed strategy for children's online safety. *Child Abuse & Neglect*, 114:104928, 2021.
- [26] Ronald A Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.
- [27] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [28] Susan E. Gathercole and Graham J. Hitch. Effects of word length on young children's memory performance. *Journal of Memory and Language*, 36(6):628–643, 1997.
- [29] A. J. Goldstein, L. D. Harmon, and A. B. Lesk. Identification of human faces. *Proceedings of the IEEE*, 59(5):748–760, 1971.
- [30] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [31] Guodong Guo, Nan Zhang, and Nan Zhang. A survey on deep learning based face recognition. *Computer Vision and Image Understanding*, 189:102805, 2019.
- [32] Juho Hamari, Jonna Koivisto, and Harri Sarsa. Does gamification work? — a literature review of empirical studies on gamification. In *2014 47th Hawaii International Conference on System Sciences*, pages 3025–3034. IEEE, 2014.
- [33] Trina Hinkley, Jennie McNeill, Valerie Carson, and Megan Teychenne. Young children's screen habits and first-time parents' reflections on screen use in socioeconomically disadvantaged swedish settings: a mixed methods study. *BMC Public Health*, 24:2027, 2024.
- [34] Geoffrey Hinton, Li Deng, Dong Yu, George Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, et al. Deep neural networks for acoustic modeling in speech recognition. *IEEE Signal Processing Magazine*, 29(6):82–97, 2012.
- [35] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [36] Originator Inc. Endless alphabet / endless numbers, 2017. Aplicación educativa.
- [37] Tangible Play Inc. Osmo learning system, 2018.

- [38] Ian T Jolliffe. *Principal Component Analysis*. Springer, 2002.
- [39] Wendy Ju and Larry Leifer. The design of implicit interactions: Making interactive systems less obnoxious. In *Proceedings of the Sixth International Conference on Design and Decision Support Systems in Architecture and Urban Planning*, 2006.
- [40] B. H. Juang and L. R. Rabiner. The past, present, and future of speech processing. *IEEE Signal Processing Magazine*, 14(5):24–48, 1997.
- [41] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Prentice Hall, 2000.
- [42] Daniel Jurafsky and James H Martin. *Speech and Language Processing*. Prentice Hall, 2000.
- [43] T. Khampumee, T. Intajuk, R. Rattanaphong, W. Kerdmanee, and S. Intharak. Effects of team-based ubiquitous learning model on students’ achievement and creative problem-solving abilities. *Cogent Education*, 11(1):2303550, 2024.
- [44] Thomas Kleber, Cheng Li, Qi Zhang, Yan Li, Joon-Seok Kim, and Feng Lu. Slam in the era of deep learning: A survey. *ACM Computing Surveys (CSUR)*, 53(4):1–35, 2020.
- [45] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. 2012.
- [46] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [47] Edward A Lee. Cyber-physical systems: Design challenges. In *11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*, pages 363–369. IEEE, 2008.
- [48] QuiverVision Limited. Quiver - 3d coloring app, 2019.
- [49] Sonia Livingstone et al. Children’s data and privacy online: growing up in a digital age. *LSE Research Online Working Paper*, 2019.
- [50] AR Flashcards LLC. Ar flashcards, 2015.
- [51] Google LLC. Toontastic 3d, 2017.
- [52] Sheri Madigan et al. Early childhood screen use contexts and cognitive and psychosocial outcomes: A systematic review and meta-analysis. *JAMA Pediatrics*, 2024. e-pub ahead of print.

- [53] Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [54] Katherine A. Marchak and D. Geoffrey Hall. Children’s understanding of proper names and descriptions. *Journal of Child Language*, 49(6):1295–1306, 2022.
- [55] Brad A. Myers. A brief history of human computer interaction technology. In *ACM interactions*, volume 5, pages 44–54, 1998.
- [56] David Nadeau and Satoshi Sekine. A survey of named entity recognition and classification. *Linguisticae Investigationes*, 30(1):3–26, 2007.
- [57] Donald A Norman. *The design of everyday things*. Basic Books, 2013.
- [58] OECD. Children in the digital environment: Revised typology of risks, 2021.
- [59] Age of Learning Inc. Abcmouse, 2016. Plataforma de aprendizaje infantil.
- [60] Jean Piaget. *The origins of intelligence in children*. International Universities Press, New York, 1952.
- [61] Jan L. Plass, Bruce D. Homer, and Charles K. Kinzer. Foundations of game-based learning. *Educational Psychologist*, 50(4):258–283, 2015.
- [62] PricewaterhouseCoopers (PwC). How vr is creating a new corporate training experience. 2020. Report.
- [63] Lawrence R Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.
- [64] Iulian Radu. Augmented reality in education: a meta-review and cross-media analysis. *Personal and Ubiquitous Computing*, 18(6):1533–1543, 2014.
- [65] Debashis Saha and Amitava Mukherjee. Pervasive computing: A paradigm for the 21st century. In *Computer*, volume 36, pages 25–31. IEEE, 2003.
- [66] Marc Ericson C. Santos, Takafumi Taketomi, Goshiro Yamamoto, Jun Miyazaki, Tomohiro Kato, and Hirokazu Kato. Exploring the educational potential of augmented reality in children’s science learning. *Interactive Learning Environments*, 24(5):1–23, 2016.

- [67] Maria Emilia Santos, Sara Lynce, Susana Carvalho, Margarida Cabela, and Ana Mineiro. Mean length of utterance–words in children with typical language development aged 4 to 5 years. *Revista CEFAC*, 17(4):1143–1151, 2015.
- [68] M. Satyanarayanan. Pervasive computing: Vision and challenges. In *IEEE Personal Communications*, volume 8, pages 10–17, 2001.
- [69] Ben Shneiderman, Catherine Plaisant, Max Cohen, Steven Jacobs, Niklas Elmqvist, and Nicholas Diakopoulos. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Pearson, 6 edition, 2016.
- [70] Massimo Stella, Nicole M. Beckage, and Markus Brede. Multiplex lexical networks reveal patterns in early word acquisition in children. *Scientific Reports*, 6:19264, 2016.
- [71] Glenn Stockwell and Yijen Wang, editors. *The Cambridge Handbook of Technology in Language Teaching and Learning*, chapter Impact of Context. Cambridge University Press, Cambridge, UK, 2025.
- [72] Ivan E. Sutherland. A head-mounted three dimensional display. *Proceedings of the December 9-11, 1968, fall joint computer conference, part I on - AFIPS '68 (Fall, part I)*, 1968.
- [73] Jeff Sutherland. *Scrum: The Art of Doing Twice the Work in Half the Time*. Currency, 2014.
- [74] Rebecca Treiman, Brett Kessler, and Derrick Bourassa. Children’s own names influence their spelling. *Applied Psycholinguistics*, 22(4):555–570, 2002.
- [75] Matthew Turk and Alex Pentland. Face recognition using eigenfaces. *Proceedings CVPR*, 1991.
- [76] Matthew A. Turk and Alex P. Pentland. Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1):71–86, 1991.
- [77] M. Weiser. The computer for the 21st century. *Scientific American*, 265(3):94–104, 1991.
- [78] Mark Weiser. The computer for the 21st century. *Scientific American*, 265(3):94–104, 1991.
- [79] Rabia M. Yilmaz. Educational magic toys developed with augmented reality technology for early childhood education. *Computers in Human Behavior*, 54:240–248, 2016.

-
- [80] Shoshana Zuboff. *The Age of Surveillance Capitalism*. Profile Books, 2019.
- [81] E.Ë. Şimşek et al. The use of augmented reality-supported activities in environmental education for children: Effects on learning and motivation. *Sustainability*, 16(23), 2024.

Apéndice A

Manual de Usuario

A.1. Requisitos

Como se ha explicado en el apartado de requisitos de este mismo documento, los requisitos de sistema necesarios para ejecutar Luminia son:

Requisitos hardware

- Equipo con procesador de al menos 2.2 GHz y 8 GB de RAM.
- Cámara HD (mínimo 720p) y micrófono.
- GPU deberá ser compatible con OpenGL 3.0 o superior para soportar renderizado RA.

Requisitos software

- El sistema deberá ser compatible con Windows 10, Ubuntu 22.04 LTS o macOS 13.
- Deberá ejecutarse sobre Python 3.13 con las librerías OpenCV, SpeechRecognition, PyAudio, PyMongo, Threading y NumPy.
- Deberá integrarse con una base de datos MongoDB gestionada en un contenedor Docker.

A.2. Instalación y ejecución

Para preparar el equipo y ejecutar *Luminia* correctamente, se deben seguir los siguientes pasos:

1. **Crear un entorno virtual:** Se recomienda instalar y ejecutar el proyecto dentro de un entorno virtual de Python. Para crearlo y activarlo, utilice los siguientes comandos:

```
python3 -m venv prueba_env  
source prueba_env/bin/activate
```

2. **Clonar el repositorio:** Descargue el código fuente en el entorno virtual desde GitHub:

```
git clone https://github.com/mmnuria/luminia
```

3. **Instalar dependencias:** Instale las dependencias necesarias ejecutando:

```
pip install -r requirements.txt
```

Este comando instalará automáticamente todas las librerías requeridas por el proyecto.

4. **Calibración de la cámara:** Para una correcta ejecución de la aplicación, es necesario calibrar la cámara:

- a) Imprima el patrón de calibración disponible en `config/charuco.tiff`, respetando sus dimensiones originales.
- b) Ejecute el siguiente comando en la terminal o en su entorno Python:

```
python calibrar_camara.py
```

- c) Mantenga el tablero frente a la cámara, plano y visible. La ventana mostrará cuántas imágenes se han capturado. Pulse ESC si desea finalizar antes de llegar al número máximo de capturas.
 - d) Al finalizar, se generará automáticamente el archivo `camara.py` con los parámetros de calibración de la cámara, listos para ser usados por la aplicación.
5. **Crear el contenedor Docker de la base de datos:** Ejecute el siguiente comando para levantar un contenedor MongoDB:

```
docker run -d -p 27017:27017 \  
-e MONGO_INITDB_ROOT_USERNAME=root \  
-e MONGO_INITDB_ROOT_PASSWORD=example \  
mongo
```

6. **Ejecutar la aplicación:** Con el entorno virtual activado y la cámara calibrada, ejecute la aplicación:

```
python main.py
```

La aplicación se iniciará correctamente y estará lista para su uso.

A.3. Descripción

En esta sección se describe el funcionamiento del sistema. El usuario puede interactuar mediante comandos hablados para navegar entre menús, iniciar sesión, registrarse, seleccionar mundos de juego o modificar su perfil. A continuación, se detallan las fases del sistema y los comandos disponibles en cada una.

A.3.1. Fase: Inicio

En esta fase se escucha la introducción de Tina y aparecen los primeros mensajes de bienvenida.

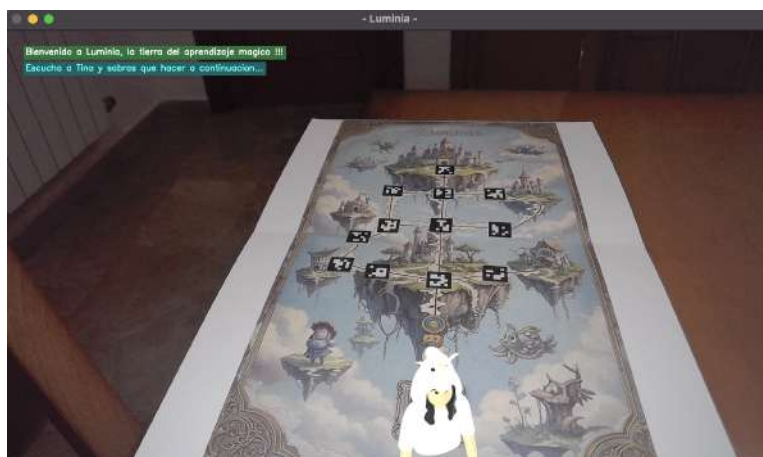


Figura A.1: Pantalla de inicio.

A.3.2. Fase: Reconocimiento Facial

En esta fase el sistema espera comandos para iniciar sesión o registrar un nuevo usuario. Los comandos válidos son:

- **“iniciar sesión”** o **“iniciar sesion”**: intenta identificar al usuario mediante reconocimiento facial.

- “registrarme” o “crear cuenta”: inicia el proceso de registro.

Mensajes del sistema:

- Si no se detecta rostro: “No se detectó la cara. Mira a la cámara.”
- Si el rostro no está registrado: “Cara no registrada. Di ‘registrarme’ para crear cuenta.”

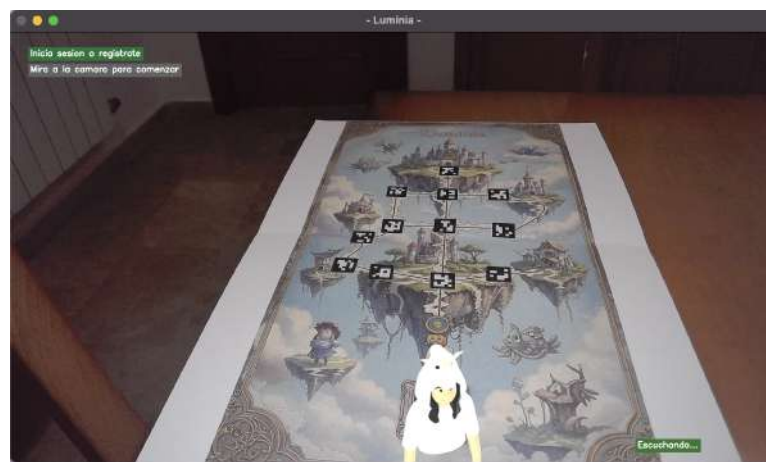


Figura A.2: Pantalla de reconocimiento facial.

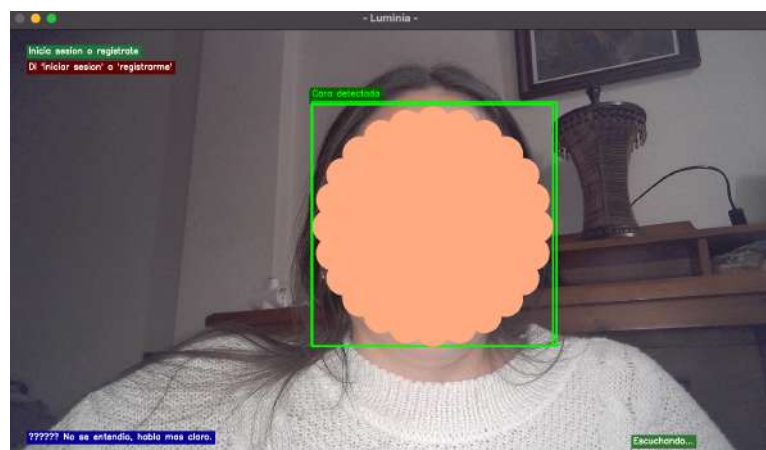


Figura A.3: Pantalla de reconocimiento facial con detección del rostro.

A.3.3. Fase: Registro de usuario

El proceso de registro tiene dos subfases:

1. Ingreso de nombre

El usuario dice su nombre. Ejemplo: “Mi nombre es Ana” o simplemente “Ana”.

Mensajes del sistema:

- Si el nombre ya existe: “El nombre ‘Ana’ ya existe. Di otro nombre.”
- Si el nombre es válido: pasa a la fase de idioma.

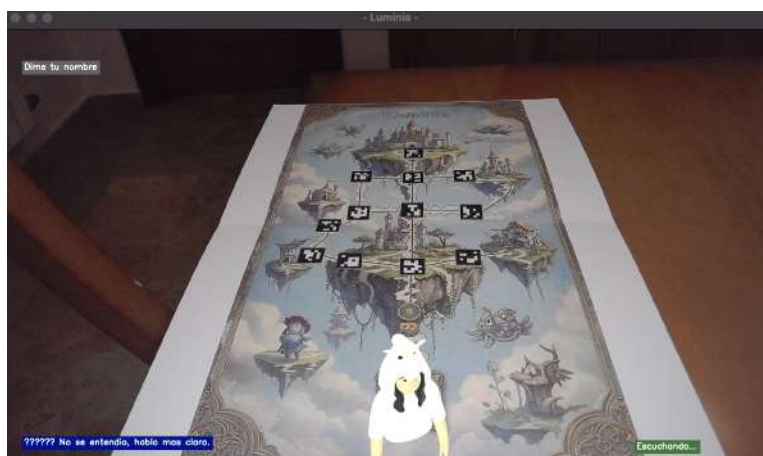


Figura A.4: Fase de registro: introducción del nombre.

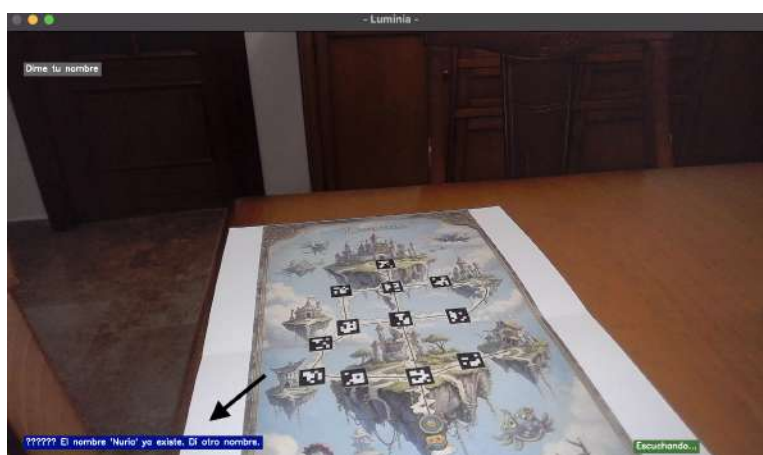


Figura A.5: Fase de registro: nombre existe.

2. Selección de idioma

Comandos válidos:

- “Español”
- “Inglés”

Mensajes del sistema:

- “Idioma no válido. Di ‘español’ o ‘inglés’.”
- Si el registro es exitoso, el usuario pasa al menú principal.

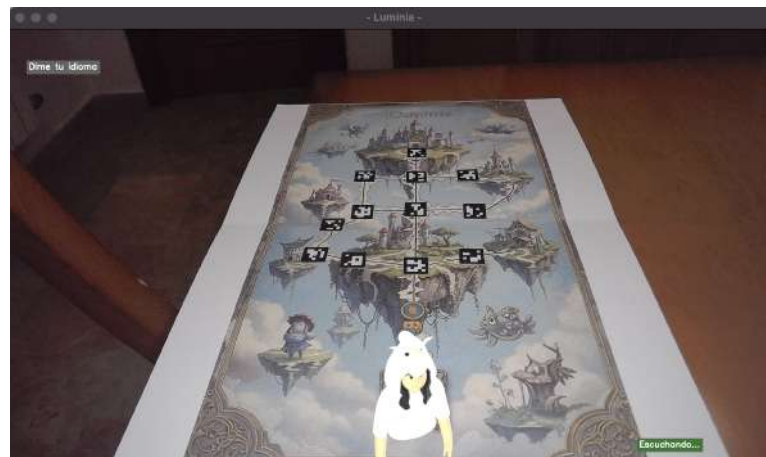


Figura A.6: Fase de registro: introducción del idioma.

A.3.4. Fase: Menú Principal

El usuario puede acceder a distintas secciones del sistema diciendo palabras concretas. Comandos válidos:

- “**perfil**” → abre el menú de perfil del usuario.
- “**progreso**” → muestra los avances.
- “**disfraces**” → abre la tienda e inventario de disfraces.
- “**mundo letras**”, “**mundo animales**”, “**mundo frutas y verduras**”, “**mundo números**”, “**mundo final**” → entra a cada mundo de aprendizaje.
- “**salir**” o “**cerrar**” → cierra la aplicación.



Figura A.7: Menú principal: navegación por voz entre mundos y opciones.

A.3.5. Fase: Perfil

Permite modificar el nombre e idioma del usuario. Comandos válidos:

- **“nombre nuevo_nombre”** → cambia el nombre registrado.
- **“idioma nuevo_idioma”** → cambia el idioma preferido (“español” o “inglés”).
- **“atrás”** o **“volver”** → regresa al menú principal.

Mensajes del sistema:

- “Nombre actualizado a nuevo_nombre”
- “Idioma actualizado a nuevo_idioma”

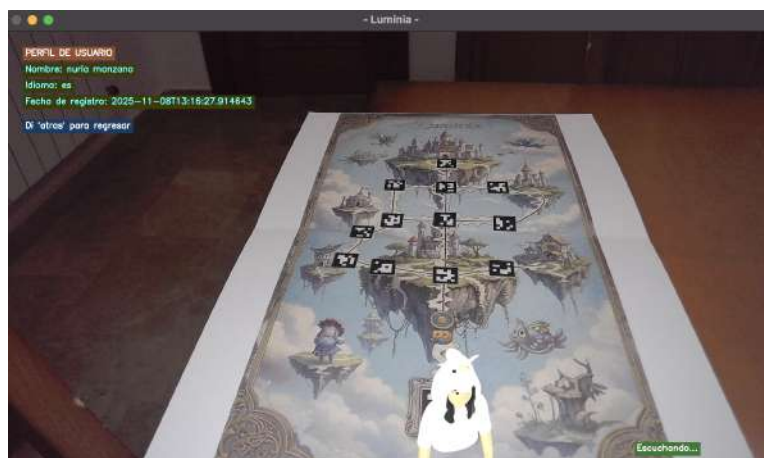


Figura A.8: Gestión del perfil.

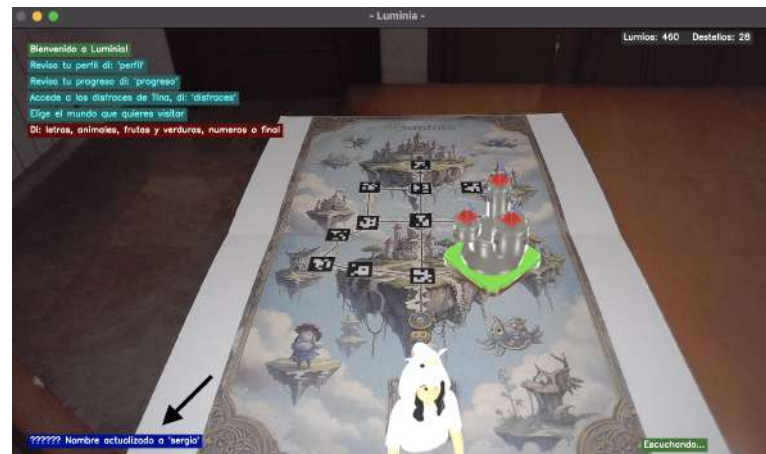


Figura A.9: Ejemplo actualización nombre usuario.

A.3.6. Fase: Progreso

El usuario puede revisar su progreso actual.

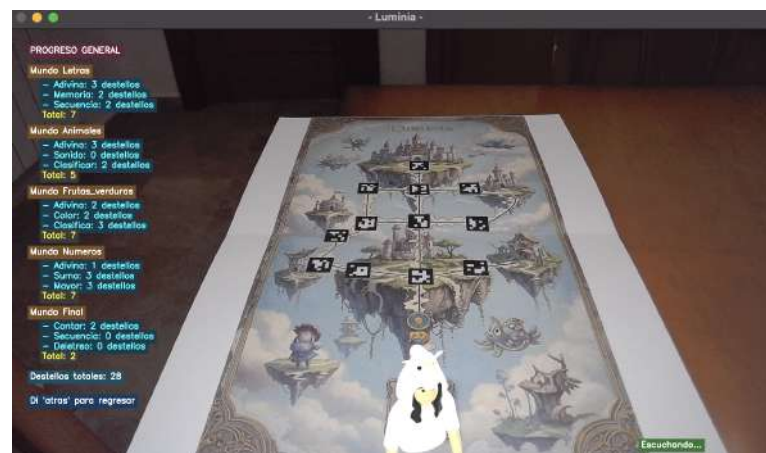


Figura A.10: Visualización del progreso de un usuario.

A.3.7. Fase: Disfraces

El usuario puede comprar o equipar disfraces desbloqueados.

- “comprar nombre del disfraz”
- “equipar nombre del disfraz”
- “atrás” o “volver” → regresa al menú principal.

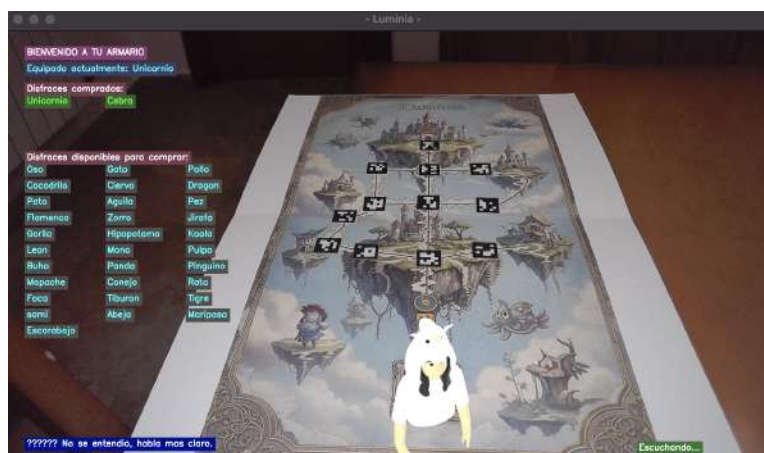


Figura A.11: Pantalla de disfraces.

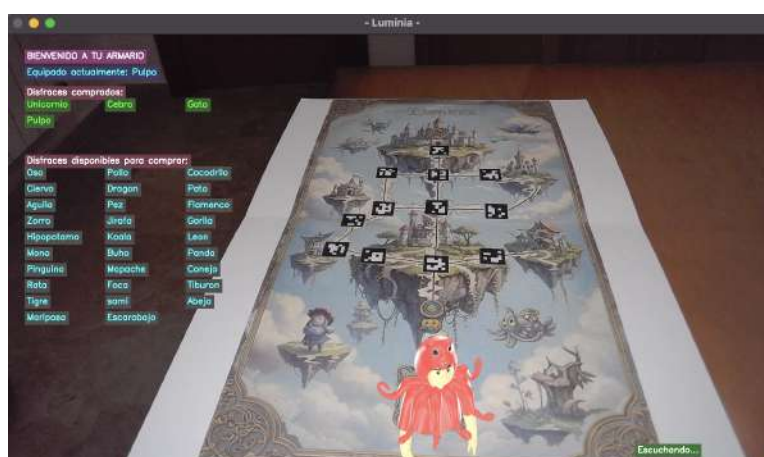


Figura A.12: Muestra de la compra y equipamiento de un disfraz.

A.3.8. Fase: Mundos de Juego

En los mundos temáticos (letras, animales, frutas, números, etc.) el usuario puede iniciar minijuegos mediante comandos:

- “adivina”, “memoria”, “secuencia”, “contar”, “deletreo”, “suma”, etc.
- “atrás” o “volver” → regresa al menú principal.

Muestro algunos ejemplos de la vista al entrar en algunos mundos educativos. Todos siguen la misma vista. Se muestra a Tina y el castillo del mundo seleccionado; además, en la interfaz aparecen los nombres de los minijuegos disponibles.

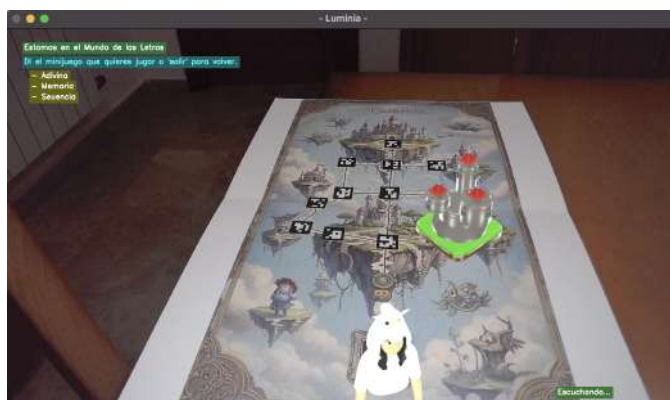


Figura A.13: Ejemplo del mundo de las letras con comandos de minijuego.

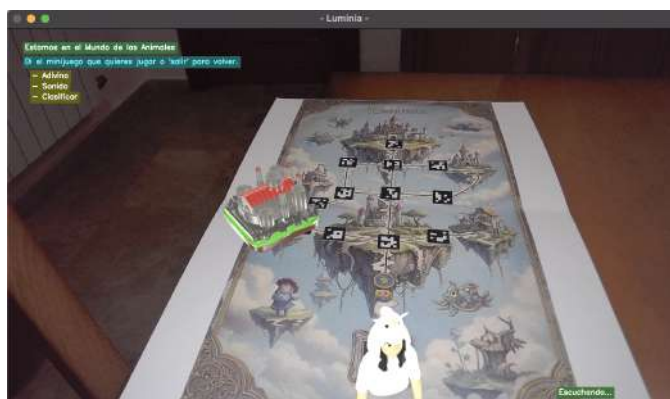


Figura A.14: Ejemplo del mundo de los animales con comandos de minijuego.

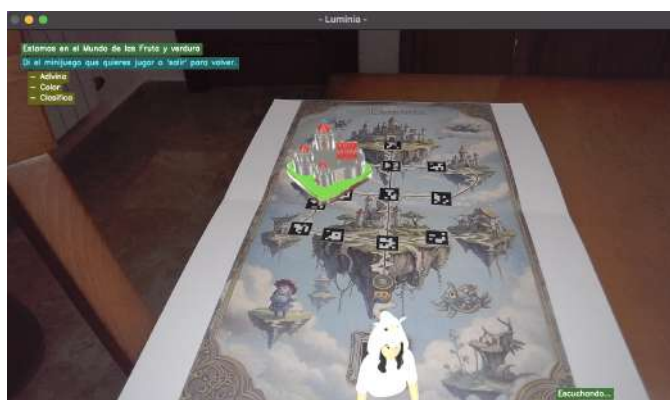


Figura A.15: Ejemplo del mundo de las frutas y verduras con comandos de minijuego.

A.3.9. Fase: Jugando

Durante un minijuego activo, los comandos se interpretan según las reglas del propio juego. Comando universal:

- “atrás” o “volver” → termina el juego actual y retorna al menú principal.

Muestro ejemplos de diferentes minijuegos para visualizar la variedad de ellos.

Juego adivina del mundo de las letras

En este minijuego, se muestran letras sobre el tablero y el usuario debe identificarlas pronunciándolas en voz alta. Por ejemplo, en la imagen presentada, la letra que corresponde decir sería la “T”, con el comando “letras T”.

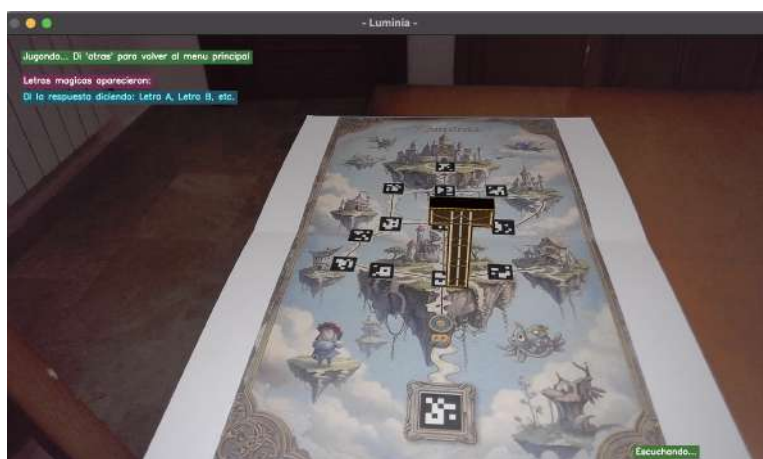


Figura A.16: Ejemplo de interacción durante el minijuego adivina del mundo de las letras.

Juego clasificar del mundo de los animales

En este minijuego, se presentan diferentes animales sobre el tablero y el usuario debe responder de acuerdo con la pregunta formulada. Existen tres tipos de preguntas: “¿Qué animal vive en la tierra?”, “¿Qué animal vive en el agua?” y “¿Qué animal puede volar?”. El usuario debe decir en voz alta el nombre del animal correspondiente; por ejemplo, en la situación que se muestra, la respuesta correcta sería “gato”.

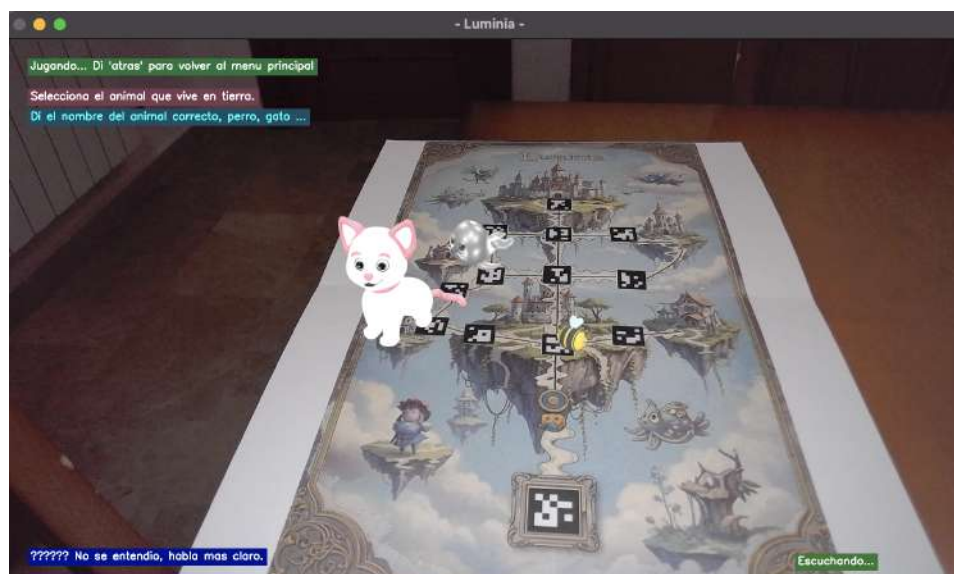


Figura A.17: Ejemplo de interacción durante el minijuego clasificar del mundo de los animales.

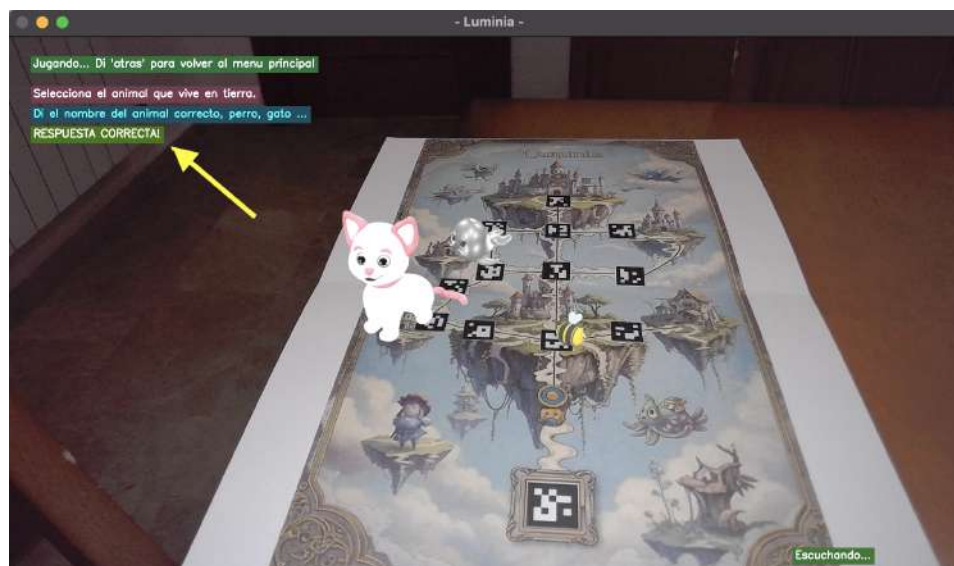


Figura A.18: Ejemplo de respuesta correcta durante un minijuego.

Juego adivina del mundo de las frutas y verduras

En este minijuego, se muestran frutas o verduras sobre el tablero y el usuario debe identificarlas pronunciándolas en voz alta. Por ejemplo, en la imagen presentada, la respuesta que corresponde decir sería “brócoli”.

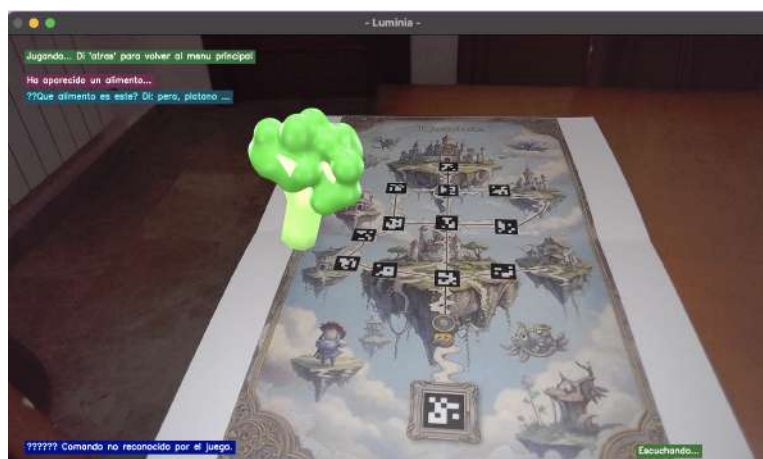


Figura A.19: Ejemplo de interacción durante el minijuego adivina del mundo de las frutas y verduras.

Juego adivina del mundo de los números

En este minijuego, se muestran números sobre el tablero y el usuario debe identificarlos pronunciándolos en voz alta. Por ejemplo, en la imagen presentada, la respuesta que corresponde decir sería “45”.

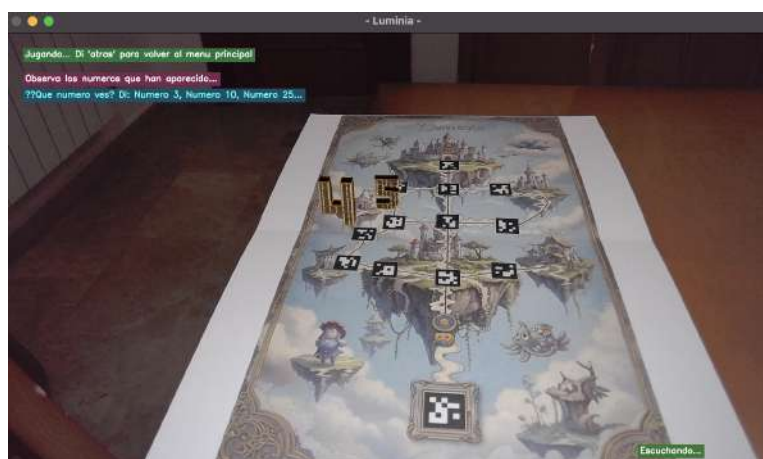


Figura A.20: Ejemplo de interacción durante el minijuego adivina del mundo de los números.

Juego contar del mundo final

En este minijuego, se muestran diferentes modelos sobre el tablero, que pueden pertenecer a cualquier temática educativa. El usuario debe contar cuántos objetos específicos aparecen de acuerdo con la pregunta planteada. Por ejemplo, en la imagen presentada, la pregunta es “¿Cuántos pájaros hay?”, la respuesta correcta sería “hay uno”.

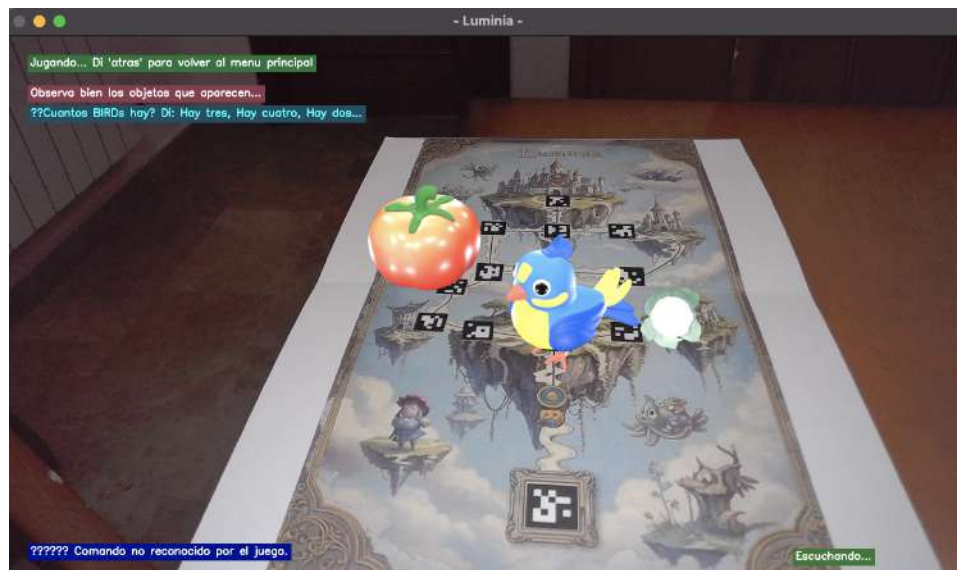


Figura A.21: Ejemplo de interacción durante el minijuego contar del mundo de final.

A.3.10. Mensajes de error comunes

- “No se entendió, habla más claro.” → Error de reconocimiento de voz.
- “Comando no reconocido por el juego.” → Error de reconocimiento de voz mientras se está jugando.
- “Error de conexión con la base de datos. Intenta de nuevo.” → Fallo con MongoDB.
- “Error inesperado en reconocimiento de voz.” → Problema interno del sistema.

Se muestran algunos de los mensajes de error más comunes:

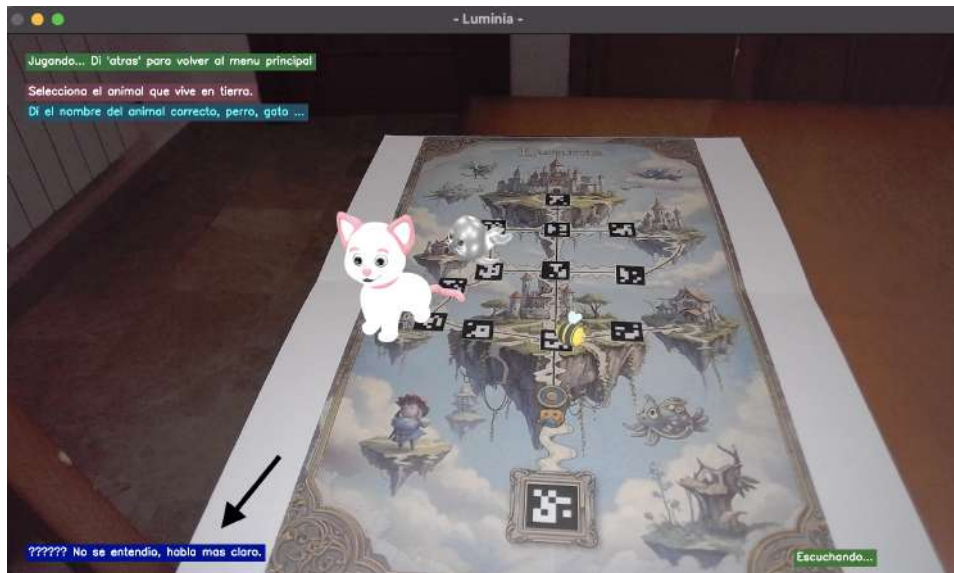


Figura A.22: Ejemplo de mensaje de error de reconocimiento de voz.

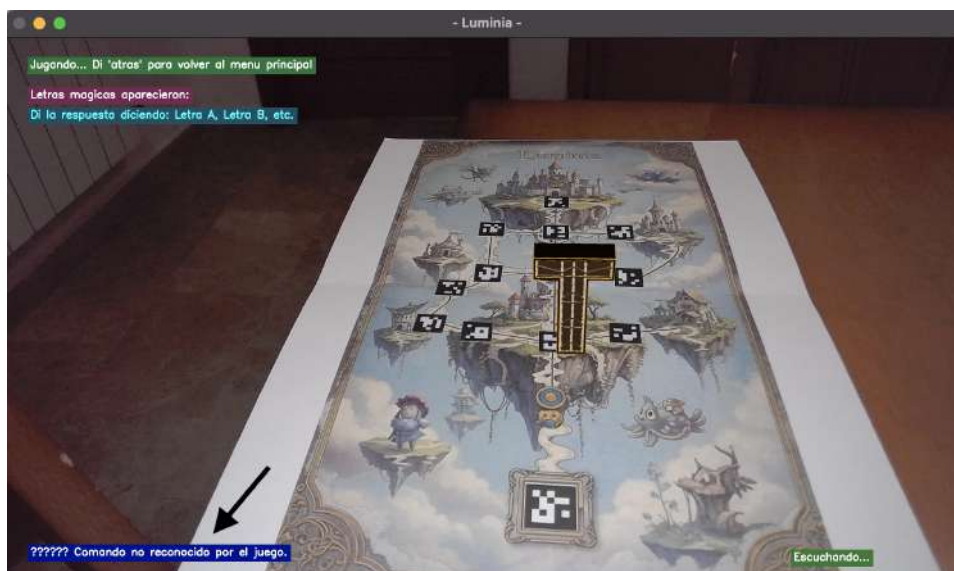


Figura A.23: Ejemplo de mensaje de error mientras se está jugando.

